

4 第四章 切分世界：分布式系统

第三章把客户端连接从双人推进到单机多人同图：服务器能够同时维护多条连接，按固定节奏处理请求，并通过同步机制保护共享状态。连接池、对象池、缓存分层和压测指标进一步说明了一台服务器内部的资源使用方式。至此，单机版本已经具备了承载多用户在线负载的基本结构，但所有连接、计算、缓存访问和持久化写入仍然汇聚到同一台机器上。

当用户规模继续扩大，单机优化会逐渐遇到无法绕开的资源上限。在线用户数和请求到达率继续增加时，单台机器的 CPU 要处理更多请求计算和消息广播，网络要发送更多状态同步消息，数据库和缓存也要承受更高频的查询与写入。细化锁粒度、减少对象分配、调整连接池容量等方式具有一定价值，但这些优化只能改善单台机器内部的效率，不能让单台机器拥有更多物理资源。

提高系统容量有两条基本路径：一条是为现有服务器增加处理器、内存或网络带宽，这称为纵向扩展；另一条是增加服务器，让多台机器共同承担负载，这称为横向扩展。本章所说的多机扩展主要指横向扩展：把原来集中在一台机器上的连接、请求计算和数据访问分配给多个节点处理，使新增节点能够承担一部分负载。换一种视角看，单机优化是在做除法，即通过更精细的资源管理压缩每次操作的开销；多机扩展则是在做乘法，即通过增加节点叠加系统可用的资源总量。这种叠加并不保证节点数增加一倍，系统容量就一定增加一倍；跨节点通信、协调开销和负载不均都会降低实际扩展收益。

单机架构中只有一条从客户端到应用服务器再到数据库的固定路径：请求由哪台服务器处理、数据保存在哪里、一次操作由哪个进程裁决，都由这条路径确定。当连接、计算和数据访问分散到多个节点后，这些默认关系随之消失。一条请求进入系统后，入口必须先从多个实例中选择处理者；操作跨越多个节点时，参与节点必须形成一个可确认的结果；已经承担处理任务的节点失效时，系统还要选择接管者，并判断此前哪些操作已经生效。增加节点由此把单机中由固定路径确定的处理关系，转化为请求分配、结果确认和故障接管三类显式决策。图 31 通过单机与多机请求路径的对照，说明这三类决策为什么会随横向扩展一同出现。

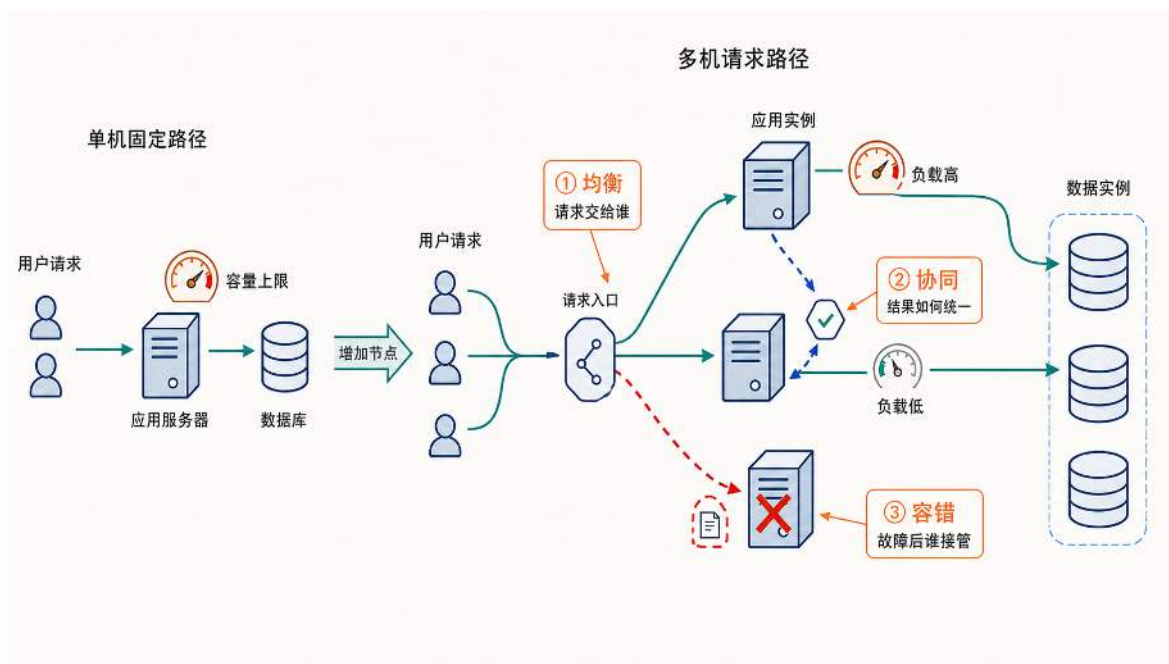


图 31: 横向扩展将单机中的固定处理关系转化为三类显式决策：入口分配请求，节点协同确认结果，故障后重新接管服务。

这三类决策分别形成多机系统的三个设计维度。**均衡**关注如何把请求、数据和计算对象分配到合适的实例，请求分配和数据拆分都属于这类问题。**协同**关注多个实例如何共同完成一次操作，并满足该操作所需的一致性要求，跨节点裁决是其中的典型场景。**容错**关注节点故障后如何恢复服务，并明确哪些操作已经生效。只有建立相应的分配、协同和接管规则，新增节点提供的资源才能被请求实际利用，并在跨节点操作和节点故障发生时维持正确服务。

本章先区分多机扩展中的两种基本拆分方式，再沿均衡、协同和容错三个维度展开相应机制及其取舍。完成本章后，读者应能判断对象应如何分配，多实例应如何共同完成操作，以及系统应如何在节点故障后恢复服务。

4.1 多机扩展的两个拆分维度：职责与对象

从单机进入多机后，新增机器需要承接原有系统中的一部分工作。设计者必须回答两类问题：哪些功能应由不同服务负责，同一类服务中的用户、会话、计算任务或数据记录又应由哪个实例负责。前者是按职责拆分，后者是按对象拆分。它们不是相互替代的两种方案：系统通常先按职责分开功能，再在每类职责内部按对象分摊容量。

按职责拆分先确定每类服务承担什么功能。单机阶段，连接接入、状态推进、消息路由、任务排队和数据读写可以放在同一个进程中完成；规模扩大后，这些功能的资源需求、故障影响和迭代节奏逐渐分化。系统可以据此拆出维护连接和转发消息的网关服务、处理请求和推进状态的业务逻辑服务，以及负责缓存访问和持久化写入的数据服务。职责分开后，各类服务可以独立演进，某一类服务的故障也更容易被限制在相应的职责范围内。

但是，按职责拆分只改变了功能由哪一类服务承担。如果每类服务仍然只有一个实例，网关仍无法维护不断增长的连接，逻辑服务仍无法处理所有房间或任务，数据服务也仍无法保存和访问全部用户记录。为了继续扩大容量，系统还要在每类职责内部部署多个同类实例，并让不同实例分别负责一部分对象，这就是按对象拆分。网关按连接、会话或入口区域做**连接分片**，计算服务按房间 ID、任务 ID 或游戏实例 ID 做**计算分片**，数据库按用户 ID、组织 ID 或区域 ID 做**存储分片**。三类分片共同解决单一服务实例的容量上限。

一次用户请求可以同时经过这两个拆分维度。系统先根据职责把请求交给网关服务，再根据连接归属选择一个网关实例；网关把请求转交给业务逻辑服务后，还要根据房间、任务或游戏实例的归属找到具体的计算实例；读取用户状态时，又要根据用户 ID 找到相应的数据分片。按职责拆分决定请求需要经过哪些服务，按对象拆分决定请求在每类服务中落到哪个实例。

本章后续首先沿按对象拆开展开，因为同类实例增加后，系统必须确定对象归属、实例映射、请求路由和动态调整规则，这些问题构成下一节讨论的均衡机制。与此同时，按职责拆分把单体程序中的函数调用变成了跨服务网络调用，按对象拆分又使一次操作可能跨越多个实例；由此产生的超时、部分失败、结果确认和故障恢复问题，将在后面的协同与容错部分继续讨论。

4.2 均衡：分片与动态调整

多机扩展中的**均衡**，是把请求、数据和计算负载分散到多个实例，避免少数实例过载而其他实例长期空闲，使集群中的资源得到有效利用。均衡并不要求每个实例承受完全相同的压力；当机器容量、对象访问频率和处理成本不同时，更重要的是让各实例的负载保持在可承受范围内，并避免长期热点。

增加实例本身不会自动形成均衡。系统必须建立对象到实例的分配规则，使连接、计算任务和数据能够分散到合适的位置，这种按对象划分负载的方法称为**分片**。分片决定初始归属；当热点、实例数量或节点状态发生变化时，系统还需要根据监测结果动态调整归属，才能在运行过程中继续维持均衡。

在本章讨论的在线系统中，对象分片主要出现在三个位置：网关层分配连接和会话，形成**连接分片**；逻辑服务层分配房间、任务或游戏实例，形成**计算分片**；存储层分配用户或组织数据，形成**存储分片**。图 32 将三类分片各自处理的对象、常用分片键和目标实例放在同一条请求路径中，并给出它们共享的处理机制。

本节先从三类分片出发，讨论分片键以及范围分片、哈希分片和一致性哈希等映射策略；再讨论分片路由、负载监测和动态调整，说明请求如何找到目标实例，以及系统如何应对热点与扩容；最后进入存储层和缓存层，分析跨分片查询、跨分片事务与集群化带来的具体问题。

4.2.1 分片键与基本映射策略

选择分片键：划分对象的依据 把负载分散到多台机器之前，系统要先确定哪些对象应当保持共同归属。同一会话的后续消息需要回到保存会话状态的网关；同一房间的参与者需要由一个逻辑服务实例共同推进状态；同一用户的资料、配置和业务状态也通常需要存放在同一个数据分

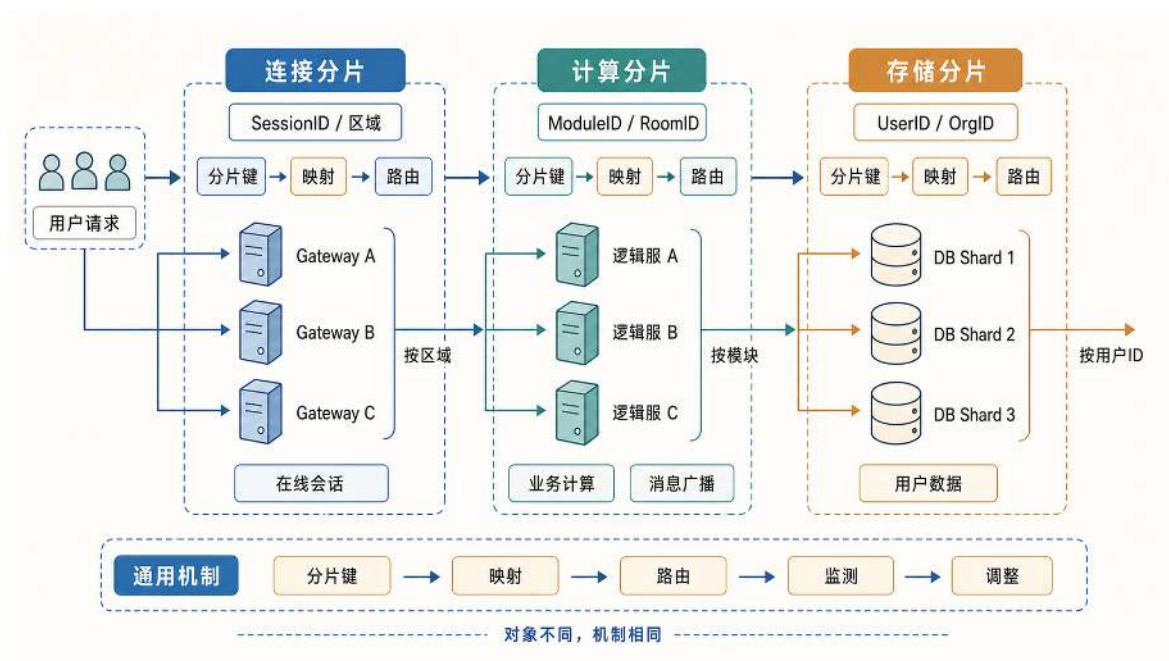


图 32: 连接、计算和存储分片处理不同对象，但都通过分片键、映射和路由建立对象归属，并通过监测和调整维持运行时均衡。

片。系统需要从请求中取得一个稳定标识，把这些相关对象归入同一组。

这个用于表示对象归属的标识称为分片键 (Sharding Key)。连接分片通常使用会话 ID、连接 ID 或接入区域，计算分片通常使用地图 ID、房间 ID 或游戏实例 ID，存储分片通常使用用户 ID 或组织 ID。只要分片键保持不变，不同服务器在不同时间处理同一对象时，就能依据相同标识查找它的归属。

一个合适的分片键首先要避免负载集中。如果交易记录按日期分片，当天产生的新记录会全部写入最新分片；当常见访问围绕用户展开时，改用分布较散的用户 ID，写入更容易落到多个分片。不过，字段值分散并不意味着实际压力一定均匀，少数高频用户或大型组织仍可能形成热点。

分片键要尽量保留访问局部性。用户资料若按物品类型拆分，查询一个用户的全部数据就要访问多个分片；按用户 ID 拆分，则可以把这次查询限制在一个分片。实时计算也有相同要求：以房间 ID 为键，可以让需要共同推进状态的参与者留在同一计算实例，减少跨节点同步。

负载均匀性和访问局部性有时会相互制约。划分过细可以把负载分得更散，却可能拆开需要共同处理的对象；划分过粗可以减少跨分片访问，却容易让大对象形成热点。确定分片键，就是在这两项要求之间选择适合当前访问模式的归属单位。

分片键只确定哪些对象属于同一组，还没有指定这一组由哪个实例负责。下面以用户 ID 和数据库实例为例，比较范围分片和哈希分片两种基本映射策略。

范围分片：按连续区间分配实例 范围分片 (Range-based Sharding) 按照分片键的数值或字典序区间分配实例。假设有三个数据库实例，可以按 ID 数值范围分配：ID 从 1 到 1,000,000 的用户存放在第一个数据库 (分片 0)，ID 从 1,000,001 到 2,000,000 的存放在第二个数据库 (分

片 1)，ID 从 2,000,001 以上的存放在第三个数据库（分片 2）。这种方式的优点在于规则简单明了，容易理解和管理。当需要查询某个用户的数据时，只需要简单地判断 ID 落在哪个区间即可。扩容时也可以把新的 ID 范围交给新分片，未来注册的用户会自然流向新机器。

但范围分片有一个明显隐患：负载热点。如果用户注册在时间上非常集中，比如系统刚上线或举办大型活动期间，短时间内会有大量新用户涌入，他们会被连续分配到相邻的 ID 区间。假设一周内新注册了 50 万用户，ID 都落在 2,000,001 到 2,500,000 这个范围内，全部被分配到分片 2。而这些新用户往往最活跃，登录、操作、做任务都更频繁，分片 2 的负载就会明显高于其他分片。新建一个分片只能承接后续用户，不能自动减轻这个已经变热的区间。要处理已有热区，还是得把旧区间拆开，把一部分存量数据搬出去。

哈希分片：打散连续键的分布 为了避免连续区间形成热点，可以使用哈希分片（Hash-based Sharding）。具体做法是：对每个用户 ID 执行一个固定的哈希运算，将结果对数据库实例数量取模，根据余数来决定数据存放的位置。用伪代码表示如下：

哈希分片的映射计算

```
1 // 假设有4个数据库分片
2 const numShards = 4
3
4 // 根据用户ID计算目标分片
5 func getShardID(userID int) int {
6     // 教学示例：为便于观察取模效果，这里直接用 userID 数值代替其哈希结果；
7     // 真实系统中应先经过 MurmurHash 等确定性哈希函数。
8     hash := userID
9     // 对分片数量取模，得到分片编号（0-3）
10    shardID := hash % numShards
11    return shardID
12 }
13
14 // 示例：不同用户ID被分配到不同分片
15 // 用户1 -> 分片1，用户2 -> 分片2，用户3 -> 分片3
16 // 用户4 -> 分片0，用户5 -> 分片1，用户6 -> 分片2
17 // 即使ID连续，也会被打散到不同分片
```

由于哈希函数对不同输入产生的输出在数值空间上分布较为均匀，即使用户 ID 在时间上连续注册（1, 2, 3, 4...），经过哈希和取模之后也会被打散到不同的数据库实例中。哈希函数在这里的作用，是把用户 ID 转成一个稳定且分布较均匀的整数结果。真实系统通常会把用户 ID 转成固定格式的字节串，再交给一个确定性的哈希函数，例如 MurmurHash、xxHash 或 FNV。只要输入用户 ID 相同，任何一台应用服务器、任何一次进程重启后都应得到相同哈希值。随后系统再用这个哈希值对分片数量取模，得到目标分片。上面的代码为了便于观察，直接用用户 ID 代

替哈希值；真实路由实现中，确定性哈希函数可以减少 ID 生成规则、批量导入或特殊编号带来的分布偏差。

哈希分片把连续注册的用户打散到不同分片，缓解了范围分片中的新用户热点。但这个规则有一个前提：分片数量保持不变。当系统从 4 个分片扩展到 5 个分片时，路由公式会从 $\text{hash}(\text{userID}) \% 4$ 变成 $\text{hash}(\text{userID}) \% 5$ 。取模基数一变，许多用户的余数都会改变；即使用户 ID 和哈希函数都没有变化，目标分片也可能变成另一台数据库。取模基数变化会直接改写对象与实例之间的映射关系。图 33 以用户 ID 为例，对比从 4 个分片扩展到 5 个分片前后的归属结果。

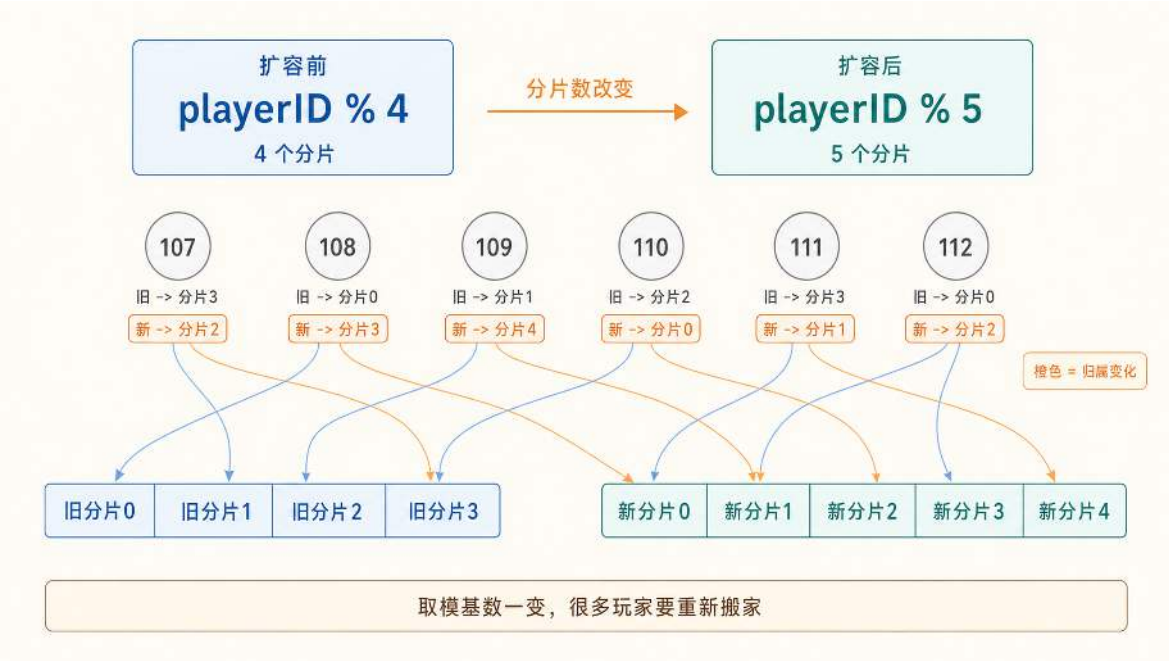


图 33: 以用户 ID 为例，普通哈希分片从 4 个分片扩展到 5 个分片后，取模基数变化导致大量对象重新归属。

图中的用户在扩容后都改变了归属。例如，用户 107 原本在分片 3 ($107 \% 4 = 3$)，扩容后被分到分片 2 ($107 \% 5 = 2$)。这组样本并不表示每个用户都一定迁移，但说明取模基数变化可能使大量用户重新计算目标分片。系统必须重新判断每个用户应该迁到哪里，再搬动大量数据。对于数百万用户的系统，这类迁移耗时长，也容易影响线上服务。

这正是普通哈希分片的主要缺点：对象归属与当前实例数量直接绑定。实例增加、减少或故障退出时，取模基数随之变化，大量对象会被重新映射到其他实例，进而产生大规模数据迁移和服务切换开销。要降低这类开销，映射策略需要把节点变化的影响限制在局部范围，而不是重新计算大部分对象的归属。

4.2.2 一致性哈希：限制节点变化的影响范围

一致性哈希（Consistent Hashing）¹⁹ 的主要优点，是在节点集合变化时保留大部分已有映射关系。新增节点时，只有相邻区间中的部分对象转移到新节点；删除节点时，也只有原来属于该节点的对象需要迁出，其余对象与留存节点之间的归属关系保持不变。因此，一致性哈希能够显著减少扩缩容和节点退出时的迁移范围。

为实现这种局部变化，一致性哈希不再把哈希值直接对节点数量取模，而是把实际数据库节点和用户 ID 映射到同一个首尾相接的哈希空间，再根据对象与节点在空间中的相对位置确定归属。它减少的是节点变化引起的迁移范围，并不自动保证负载均匀；节点在哈希空间中的位置偏斜还需要通过虚拟节点缓解，少量热点键也需要其他机制处理。这里的实际节点也称为物理节点，指真正承担数据读写的数据库实例，并不要求每个实例独占一台物理服务器。最简单的做法是让每个实际节点在哈希空间中只对应一个位置。查询用户数据时，系统计算用户 ID 的哈希值，再沿顺时针方向找到第一个节点位置，并访问该位置对应的数据库实例。但是，少量实际节点经过哈希后未必均匀分布：相邻节点之间的区间可能大小悬殊，负责较大区间的数据库会承载更多数据和请求。

为减小这种偶然偏差，系统为同一个实际节点生成多个分散的代表位置，这些位置称为虚拟节点。例如，db1#0、db1#1 和 db1#2 是实际数据库 db1 在哈希空间中的三个代表位置。用户 ID 命中其中任意一个虚拟节点时，最终访问的仍然是 db1。所有实际节点都拥有多个虚拟节点后，每台数据库负责的区间会被拆成散布在哈希空间中的多段小区间，数据和请求通常能够分布得更均匀。新增实际节点时，只有各新虚拟节点与其逆时针方向前一个节点之间的区间改变归属，其他区间保持不变。

一致性哈希需要同时回答两个问题：正常请求怎样找到负责该对象的节点，节点加入后又有哪些对象必须改变归属。在图 34 中，这两个过程放在同一个哈希环上。数据库节点 A、B、C 先通过多个虚拟节点映射到环上；用户 ID 经过哈希得到环上的数据键位置，再沿顺时针方向找到最近的虚拟节点，并由该虚拟节点确定实际数据库。新增节点 D 时，新的虚拟节点插入原有顺序，只接管各插入位置之前的一小段区间；环上其他区间的顺序和归属保持不变。

¹⁹一致性哈希最早由 Karger 等人在 Web 缓存场景中系统提出，见 David Karger et al., “Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web”, STOC 1997, <https://people.csail.mit.edu/karger/Papers/web.pdf>。

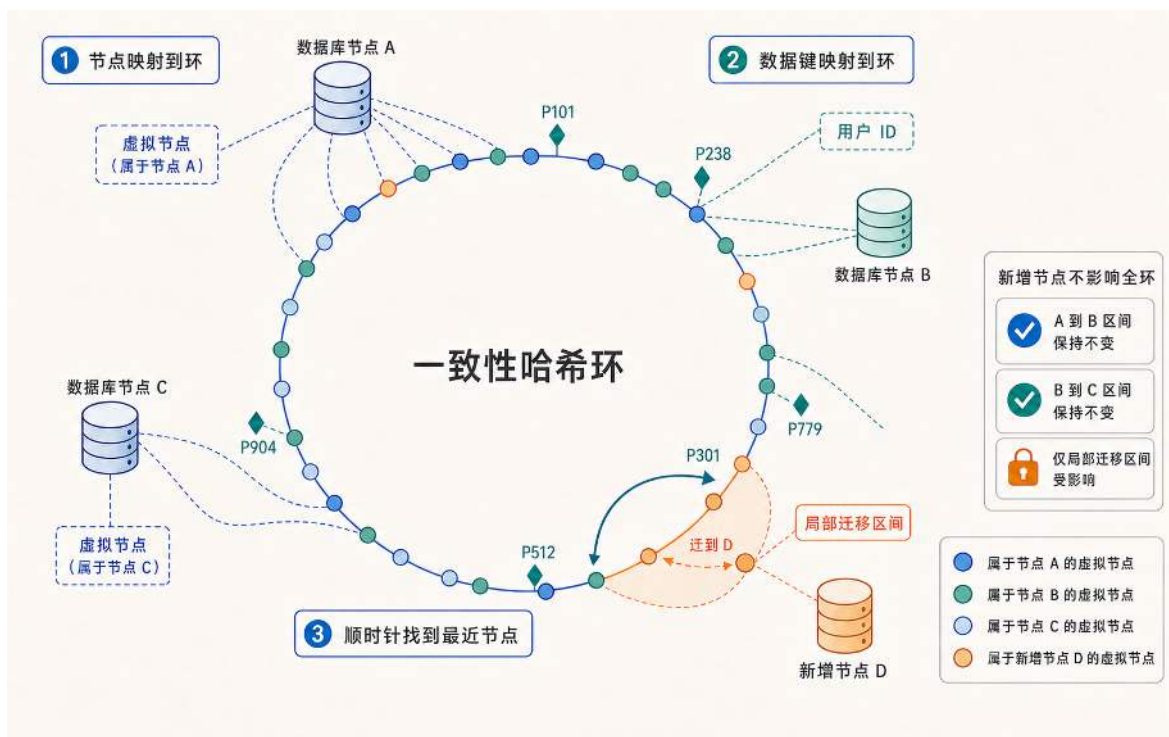


图 34: 物理节点通过多个虚拟节点映射到哈希环，数据键沿顺时针方向找到归属节点；新增节点 D 只接管插入位置相邻的局部区间。

正常查询和扩容迁移都依赖同一个条件：系统必须稳定保存虚拟节点在环上的顺序，以及每个虚拟节点对应的物理节点。圆环只是这种循环顺序的可视化；实际实现通常使用一张按哈希值排序的路由表，而不需要维护圆形数据结构。表中的每一项以虚拟节点在哈希空间中的整数位置为键，以对应的物理节点为值。有了这组有序关系，系统便可以依次完成三个动作：建立哈希环、查询对象归属，以及在节点变化时计算迁移区间。

下面的算法一实现图中的第一步，把每个物理节点展开为多个虚拟节点，并建立可供后续查询的有序路由状态。结构体 `HashRing` 中，`ring` 记录“虚拟节点位置到实际数据库节点”的映射，`sorted` 保存所有虚拟节点位置，并按从小到大排序。`BuildRing` 负责初始化这两组数据，`AddNode` 则把一个实际数据库展开成多个虚拟节点并插入哈希环。

算法一：构建一致性哈希环

```
1 type HashRing struct {
2     ring          map[uint32]string
3     sorted        []uint32
4     virtualNodesPerNode int
5 }
6
7 func BuildRing(nodes []string, virtualNodesPerNode int) *HashRing {
8     r := &HashRing{
9         ring:          make(map[uint32]string),
10        virtualNodesPerNode: virtualNodesPerNode,
11    }
12    for _, node := range nodes {
13        AddNode(r, node)
14    }
15    return r
16 }
17
18 func AddNode(r *HashRing, node string) {
19     for i := 0; i < r.virtualNodesPerNode; i++ {
20         key := hash(fmt.Sprintf("%s#%d", node, i))
21         r.ring[key] = node
22         r.sorted = append(r.sorted, key)
23     }
24     sort.Slice(r.sorted, func(i, j int) bool {
25         return r.sorted[i] < r.sorted[j]
26     })
27 }
```

每个实际节点对应的虚拟节点数 `virtualNodesPerNode` 需要在负载均衡精度和维护成本之间取一个折中。虚拟节点越多，各物理节点在环上的分布越均匀，负载偏差越小；代价是哈希环条目更多，内存占用和二分查找开销略有增加。虚拟节点数没有适用于所有系统的固定推荐值，应根据节点数量、哈希函数、键分布、实际请求偏斜和路由表维护成本进行测量。以 4 个数据库分片、每个分片 150 个虚拟节点为例，哈希环上共有 600 个条目；这个数字适合用来说明路由表规模，但是否足以平滑负载仍要由线上指标验证。

哈希环建立后，算法二实现图中的第二、三步：先把用户 ID 映射为环上的数据键位置，再沿顺时针方向找到负责该位置的虚拟节点。在有序数组 `sorted` 中，这等价于查找第一个不小于数据键位置的虚拟节点；如果查找越过数组末尾，就回到数组开头，以保持环形顺序。最后通过 `ring` 取得该虚拟节点对应的实际数据库。只要节点集合不变，同一个用户 ID 就会得到相同的归

属结果。

算法二：根据哈希环查询对象归属

```
1 func Lookup(r *HashRing, userID string) string {
2     if len(r.ring) == 0 {
3         return ""
4     }
5     h := hash(userID)
6     idx := sort.Search(len(r.sorted), func(i int) bool {
7         return r.sorted[i] >= h
8     })
9     if idx == len(r.sorted) {
10        idx = 0
11    }
12    return r.ring[r.sorted[idx]]
13 }
```

算法二能够正确查询的前提，是数据已经存放在当前哈希环所指定的数据库中。扩容时，如果先调用 `AddNode` 修改正式哈希环，请求就可能被送到尚未接收数据的新节点 `D`，导致查询失败。因此，系统不能先切换归属再搬数据，而要先根据旧环和新节点计算迁移范围。这正对应图中的橙色局部区间，算法三把它转化为迁移计划。

新增物理节点 `D` 会在环上产生若干新虚拟节点。对每一个新虚拟节点 v_{new} ，设它的位置为 p_{new} ，它在新环中的逆时针前驱位置为 p_{prev} ，那么只有原本落在区间 $(p_{prev}, p_{new}]$ 内的数据改由 `D` 负责。算法三的任务不是直接复制数据，而是找出这些区间原来所属的节点，并输出“来源节点、目标节点、区间起点和区间终点”，供后台迁移任务执行。

在哈希分布足够均匀、虚拟节点数量足够多时，整体迁移量的期望值大约是全部数据的 $\frac{1}{N+1}$ ，其中 N 是原有物理节点数。如果原来有 4 个节点，新增第 5 个节点大约只需要迁移 20% 的数据。普通取模哈希在同样场景下会让大量用户重新定位，迁移量通常更高。

为生成这份计划，算法必须同时保留扩容前后的两种节点顺序。旧环 `r.sorted` 用于判断数据当前属于哪个节点；算法复制这组位置得到 `finalSorted`，再把新实际节点对应的虚拟节点位置记录到 `newKeys`，并加入 `finalSorted` 重新排序，由此得到扩容后的节点顺序。对于每个新虚拟节点，它在新环中的前驱与自身位置共同确定需要迁移的区间；同一位置在旧环中对应的节点，则是这段数据的来源节点。算法将来源节点、目标节点和区间端点组合成 `MigrateRange`，供后续迁移任务逐段搬运数据。

算法三：新增节点时计算需要迁移的数据范围

```
1 type MigrateRange struct {
2     From      string
3     To        string
4     StartKey  uint32
5     EndKey    uint32
6 }
7
8 func MigrateRanges(r *HashRing, newNode string) []MigrateRange {
9     // 迁移计划基于旧环计算；finalSorted 只用于推演加入新节点后的区间
10    finalSorted := append([]uint32(nil), r.sorted...)
11    newKeys := make([]uint32, 0, r.virtualNodesPerNode)
12    seenNew := make(map[uint32]bool)
13
14    for i := 0; i < r.virtualNodesPerNode; i++ {
15        newKey := hash(fmt.Sprintf("%s#%d", newNode, i))
16        if _, exists := r.ring[newKey]; exists || seenNew[newKey] {
17            continue // 教学代码中直接跳过哈希碰撞
18        }
19        seenNew[newKey] = true
20        newKeys = append(newKeys, newKey)
21        finalSorted = append(finalSorted, newKey)
22    }
23    sort.Slice(finalSorted, func(i, j int) bool {
24        return finalSorted[i] < finalSorted[j]
25    })
26
27    var ranges []MigrateRange
28    for _, newKey := range newKeys {
29        // 前驱必须在完整新环中查找，避免多个新虚拟节点产生重叠区间
30        finalIdx := sort.Search(len(finalSorted), func(j int) bool {
31            return finalSorted[j] >= newKey
32        })
33        prevIdx := (finalIdx - 1 + len(finalSorted)) % len(finalSorted)
34        prevKey := finalSorted[prevIdx]
35
36        // 数据在扩容前属于旧环中顺时针方向的第一个节点
37        oldIdx := sort.Search(len(r.sorted), func(j int) bool {
38            return r.sorted[j] >= newKey
39        })
40        srcNode := r.ring[r.sorted[oldIdx%len(r.sorted)]]
41        ranges = append(ranges, MigrateRange{
42            From:      srcNode,
```

当多个新虚拟节点落在同一个旧区间内时，它们会按照新环顺序形成互不重叠的小区间，扩容时可以分批迁移。

迁移区间确定后，线上扩容还需要把这些区间转化为可执行的迁移流程。比较稳妥的做法是先计算迁移范围，再由后台任务分批搬迁存量数据；搬迁期间继续记录源分片上的增量写入，等存量搬完后追平增量；随后校验两边数据，再把这一小段哈希区间的路由切到新节点。切换后通常还会保留一段回滚窗口，确认没有异常再清理旧数据。这种流程增加了迁移步骤，但可以把停机时间和故障影响控制在更小范围内。

三个算法由此覆盖哈希环从建立、使用到扩容的完整过程。系统初始化时，算法一建立虚拟节点的有序状态；节点集合稳定时，算法二根据该状态查询每个对象的归属；新增节点时，算法三先基于旧环推演新环并生成迁移计划，系统完成数据搬迁和校验后，再由算法一把新节点加入正式哈希环。正式环更新后，算法二才会把相应对象的后续请求指向新节点。

到这里，分片策略已经形成了一条完整逻辑判断链：分片键决定用哪个业务字段表示对象归属，范围分片、哈希分片和一致性哈希决定这个字段如何映射到节点；节点变化时，一致性哈希还可以限制并计算需要迁移的区间。算法二虽然能够算出目标节点，但它返回的只是节点标识，还没有建立网络连接或转发请求。因此，以上内容回答的是“对象的归属”，下一步还要回答“请求怎样真正到达”。

4.2.3 分片路由

分片策略和一致性哈希确定了对象归属以及节点变化时的迁移范围，但一次请求进入系统后，还要把算法二得到的节点标识转化为实际请求路径。分片路由负责完成这一过程，负载监测则为后续动态调整提供输入。在存储层，这意味着从用户 ID 找到目标数据库连接；在计算层，这意味着从模块 ID 找到目标逻辑服地址；在连接层，这意味着从会话标识找到目标网关入口。三种场景的路由实现细节不同，但结构相似：系统需要维护能够从分片键确定目标实例的路由规则与元数据，并在扩缩容时一致地更新这些信息。它可以是范围表、槽位表或一致性哈希环，并不意味着为每个业务键保存一条独立记录。

无论分片键是用户 ID、模块 ID 还是会话标识，路由过程都包含三个环节：从请求中提取分片键，根据路由元数据确定对象归属，再把请求转发到对应的数据库分片、逻辑服或网关实例（见图 35）。

完成分片键到目标实例的映射后，路由逻辑可以放在应用进程内，也可以集中到代理或协调节点中。两种方式都在维护同一件事，即分片键到目标实例的运行时映射。

下面以存储分片路由为例，说明路由层的两种常见放置方式。第一种是应用内路由：路由逻辑直接放在应用服务器进程中。应用服务器启动时读取分片规则和数据库实例表，建立到各个分片的连接池；处理用户请求时，先根据用户 ID 计算目标分片，再从本地连接池中取出对应数据库连接并发起查询。下面的代码展示的就是这个逻辑：`ShardRouter` 保存分片数量和各分片连接，`getDBConnection` 根据用户 ID 选择连接，`queryUser` 再用这个连接查询用户资料。

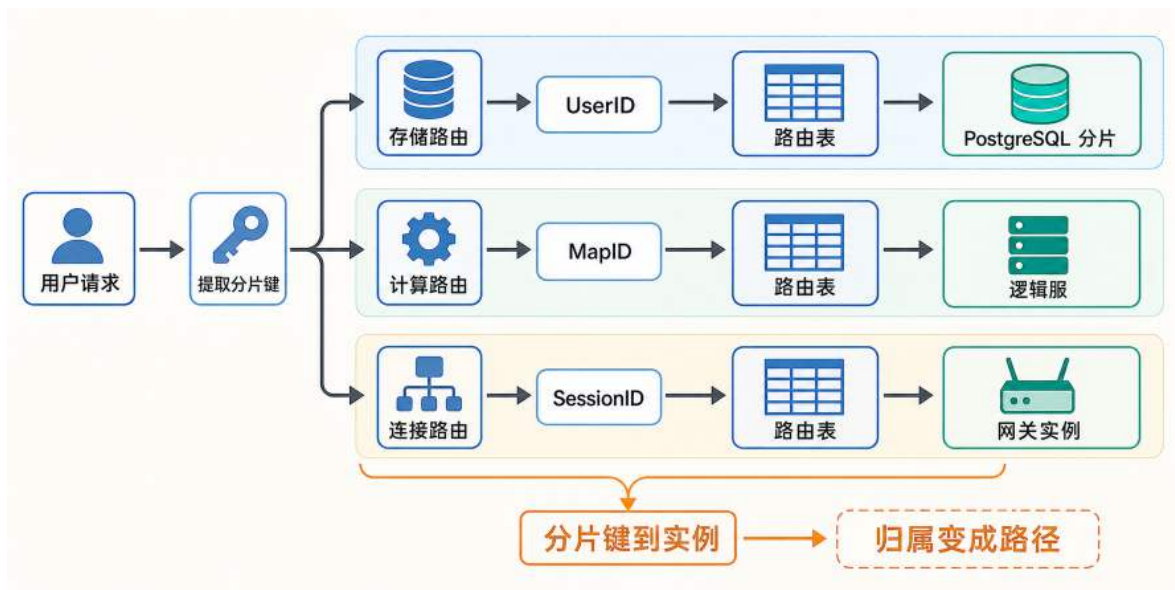


图 35: 分片路由把对象归属转化为请求路径：用户 ID、模块 ID 和会话标识分别经过路由元数据，到达数据库分片、逻辑服或网关实例。

应用层分片路由的实现示例

```

1 // 分片路由器
2 type ShardRouter struct {
3     numShards int
4     dbConnections []*sql.DB // 各分片的数据库连接
5 }
6
7 // 根据用户ID获取目标分片的数据库连接
8 func (r *ShardRouter) getDBConnection(userID int) *sql.DB {
9     shardID := userID % r.numShards
10    return r.dbConnections[shardID]
11 }
12
13 // 查询用户数据
14 func (r *ShardRouter) queryUser(userID int) (*User, error) {
15     // 1. 计算目标分片
16     db := r.getDBConnection(userID)
17
18     // 2. 向目标分片发起查询
19     var user User
20     err := db.QueryRow(
21         "SELECT id, name, power FROM users WHERE id = $1",
22         userID,
23     ).Scan(&user.ID, &user.Name, &user.Power)
24
25     return &user, err
26 }
  
```

应用层路由适合分片规则简单、服务数量不多的场景。应用服务器在本进程内完成分片计算和连接选择，不需要再经过一层代理服务，额外网络开销较少；分片计算本身通常只是一次哈希或取模，CPU 成本也很低。这种方式适合服务数量较少、开发语言统一、分片规则相对稳定的阶段。

它的代价是规则更新会直接影响应用进程。如果新增分片、调整一致性哈希环或切换分片表版本，所有应用服务器都需要及时加载新规则。如果某些进程还停留在旧规则上，同一个用户 ID 就可能被不同服务器路由到不同位置。因此，应用内路由通常还要配合配置中心、版本号和灰度切换机制，保证分片表更新过程可控。

第二种方式是代理中间件路由。当接入数据库的应用进程较多、开发语言不一致或分片规则经常变化时，在每个进程中分别实现路由会增加规则同步和版本管理的成本。代理中间件位于应用服务器与数据库分片之间，把分片映射、连接选择和请求转发集中到统一入口，应用层只需要连接代理。

应用服务器提交 SQL 和参数后，代理从 `player_id` 取得分片键，依据当前分片规则计算唯一的目标分片，再从连接池选择对应连接。若计算结果为分片 2，SQL 只会发送到分片 2，其他分片不参与这次查询；查询结果再经代理返回应用服务器。图 36 将这条请求闭环组织成一条确定路径，说明分片路由先确定数据归属，再沿该归属完成访问，而不是在多个分片之间分配同一条查询。

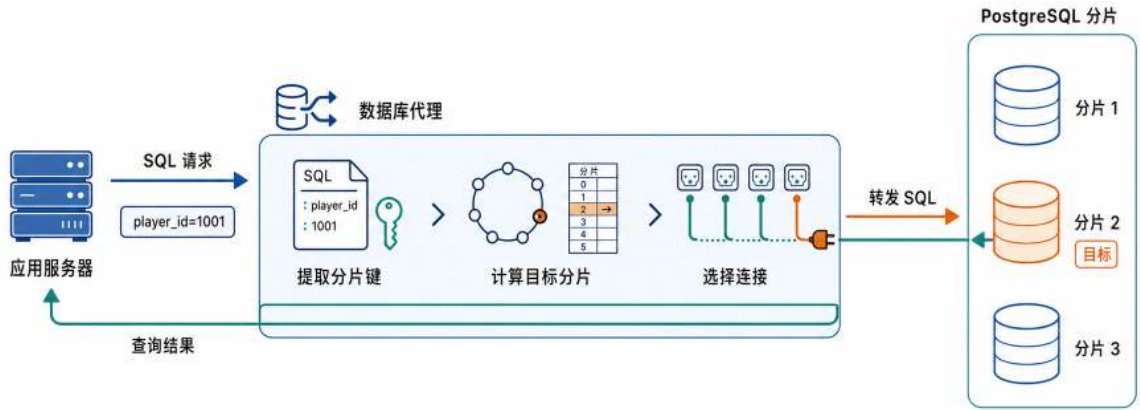


图 36: 数据库代理把分片键计算转化为确定的访问路径：SQL 只进入计算得到的目标 PostgreSQL 分片，查询结果再经代理返回应用服务器。

代理路由由此把应用依赖从多个数据库地址和分片规则收缩为一个稳定入口。分片表更新、节点扩容和连接池管理可以在代理侧集中完成，不必由各应用进程分别实现；在多种语言或多个业务服务共同访问一组分片时，这种集中维护尤其能够减少重复逻辑和规则版本不一致。

它的代价是请求路径多了一跳。每次查询都要先到代理，再由代理转发到目标分片；代理还要解析 SQL、提取分片键、维护连接池，并保证自身高可用。如果代理容量不足，它也可能成为

新的瓶颈。因此，代理中间件适合接入方较多、规则变化频繁、希望集中管理分片规则的系统；应用内路由则更适合路径短、规则稳定、服务规模较小的阶段。

工程中已经有多种成熟的代理中间件、分片网关和协调节点，为应用提供统一的数据库访问入口。它们的具体形态不同，但都把分片元数据和路由判断从业务服务中分离出来，使业务服务不必直接面对一组不断变化的数据库节点。Apache ShardingSphere-Proxy 是透明数据库代理，应用像连接普通数据库一样连接代理，由代理处理分片规则、SQL 路由和结果归并。²⁰ Vitess 面向 MySQL 分片场景，应用连接 VTGate，VTGate 根据 VSchema、表结构和查询条件把请求路由到正确分片。²¹ Citus 面向 PostgreSQL 分布式表，应用连接 coordinator，coordinator 根据分布式表元数据把查询路由到单个 worker 或并行发送到多个 worker。²² 这些系统的差异表明：分片路由不一定只能写在业务代码里，也不一定只是一个简单代理。它可以是透明数据库代理，可以是分片网关，也可以是分布式数据库自己的协调节点。选择哪一种形态，取决于系统希望把分片规则、连接池、SQL 路由和扩容变更交给谁来维护。

前面的数据库代理是分片路由在存储访问中的一种实现。回到整个在线系统，计算层、存储层和路由层承担不同职责：计算层承接玩家连接、会话和业务计算，存储层保存持久化数据，路由层则维护分片键到目标实例的映射，把对象归属转化为实际请求路径。路由层在逻辑上连接计算与存储，使两层无需维持固定的连接关系。

当游戏服务器需要读写持久化数据时，它将玩家 ID 和请求交给分片路由层，由路由层依据当前映射选择数据库分片。同一台游戏服务器因而可以服务数据分布在不同分片上的玩家，同一个数据库分片也可以接收来自多台游戏服务器的请求。两层的访问关系由路由映射建立，不再依赖部署时的一一绑定。

图 37 用两条访问路径说明：游戏服务器发出的请求先经过分片路由层，再到达玩家数据所属的数据库分片。路径 A 和路径 B 从不同的游戏服务器出发，经过共享的分片路由层后，分别进入玩家 ID 所属的数据库分片。请求来自哪台游戏服务器只表示计算入口，玩家 ID 才决定数据的存储位置；两条路径共同说明，计算节点与数据库分片之间不需要保持一一对应。

这种多对多访问关系使两层可以按各自承担的压力独立伸缩：在线用户数量增加时扩展游戏服务器，数据总量或读写压力增加时扩展数据库分片。不过，分片路由只回答请求按照对象归属应当去哪里，不会说明目标实例当前有多忙。系统要识别热点并决定新请求如何分配，还需要持续获得各实例的运行状态。

4.2.4 负载监测与入口调度

分片映射只保证对象归属正确，不保证每个实例上的对象活跃度和运行压力始终相近。对象数量相近的两个分片，可能因为业务活跃度不同而产生完全不同的运行压力。例如，两个计算分片负责的对象数量相同，其中一个分片承载高频交互的计算任务，另一个分片上的对象大多处于

²⁰ Apache ShardingSphere-Proxy 官方文档：<https://shardingsphere.apache.org/document/current/en/quick-start/shardingsphere-proxy-quick-start/>。

²¹ Vitess VTGate 路由说明：<https://vitess.io/docs/faq/advanced-configuration/vtgate/how-does-vtgate-know-which-shard-to-route-a-query/>。

²² Citus 概念文档：https://docs.citusdata.com/en/stable/get_started/concepts.html。

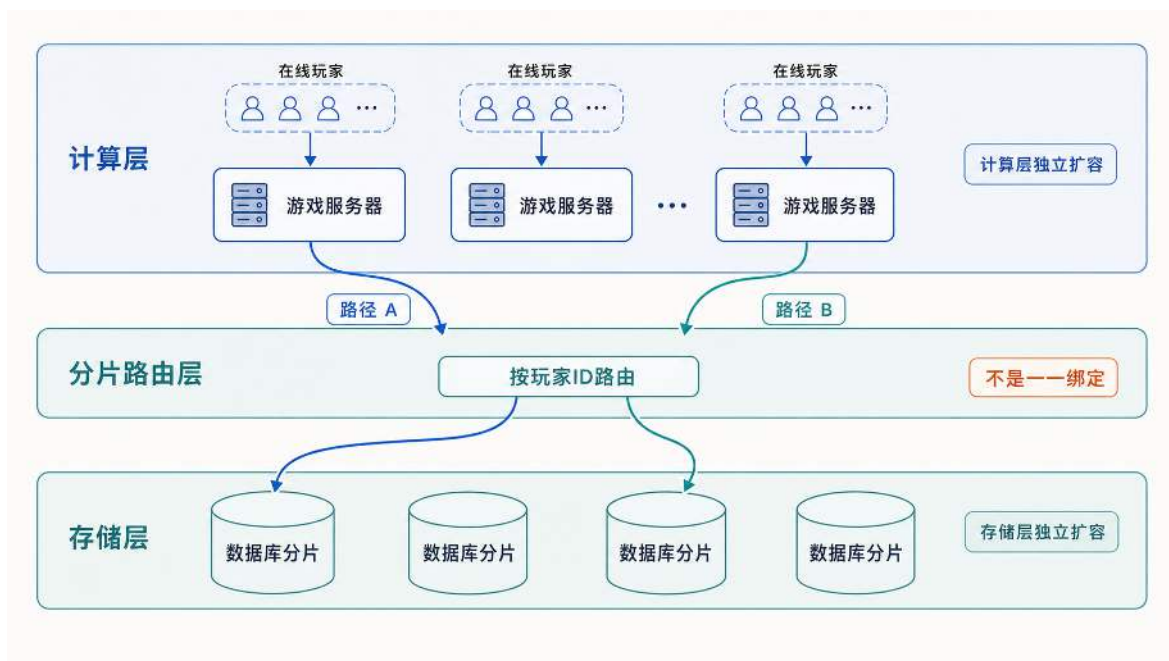


图 37: 计算层与存储层解耦：游戏服务器承接玩家连接，分片路由按玩家 ID 选择数据库分片，使两层形成可独立扩展的多对多访问关系。

空闲等待状态；前者的 CPU 使用率、消息队列长度和响应时间会显著高于后者。存储分片也可能因为少数活跃用户集中而承受更高的查询和写入压力。因此，入口调度和后续动态调整不能只依赖分片映射，还需要通过负载监测持续识别哪些分片过载、哪些分片仍有余量。

负载监测把分片实例的运行状态整理成可比较的数据。最基本的监测是存活状态，即节点是否可达、进程是否正常；进一步可以统计对象规模，例如连接数、在线用户数、模块数或缓存键数量；更细致的压力指标包括 CPU 使用率、内存占用、网络带宽、请求速率、队列长度和响应时间。对象数量描述分片承载了多少任务，压力指标则反映这些任务实际消耗了多少资源。所以监测服务需要同时记录对象规模和运行压力，不能只依赖某一类指标。

状态采集通常采用推拉结合的方式。分片实例通过推送（Push）定期上报心跳和关键指标，监测服务据此维护状态基线；监测服务也可以通过拉取（Pull）周期性探测实例，或在指标异常时发起补充检查。心跳负责持续更新，主动探测用于缩短上报间隔形成的观测盲区。一条通用的分片状态消息可以同时携带对象规模、资源压力和服务质量指标：

分片实例心跳上报的关键数据

```
1 type ShardStatus struct {  
2     ShardID      string  
3     ActiveObjects int    // 连接、用户、模块或键的数量  
4     CPUUsage     float64 // CPU使用率 (0-100)  
5     MemUsage     float64 // 内存使用率 (0-100)  
6     RequestsPerSec float64 // 每秒请求数  
7     P99Latency   int      // P99响应时间 (毫秒)  
8 }  
9 // 每隔固定间隔发送一次心跳到监测服务
```

图 38 把负载监测的维度与感知方式放在同一框架中比较。左侧说明监测信息如何从简单的存活检测逐步丰富到多维资源压力，信息越丰富、调度越准，采集成本也越高；右侧说明服务器主动上报与监测方主动查询如何互补，工程上通常采用推拉结合的方式获取最新状态。

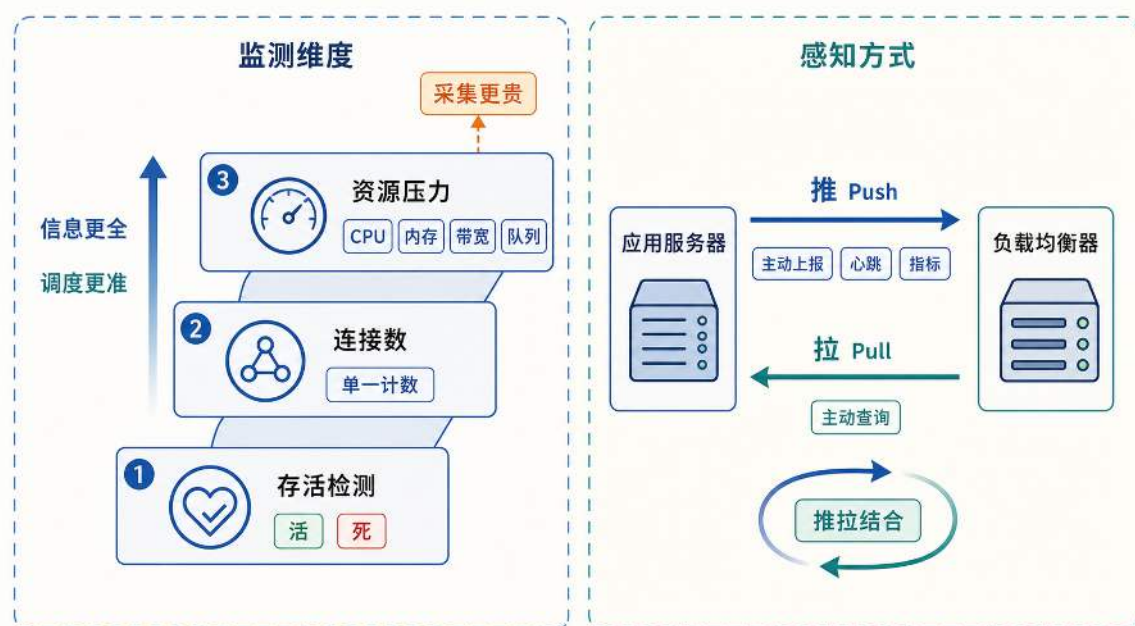


图 38: 负载监测的维度从存活检测到资源压力逐步丰富，感知方式通过推拉结合获取：服务器主动上报心跳和指标，均衡器在需要时主动查询最新状态。

监测服务为每个分片维护最近一次状态快照，并把不同量纲的指标归一化后计算综合负载值。连接分片可以提高连接数和网络带宽的权重，计算分片可以提高 CPU 使用率和队列长度的权重，存储分片则可以提高请求速率和尾延迟的权重。综合负载值用于识别“对象多但压力低”和“对象少但压力高”两类状态，并为过载告警、扩容判断和迁移决策提供输入。

监测结果首先可以用于没有既有会话归属的首次接入。用户第一次进入系统时，尚未与任何应用服务器形成会话归属。负载均衡器可以在健康的应用服务器副本中比较在线人数、CPU 使用率等状态，再选择一个能够承接连接的节点；当这些副本无状态、配置相同且负载接近时，简

单轮询或随机选择也可以成立。

用户接入后，应用服务器可能在内存中保留会话状态。网络连接中断并不意味着这份状态立即消失；客户端重新连接时会携带会话标识，入口层可以据此查找会话归属。如果服务器 C 仍然可用并保存着该会话，重连请求就应优先返回 C，避免在其他服务器上丢失或重新加载状态。这种让后续连接尽量回到原服务器的机制称为**会话粘性**（Session Stickiness）。

上述两个接入场景的节点选择依据并不相同，图 39 将其并列呈现。首次接入依据状态表在可用副本中选择服务器 B；断线重连则依据会话标识找到归属服务器 C，并在 C 仍可用时返回该节点。

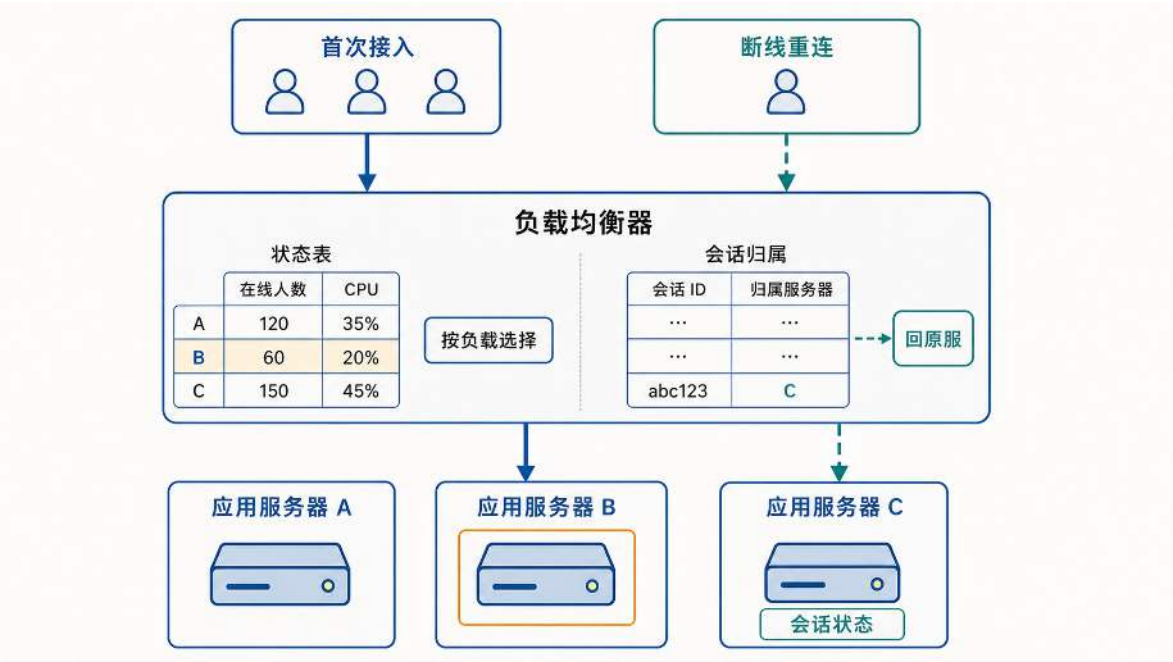


图 39: 同一入口下的两种接入场景：首次接入依据负载状态选择服务器 B，断线重连依据会话归属返回仍保存会话的服务器 C。

会话粘性成立的前提是原服务器仍然可用且会话尚未失效。如果原服务器不可达，入口层需要选择新的承接节点，并从外部状态恢复会话或重新建立会话，不能继续依赖原有归属。

Nginx 是入口负载均衡器的一种实现。对于没有会话归属约束的首次接入，它可以使用加权轮询或最少连接等策略选择上游实例；当业务会话已经具有归属时，入口还需要获得并遵循相应的会话映射。²³

入口调度是在可承接请求的计算节点之间进行选择；存储层仍然按照分片键维持数据归属，不会因为其他数据库分片更空闲而临时改变请求去向。

负载监测本身不会改变对象归属。即使监测服务发现分片 A 过载、分片 B 空闲，路由层仍必须把属于分片 A 的请求送到分片 A；在状态尚未迁移之前，直接改发请求会造成数据或运行状态缺失。因此，发现负载失衡后，系统还必须执行动态调整：整体容量不足时增加实例，容量

²³Nginx Stream 上游模块支持加权轮询、最少连接和哈希等连接分配方式：https://nginx.org/en/docs/stream/nginx_stream_upstream_module.html。

长期过剩时移出实例；总容量足够但对象分布不均时，则重新划分对象归属。无论采用哪种调整，只要分片集合或映射关系发生变化，就需要把受影响的对象状态迁移到新的归属节点。

4.2.5 动态调整：扩缩容与热迁移

动态调整可能由扩缩容、持续的负载不均或实例故障触发，并依次解决两个问题：首先确定哪些对象需要改变归属，其次将这些对象的运行状态转移到新的实例。当调整涉及实例增减时，**一致性哈希**用于限制映射变化的范围，减少需要迁移的对象；确定迁移对象后，**热迁移**负责实际转移状态，并控制迁移过程中的服务中断。前者在上文映射策略中已经给出算法，本节聚焦后者。

热迁移要解决的不只是状态传输，还包括处理权和请求路径的交接。设计算分片 A 因突发流量持续过载，分片 B 仍有余量，路由表当前把会话 u 指向 A。A 的内存中保存着 u 的位置、业务状态、任务进度、计时器、最后处理的请求序号和尚未完成的操作；与此同时，客户端仍在发送会改变这些状态的新请求。如果入口立即把请求改发到 B，B 没有这些状态，无法从 A 停止的位置继续处理；如果 A 在继续处理请求的同时只向 B 发送一次快照，传输完成时这份快照又可能已经落后。因此，迁移必须协调状态复制、处理权转移和路由切换，不能把它们拆成互不相关的操作。

这个例子中的迁移单元是应用层会话，而不是一条正在传输字节的 TCP 连接。系统需要转移的是继续处理会话所需的业务状态、请求进度和连接上下文。如果客户端通过稳定入口访问，入口可以在切换后把后续请求转发给 B；如果客户端直接连接业务服务器，则仍需要与 B 重新建立连接。同样的机制也适用于视频会议的房间状态、实时协作的文档会话，以及其他需要在实例之间改变处理者的有状态对象。

因此，一次正确的迁移至少要满足三个条件。第一，B 接管时必须拥有截至切换点已经生效的状态，以及继续处理未完成请求所需的信息；第二，同一时刻只能有一个实例拥有该会话的写入权，避免 A 和 B 同时修改状态；第三，只有 B 完成状态装载并确认可以服务后，系统才能切换对象归属和请求路由，A 也只能在接管确认后清理旧状态。

围绕这些条件，迁移通常包含一组共同动作：控制器选择迁移对象和目标实例，目标实例完成必要的资源准备；源实例复制状态，并根据方案停止状态修改或记录复制期间的增量；目标实例装载并校验状态；控制器把唯一写入权和请求路由切换到目标实例；目标实例恢复服务，源实例在收到接管确认后清理旧状态。不同方案都要完成这些动作，区别在于何时停止状态修改，以及哪些数据必须在停止服务期间传输。

如果源实例从复制开始就停止修改，状态容易保持一致，但停服窗口会覆盖整个传输过程；如果源实例在复制期间继续运行，系统就必须追踪并补齐其间发生的变化。下面先以停止复制法建立对照基线，再讨论预复制和按优先级分波迁移如何缩短停服窗口。

方案一：停止复制法 (Stop-and-Copy) 最直接的做法是“先停、再搬、再启”，即停止复制法，如图 40 所示。迁移开始时，源服务器立即停止接收该用户的新请求，并暂停会修改会话的计时器和后台任务，将用户的完整会话状态序列化为一个可传输的数据结构，通过网络传输到目标服务器，目标服务器反序列化后恢复用户会话，再切换会话归属和请求路由，由目标服务

器继续提供服务；客户端直接连接业务服务器时，还需要重新连接到目标服务器。这种方案的优点是实现极为简单：在所有状态修改入口都已暂停的前提下，停机期间状态不再变化，序列化的快照可以保持一致，目标服务器也不必追赶复制期间的新变化；系统仍需按顺序完成目标确认、归属切换和路由切换。缺点也同样明显：用户在整个迁移过程中处于完全停服状态，停机时长与需要迁移的状态数据量成正比。对于一个状态复杂的用户（如实时协作或高频交互场景中的大量并发状态），全量序列化和传输可能耗时数秒甚至更长，用户会明显感知到连接中断或请求超时。因此，停止复制法通常只适用于离线维护窗口或用户处于彻底空闲状态的场景，不适合作为生产环境中的主动负载均衡手段。

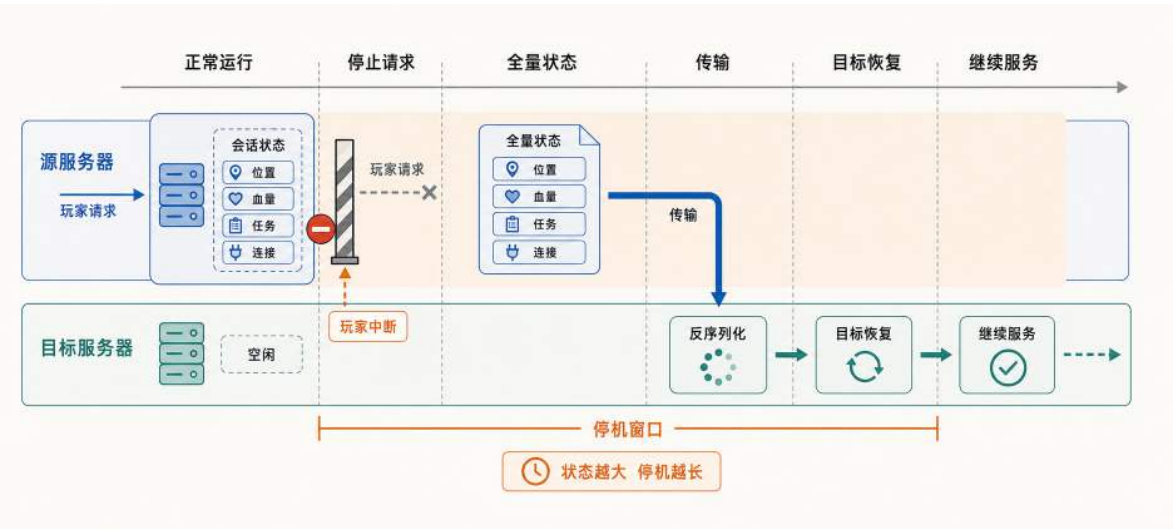


图 40: 停止复制法：源服务器暂停服务后传输全量状态，停机时间随状态体量增长。

停机窗口覆盖了整个状态序列化与传输过程。状态越大、网络越慢，用户等待的时间就越长。这自然引出一个优化方向：能不能把大部分传输提前到停机之前完成？

方案二：预复制法（Pre-Copy） 为了缩短用户感知到的停机时间，可以把大部分数据的传输提前到停机之前完成，这就是预复制法，如图 41 所示。其核心思路是：在源服务器仍然运行的同时，先把一份状态快照传输到目标服务器；每一轮传输结束后，再追踪本轮期间发生变化的状态块，并在下一轮只传输这些增量，直到每轮增量收敛到足够小；最后才短暂停止状态修改，将最后一批增量传输过去，完成最终切换。如果迁移单元是虚拟机或完整进程，变化追踪可以发生在内存页层，修改过的页通常称为脏页（Dirty Pages）；如果迁移单元是应用层会话，更常见的做法是把会话编码为带版本号的逻辑状态块，或者用变更日志记录快照之后的增量。两种实现的共同前提是：状态变化速度低于增量传输速度。绝大多数用户数据（已完成的任务记录、不在使用的配置项、离线关系列表等）在单次传输间隔内根本不会被修改，只有当前正在活跃变化的那部分状态（如实时位置、状态值）才会产生新的增量。随着迭代轮次增加，每轮需要重传的数据逐渐减少，最终只需在较短的停机窗口内传输最后一批增量就能完成切换；实际窗口取决于剩余增量、网络带宽、序列化成本和切换协议，没有固定的毫秒数保证。预复制法的代价是总传输量会超过全量状态大小。部分状态块在多轮迭代中被反复传输，网络带宽的消耗高于停止复制法；

对于那些持续高频修改的“热状态”，无论迭代多少轮都无法在停机前完全收敛，最终仍然要在停机阶段传输，这构成了预复制法停机时间的理论下界。

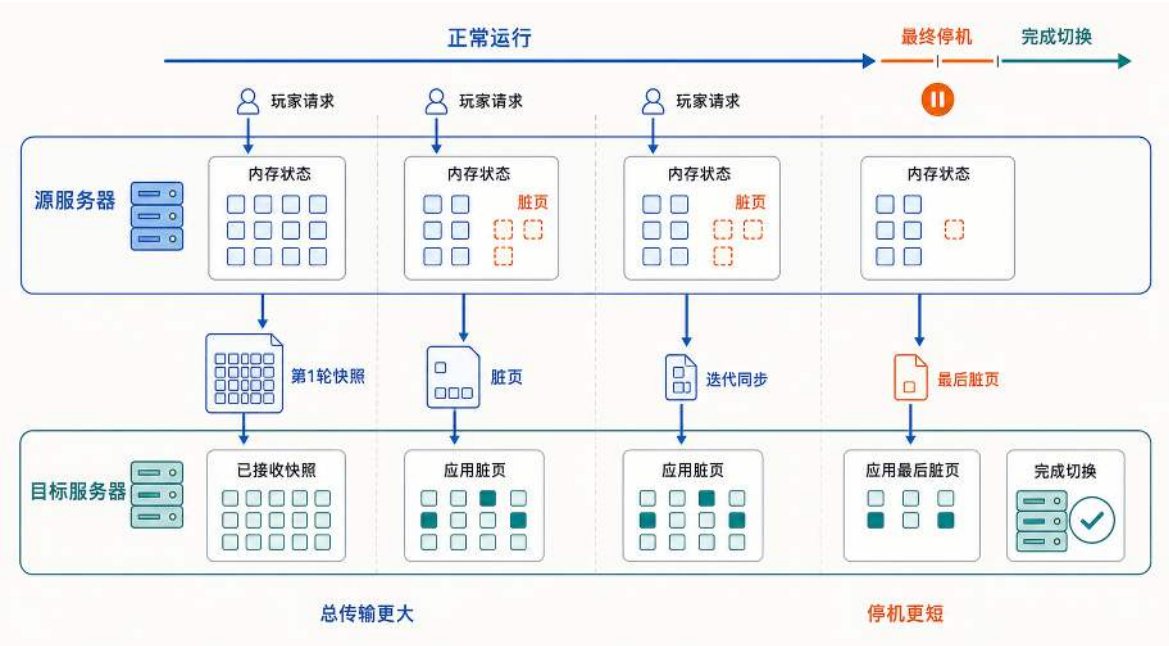


图 41: 预复制法：运行中迭代传输变化的状态块，以总传输量增加为代价，把最终停机窗口压缩到最后一轮。虚拟机可以按脏页追踪变化，应用会话通常按逻辑状态块或增量日志追踪变化。

预复制法把大部分数据传输挪到了源服务器仍在运行的阶段，停机窗口只剩最后未收敛的增量传输。代价是传输期间会持续消耗网络带宽，且状态变化速率必须低于传输速率，否则停机窗口无法缩小。

方案三：按优先级分波迁移（Wave-Based Migration） 预复制法是虚拟机热迁移中的典型思路，停止复制法则提供了便于比较停服时间的基线。在虚拟机场景中，迁移单元是整个虚拟机的内存状态（包括全部内存页和设备寄存器状态）。在线服务的状态迁移有所不同：应用能够识别哪些状态决定后续请求的正确性，哪些只是可重新加载的缓存、派生视图或背景数据。切换服务器时，所有会影响业务判断和后续写入的权威状态都必须进入第一波；只有不参与当前决策、允许重新获取或短暂陈旧的数据，才能延后同步。基于这种分类，可以设计按优先级分波迁移法，如图 42 所示。

分波迁移包含五个连续阶段。第一步是预热：目标服务器提前加载静态资源，把可预先完成的初始化移出切换路径。第二步是交接准备：源服务器停止接收该会话的新写入，排空正在执行的请求，并为本次迁移生成单调递增的版本号或 epoch。第三步是关键状态传输：源服务器发送第一波权威状态，目标服务器装入并校验这些状态后只返回“准备完成”，暂不对外提供服务。第四步是所有权切换：路由表或租约服务以本次 epoch 把唯一写入权转交给目标服务器，旧 epoch 的请求随后被拒绝；只有这一步成功后，目标服务器才能接收新请求。第五步是后台补齐：可重新获取的缓存、派生视图和背景数据继续分批同步。任何会影响业务判定的状态都不能留到这一

阶段。其核心实现逻辑如下：

按优先级分波迁移的核心逻辑

```
1 // 阶段1: 提前预热目标服务器 (后台静默执行)
2 targetServer.PreloadStaticAssets(session.ScopeID)
3
4 // 源端进入交接状态: 停止新写入并排空正在执行的请求
5 epoch := sourceServer.BeginHandoff(session.ID)
6 sourceServer.DrainInFlight(session.ID)
7
8 // 第一波: 所有影响后续业务判断的权威状态
9 wave1 := SessionCriticalState{
10     Position:      session.Position,
11     StateSnapshot: session.StateSnapshot, // 保证状态连续
12     CoreValue:     session.CoreValue,
13     LastCommandSeq: session.LastCommandSeq,
14     PendingActions: session.PendingActions,
15     TimerState:    session.TimerState,
16 }
17 targetServer.Prepare(epoch, wave1) // 此时尚未接收用户请求
18
19 // 路由或租约服务原子转移唯一写入权; 旧 epoch 的请求会被拒绝
20 ownership.Transfer(session.ID, sourceServer, targetServer, epoch)
21 targetServer.Activate(epoch)
22
23 // 后续波次只能包含可重建、可短暂陈旧的非权威状态
24 wave2 := collectDerivedViews(session)
25 targetServer.ApplyWave(2, wave2)
26
27 wave3 := collectReloadableCaches(session)
28 targetServer.ApplyWave(3, wave3)
29
30 // 目标端确认接管后, 源端清理旧 epoch 的状态
31 sourceServer.CleanupSession(session.ID, epoch)
```

分波迁移以会话为单位，转移的是继续处理该会话所需的应用状态及其唯一写入权。它与同一进程中的线程切换有一个共同点：都要保留后续执行所需的状态；区别在于，线程切换时应用状态仍留在原机器的内存中，而跨服务器迁移还要通过网络复制状态，并在两台服务器之间转移唯一写入权。

停止新写入并排空在途请求后，源服务器生成一份不再变化的权威状态快照，并用本次 epoch

标识它的版本。除了位置和业务状态，这份状态还要包含最后处理的请求序号、尚未完成的操作、定时器以及防止重复执行所需的信息，使目标服务器能够从源服务器停止的位置继续处理。目标服务器通过 `Prepare(epoch, wave1)` 装入并校验这些状态，但此时写入权仍属于源服务器，因此不能接收新请求。

关键状态传输完成并不等于服务已经恢复。路由表或租约服务成功转移唯一写入权后，目标服务器执行 `Activate(epoch)`，入口层才能把该会话的后续请求转发到目标服务器。客户端经过稳定入口访问时，通常只会感受到短暂停顿或请求重试；如果客户端直接连接业务服务器，则需要重新建立连接。

静态资源可以在交接前预热，缓存和派生视图也可以在资源允许时提前预复制，但它们不能成为目标服务器接管写入的依据。源服务器仍在运行时，这些非权威状态可能继续变化，提前传输的副本因而可能过期；所有权切换后，目标服务器还要检查版本，并刷新或重新生成这些状态。分波迁移先恢复正确处理请求所需的最小状态，再逐步恢复完整运行状态，从而避免缓存和背景数据延长停写时间。图 42 按这一顺序标出了关键状态传输、所有权切换、恢复服务和后台补齐之间的关系，并补出了目标服务器确认接管后，源服务器才清理旧状态这一收尾动作。

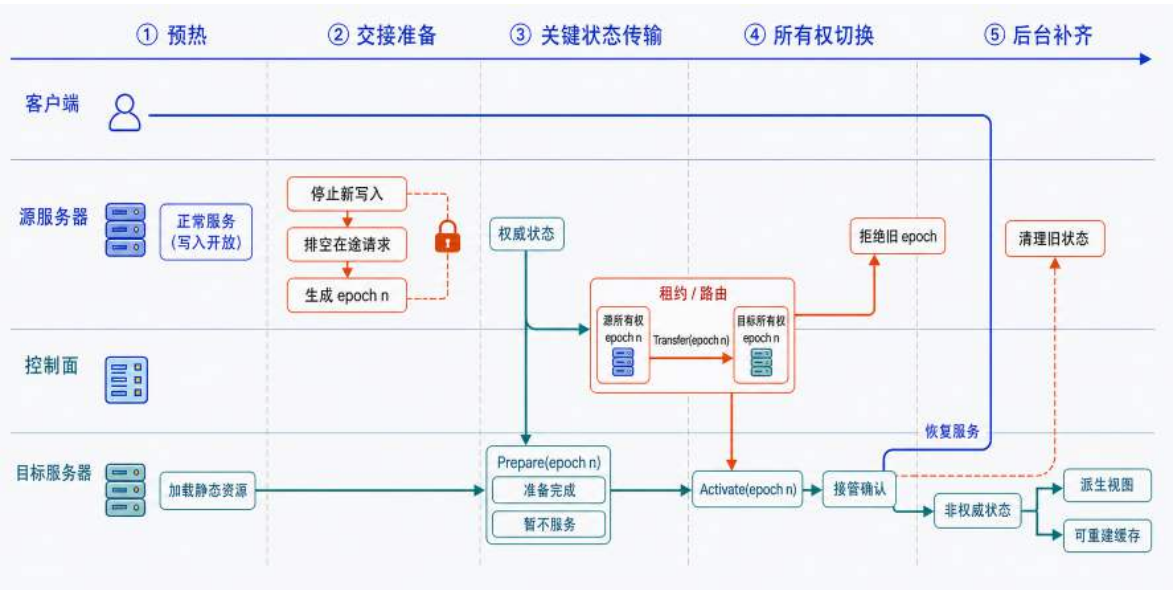


图 42: 分波迁移先传输权威状态并完成唯一写入权交接；目标服务器确认接管后，源服务器清理旧状态，非权威状态则在后台继续补齐。

三种方案的本质区别在于如何权衡停机时间、传输量和实现复杂度。停止复制法以最简单的实现换来最长的停机时间，适合维护窗口或离线场景。预复制法通过迭代传输变化状态压缩停机时间，但总传输量更大，适合通用服务器迁移场景，VMware 的 vMotion 和 Xen 的虚拟机热迁移均采用此思路。按优先级分波迁移法利用状态本身有重要性层次这一特点，把权威状态交接与非权威状态补齐分开，代价是开发者必须完成状态分类、版本控制、写入冻结和路由切换，实现复杂度最高，但在这些条件成立时可以进一步缩短用户感知的中断。

这三种方案比较的是有状态对象迁移中的通用取舍：停止复制提供最简单的正确性基线，预复制通过追踪增量减少停服时间，分波迁移则进一步利用应用能够区分权威状态与非权威状态这

一条件。这些思路不只适用于用户会话，也可以用于视频会议房间、实时协作文档以及其他有状态服务；其中分波迁移只有在非权威状态允许延后恢复时才成立。

运行时迁移并非所有应用服务器都必须具备的能力。生产环境中更常见的做法是以房间或游戏实例为单位等待任务自然结束后再摘流，或者在客户端重连时根据会话令牌从外部存储恢复状态。只有当系统不能等待对象自然结束，又需要尽快释放过载实例时，主动迁移的额外复杂度才可能值得承担。

决定执行迁移后，调度器还要选择合适的对象。高频交互中的会话持续修改位置、状态值和计时器，迁移期间需要追踪的变化多，难以迅速收敛；正在执行交易、资产变更等一致性敏感操作的会话，还需要额外协调事务状态，通常不应作为优先迁移对象。更合适的对象是当前状态变化较少的会话，例如长时间未操作的空闲用户，或者刚结束主要任务、暂时只在查看结果的用户。这些对象需要复制的增量较少，迁移更容易在较短的停机窗口内完成。

无论采用哪种迁移方案，工程实践中都需要配合约束条件，防止迁移本身成为新的负担。

第一项约束是迁移冷却期。迁移过程本身会消耗源服务器和目标服务器的 CPU、内存和网络带宽：源服务器要序列化状态并发送，目标服务器要接收并反序列化恢复。如果允许同一台服务器在短时间内反复触发迁移，迁移操作占用的资源可能反而加重服务器负担，甚至出现“越迁越忙”的恶性循环。因此，同一台服务器在完成一次迁移后，通常需要等待一段冷却时间才能再次触发，让服务器有时间消化迁移带来的瞬时开销并稳定负载观测值。

第二项约束是单次迁移的规模上限。如果一次迁移把大量对象同时送到同一台目标服务器，目标服务器需要在短时间内完成大量会话恢复、缓存加载和连接建立。这种瞬时涌入可能导致目标服务器本身进入高负载状态，把过载从一台服务器搬到了另一台。工程上通常限制单次迁移的对象数量，并将迁移分批执行：每批迁移完成后观察目标服务器的负载变化，确认稳定后再执行下一批。

从更高的角度来看，运行时迁移把分片再均衡从“修改映射”推进到“转移状态”：系统不仅要计算新的对象归属，还要在不中断体验的前提下完成状态交接。第 6.3.2 节讨论有状态服务伸缩时会继续回到这个约束：扩容和缩容都不是把数字改大或改小就结束，还要考虑状态、连接和迁移成本。

4.2.6 存储分片：从单库瓶颈到跨分片访问

前面的分片键、映射、路由、负载监测和动态调整，描述了对对象分片的通用控制链。应用到存储层时，被划分的对象变成用户数据，承载实例变成数据库节点；数据持久化、全局查询和事务提交也随之进入分片需要讨论的问题。

在单库架构中，所有用户数据和读写请求集中在一个数据库实例上，容量上限和故障影响也集中在同一个位置。存储分片可以分散数据容量、请求压力和故障范围，却会把原本由单一数据库完成的查询与事务拆到多个节点。因此，存储分片需要回答两个连续的问题：单一数据库为什么必须拆分，以及拆分之后如何处理跨分片查询和跨分片更新。

单一数据库的瓶颈与风险 第三章中，单机服务器使用 Redis 和 PostgreSQL 做分层存储。Redis 承载在线状态、实时计数、排行榜等热数据，PostgreSQL 保存账号信息、业务记录、配置等冷数据。在单机多人原型中，应用服务器只需要连接一个 Redis 实例和一个 PostgreSQL 实例，就能完成任意用户的缓存读取、状态更新和持久化写入。

当用户规模继续扩大，数据层的压力会集中到这两个实例上。PostgreSQL 负责保存关系型数据，并通过事务、索引、锁和写前日志保证数据可靠落库。一次查询进入 PostgreSQL 后，通常要占用一个数据库连接，经过 SQL 解析、执行计划、索引查找和数据页读取；一次写入还要修改数据页，并把变更记录写入日志。查询和写入持续增加时，连接数、磁盘 I/O、锁等待和日志写入都会限制整体吞吐。Redis 的职责不同，它主要把热数据保存在内存中，用内存访问换取更低延迟。但内存容量同样有限，在线状态、实时位置和排行榜片段持续增长后，缓存淘汰会变多；缓存未命中的请求又会回到 PostgreSQL，进一步加重数据库压力。纵向扩容可以提高单机配置，却不能改变所有用户数据仍集中到同一组存储实例上的事实。

单一数据库的瓶颈首先来自请求排队。在单机架构中，登录、查询和写入等请求最终都会访问同一个 PostgreSQL 实例。每个请求进入数据库后，都要占用连接和执行资源；查询需要查找索引、读取数据页，写入需要修改数据页、记录写前日志，并在必要时等待锁释放。当并发请求超过数据库能够稳定处理的速度时，新请求不会立刻完成，而是在连接池、数据库执行队列、磁盘 I/O 队列或锁等待中排队。排队时间继续累积后，用户看到的就是登录变慢、查询响应延迟、写入请求超时，甚至新连接被拒绝。

提高单机硬件配置可以推迟瓶颈出现，但不能消除单实例结构本身的限制。更多 CPU 核心可以并行处理更多查询，更多内存可以缓存更多数据页，更快的磁盘可以提高日志写入和数据读取速度。但数据库并不是所有环节都能随硬件线性增长：事务提交仍可能等待日志刷盘，热点记录仍可能发生锁竞争，索引更新仍要维护内部结构，网络接口和内存带宽也有上限。因此，单机配置越高，成本通常增长得更快，而吞吐提升会逐渐变慢。

单一数据库还会把故障影响集中到一个位置。所有用户的核心数据都在同一个实例上时，磁盘故障、数据库进程崩溃、操作系统故障或网络中断，都会同时影响登录、存档、结算和排行榜。备份可以降低永久丢失数据的风险，但备份不能直接提供实时服务。从备份恢复到可用状态需要时间，这段时间内用户仍然无法正常读取和写入数据；如果备份间隔较长，还可能丢失最近一段时间内已经发生的更新。因此，单一数据库同时暴露出两个问题：单个实例的容量有限，实例故障又会影响全部用户。这两个问题需要采用不同机制处理。容量上限要求系统把数据和请求分散到多个数据库实例；故障后继续提供服务则需要副本和故障切换机制。本节先解决容量问题：系统按照分片键把数据集合划分给多个数据库实例，并让路由层把请求送到保存目标数据的实例，这种横向拆分称为**数据库分片**。

图 43 使用同一组玩家比较拆分前后的数据归属和请求流向。在左侧的单库结构中，玩家组 A、B、C、D 共享同一个请求队列和数据库，所有请求都会竞争同一组连接、执行和存储资源。在右侧的分片结构中，路由层按照玩家 ID 将四组请求分别送入对应分片，每个数据库实例只保存并处理其中一部分玩家的数据。以玩家组 C 为例，它的请求进入分片 3；分片 3 发生故障时，玩家组 C 暂时无法访问数据，但其他三个分片仍可继续处理各自玩家组的请求。

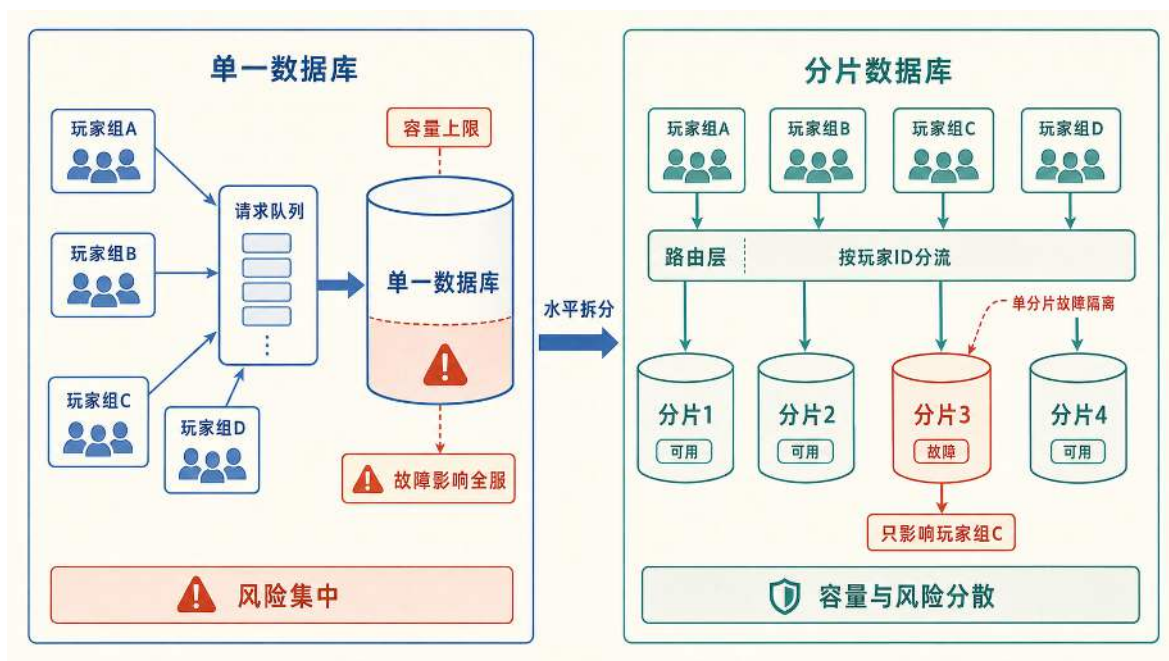


图 43: 水平拆分按玩家 ID 分散请求和数据容量，并把单个数据库故障的影响限制在对应玩家组；分片缩小了故障范围，但不能保证故障分片继续提供服务。

对于单个玩家的局部访问，分片使多个数据库能够并行承接请求，并把单个实例的故障影响限制在其负责的玩家范围内。但分片并不等于高可用：分片 3 故障后，玩家组 C 仍然无法读写数据；要让该分片继续提供服务，还需要在分片内部配置副本和故障切换机制。

上述容量扩展和故障范围缩小都依赖稳定的数据归属。同一玩家的请求持续进入同一分片，资料、配置和状态等局部访问才能在一个数据库内部完成；与此同时，需要覆盖全部玩家或同时修改多个玩家数据的操作，也会因为数据已经分散而跨越多个分片。下面分别讨论跨分片查询和跨分片更新带来的问题。

跨分片查询 当查询条件包含分片键且只涉及一个用户时，路由层可以直接定位目标分片，查询仍由单个数据库完成。例如，用户 1001 的资料、配置和状态位于分片 1，应用服务器只需访问该分片。

当查询范围超过一个分片时，单个数据库只能返回局部结果。比如，全服排行榜需要比较所有用户的战力分数，而每个分片只保存其中一部分用户；查询任意一个分片，都无法得到完整排名。因此需要读取多个分片来汇总结果，这类操作称为**跨分片查询**。

以全服排行榜为例，在单一数据库中，全部用户记录位于同一个查询范围内，可以通过一条排序语句得到全局前一百名：

单一数据库中的排行榜查询

```
1 SELECT id, name, score
2 FROM users
3 ORDER BY score DESC
4 LIMIT 100;
5 -- 在单个数据库内完成全局排序并返回前100名
```

数据库分片后，完整用户集合分散在多个数据库实例中。在任意一个分片上执行上述 SQL，都只能得到该分片的局部 Top100，不能直接代表全服排名。为了得到全局 Top100，系统需要在所有分片上并行执行这条查询，收集各分片的局部 Top100，再合并候选记录、重新排序并截取前一百名。汇总时不必读取各分片的全部用户记录，每个分片只需返回自己的局部 Top100。下面的伪代码给出了并行查询和集中汇总的核心过程：

跨分片排行榜的并行汇总伪代码

```
1 func getGlobalTop100(shards []Shard) ([]User, error) {
2     // 1. 并行查询每个分片的局部Top100
3     localRankings, err := parallelQuery(
4         shards,
5         queryLocalTop100,
6     )
7     if err != nil {
8         return nil, err
9     }
10
11     // 2. 合并候选记录并重新排序
12     candidates := merge(localRankings)
13     sortByScoreDescending(candidates)
14
15     // 3. 截取全服Top100
16     return firstN(candidates, 100), nil
17 }
```

并行查询避免了逐个等待所有分片，但整体延迟仍由最慢的分片决定。任何一个分片超时或失败，都可能使本次排行榜缺少一部分候选记录；应用层还需要处理多次网络请求、各分片的局部排序以及候选结果的二次合并。因此，跨分片查询的主要代价不是 SQL 语句变长，而是一次数据库内部查询变成了跨多个节点的并行查询、等待和汇总过程。

实时跨分片查询的问题在于，每个用户查看排行榜时，系统都要重新向所有分片发起查询并合并结果，而不同用户读取的其实是同一份全服排名。如果每次请求都重复执行相同的分片查询和排序，数据库、网络和应用服务器会反复承担同一项计算。

排行榜通常允许短时间延迟，不要求每次查询都包含刚刚结束的最后一批操作。因此，可以

把全局合并从用户的查询路径移到后台更新路径，这种做法称为异步汇总。排行榜汇总服务定期拉取各分片的局部 Top100，或者持续消费用户分数变化事件，在后台计算全服 Top100，再把结果写入 Redis 缓存。这份提前计算并保存的全局结果可以看作排行榜的物化视图（Materialized View），把查询结果预先计算并存储，之后直接读取，减少重复计算与访问。

Redis 在这里不保存用户分数的权威记录，也不负责实时执行跨分片查询。它保存的是汇总服务已经计算完成的全服榜单：用户 ID 作为成员，分数作为排序值写入有序集合；用户查看排行榜时，只需从 Redis 读取分数最高的一百名。图 44 中的 Redis 榜单位于两条路径的交点，但职责边界并未改变：后台更新路径写入物化结果，用户查询路径只读取全服 Top100，不再实时访问 PostgreSQL 分片。

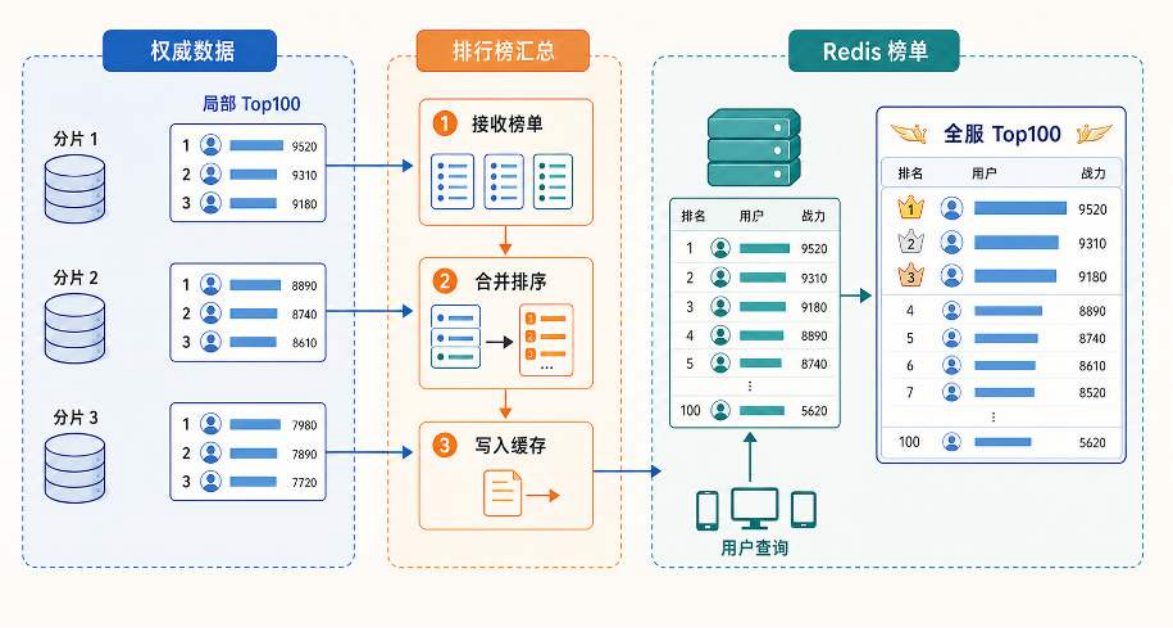


图 44: 排行榜汇总服务在后台合并各 PostgreSQL 分片的局部 Top100，并写入 Redis 物化榜单；用户请求只读取 Redis 中的全服 Top100，不再实时查询所有数据库分片。

异步汇总把一次读取从“访问所有 PostgreSQL 分片并现场合并”缩短为“一次 Redis 查询”，代价是结果具有一定延迟。刷新间隔越短，榜单越接近实时，但分片查询和缓存更新越频繁；刷新间隔越长，系统开销越低，用户看到的排名也越陈旧。汇总服务暂时失败时，系统还可以继续返回 Redis 中的上一版本榜单，待后台恢复后再生成新版本。

跨分片查询可以通过并行查询和集中汇总获得完整结果；对于允许短暂延迟的查询，还可以使用异步汇总降低读取开销。如果一次业务操作需要更新（写入）多个分片，多机系统中的处理会复杂得多。

跨分片更新：从本地事务到两阶段提交 在单个数据库内部，多项更新可以由一个本地事务统一处理，过程相对而言比较直接。以用户 A 向用户 B 转移 10 个金币为例，这次操作包含两项修改：从 A 的账户扣除 10 个金币，同时向 B 的账户增加 10 个金币。当 A 和 B 的账户记录位于同一个数据库时，数据库可以在一个事务中依次执行扣款和入账：两项修改都成功后统一提交，任一修改失败则整体回滚。

如果 A 和 B 的账户记录分别位于分片 1 和分片 2，情况就复杂得多。扣款由分片 1 的本地事务处理，入账由分片 2 的本地事务处理，原来的一次数据库事务被拆成了两个独立提交点。每个分片只能决定自己的事务是否提交，无法保证另一个分片作出相同决定。如果扣款已经提交而入账失败，金币就会消失；如果入账已经提交而扣款失败，系统又会凭空增加金币。

因此，这笔转账要求扣款和入账全部生效或全部不生效，不能留下只完成一部分的结果，这种性质称为原子性（Atomicity）。

在单个数据库中，事务管理器能够同时看到两项修改，并统一决定提交或回滚。分片后，每个数据库只能看到自己的本地事务：分片 1 不知道分片 2 是否完成入账，分片 2 也不知道分片 1 是否完成扣款。网络超时还可能使“操作失败”和“操作已经完成但响应丢失”无法区分，因此任何一个分片都不能仅根据本地结果决定整笔转账提交或回滚。

因此，跨分片更新的难点不是分别执行各项修改，而是让多个独立数据库对整项操作形成一个最终决定。两阶段提交协议（Two-Phase Commit, 2PC）为此引入协调者，收集各分片的执行状态，再向所有分片发布统一的提交或回滚决定。

协议先在准备阶段确认所有参与分片是否具备提交条件，再在决策阶段根据投票结果要求所有分片共同提交或共同回滚。准备阶段从协调者向所有参与分片发送准备请求开始。各分片执行本地事务、记录事务状态并锁定相关数据，但暂不提交；如果能够保证后续完成提交，就回复 Ready，否则回复 Abort。只有全部参与者都回复 Ready，协调者才会发布 Commit，各分片随后提交本地事务并释放锁。任一参与者回复 Abort，或者准备阶段等待超时，协调者都会发布 Rollback，各分片回滚本地事务并释放锁。这样，扣款和入账不能只提交一侧：全局结果只能是两个本地事务共同提交，或者共同回滚。

参与者回复 Ready 后，本地修改虽然尚未提交，却已经承诺能够执行后续提交，因此不能自行结束事务。从接受准备请求到收到最终决定，相关数据行必须保持锁定；图 45 以金币转账为例，分别标出扣款和入账在准备阶段的未提交状态，以及协调者在决策阶段发出的统一命令。

2PC 的原子性来自所有参与者服从同一个全局决定，它的运行代价也来自等待这个决定的过程。首先，协议需要两轮网络往返：准备阶段收集投票，决策阶段广播提交或回滚；每一轮都要等待参与分片响应，端到端延迟明显高于单分片事务。其次，参与者进入就绪状态后会持续持有行锁，其他读写相同数据的事务只能排队，高并发场景下吞吐量会被压低。最后，协调者在收集投票后、发布最终决定前发生故障，参与者可能长期停留在就绪状态，相关锁也无法及时释放。

为使事务能够在故障后恢复，协调者和参与者都要把各阶段的状态写入稳定存储中的事务日志。参与者一旦进入就绪状态，就等于已经承诺“只要全局决定提交，我就能够提交”。如果此时协调者失联，参与者既不能擅自提交，也不能随意回滚；标准 2PC 要求参与者阻塞等待协调者恢复。部分实现允许参与者询问其他节点是否知道全局结果，但这已经超出标准 2PC 的范围。生产环境中，这类故障会同时影响事务恢复、锁释放和请求延迟，排查与恢复成本也较高。

由 2PC 的运行代价可以得到一个分片设计原则：越是要求原子提交的业务事实，越应尽量减少它跨越的提交点。如果一次业务结果不可避免地涉及多个分片，2PC 可以保证这些写入最终共同提交或共同回滚，但也会把锁等待、网络等待和协调者故障带入请求主链路。因此，在线系统通常只在少数必须同步完成的操作中使用 2PC；对于允许短时间不一致的更新，则通过调整

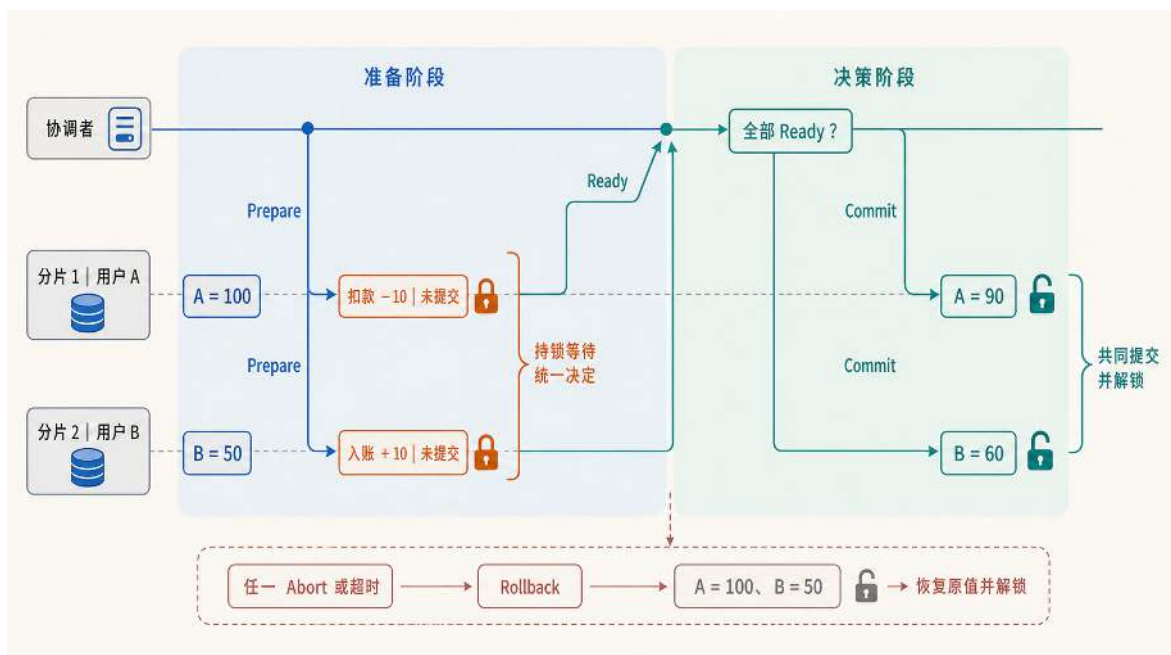


图 45: 2PC 先让两个分片完成本地修改并持锁等待；收齐 Ready 后共同提交，任一参与者中止或超时则共同回滚。

数据归属或异步传播降低跨分片协调成本。下面介绍两种常用做法。

第一种策略是异步消息队列。交互结束后，业务服务器把双方用户 ID、结果信息、时间戳等打包成一条操作结果消息，投递到消息队列。专门的记录更新服务从队列中消费消息，依次向各分片发起更新；若某次更新因分片暂时不可用而失败，消息会留在队列中等待重试。在队列持久化、重试和幂等处理都成立的前提下，丢失风险可以降到很低。这种方案把跨分片同步原子提交改为异步更新。记录可能晚几秒刷新，但在可靠投递与幂等处理成立后会逐步收敛，系统也避开了 2PC 的锁等待和协调成本。消息队列本身必须具备持久化和至少一次投递的保障，否则队列自身的故障反而会成为新的数据丢失入口。

下面的代码展示异步记录更新的核心逻辑。`finishInteraction` 只负责把操作结果写成事件并投递到消息队列；`consumeResult` 由后台消费者执行，分别访问相关用户所在分片。重要的是主链路和后台链路的分离：交互请求不再等待两个分片都更新完成。

异步记录更新的核心逻辑

```
1 type OperationResultEvent struct {
2     UserA    int
3     UserB    int
4     MatchID  string
5 }
6
7 func finishInteraction(result OperationResult) error {
8     event := OperationResultEvent{
9         UserA:    result.UserA,
10        UserB:    result.UserB,
11        MatchID: result.MatchID,
12    }
13    return messageQueue.Publish(event) // 事件持久化后才确认请求已受理
14 }
15
16 func updateOnce(userID int, matchID string) error {
17     db := shardFor(userID)
18     return db.Transaction(func(tx Tx) error {
19         if tx.HasProcessed(matchID, userID) {
20             return nil // 重试时不重复更新
21         }
22         if err := tx.UpdateRecord(userID); err != nil {
23             return err
24         }
25         return tx.MarkProcessed(matchID, userID)
26     })
27 }
28
29 func consumeResult(event OperationResultEvent) error {
30     if err := updateOnce(event.UserA, event.MatchID); err != nil {
31         return err // 保留消息，稍后重试
32     }
33     return updateOnce(event.UserB, event.MatchID)
34 }
```

在异步消息方案中，请求主链路返回前只需要确认操作结果事件已经可靠写入队列。这表示更新任务已经被系统受理，并不表示相关用户分片已经完成更新。消费者随后以 `MatchID` 和用户 ID 作为幂等标识，在每个目标分片的本地事务中完成记录更新和已处理标记。即使用户 A 已

经更新、用户 B 更新失败后触发整条消息重试，用户 A 也不会被重复更新。

如果队列中已经积压较多事件，或者某个分片写入失败后进入重试，用户界面上的记录可能晚于操作结果更新。这种方案避免了交互请求同时等待两个分片完成写入，但允许各分片的记录在短时间内处于不同进度。

本例没有在业务服务器的本地数据库中另存一份正式结果，队列中的结果事件就是后台更新各用户分片的依据，因此主链路只有“写入持久化队列”一个持久化动作。如果系统还必须在本地数据库中保存正式结果，再向其他节点发送消息，就会同时涉及数据库写入和消息投递；这两个动作无法由同一个普通数据库事务直接保护，需要采用第 4.3.4 节介绍的事务性发件箱。

不过，异步消息队列只是把跨分片写入从请求主链路移到后台。记录仍然分散在用户分片中，后台消费者最终还是要分别更新相关用户所在数据库。如果记录写入非常频繁，或者历史记录查询本身成为核心访问，系统还可以进一步改变数据归属：让记录由专门服务统一管理。

第二种策略是重新设计数据模型，从根本上减少跨分片事务出现的机会。如果记录是高频写入的核心数据，可以将记录从用户分片中完全剥离，交由一个独立的记录服务集中管理。这个服务可以使用单一数据库（若数据量可控），也可以使用专门为追加写入优化的时序数据库。所有对记录的读写操作都发生在该服务内部，从结构上消除跨用户分片写入。代价是系统多了一个独立服务和对应的数据库，运维复杂度会略有上升，但记录访问路径会更清楚。尤其是查询历史交互记录这类高频只读场景，单服务内查询通常比跨分片聚合更高效。

在代码中应用服务器在交互结束后调用 `reportResult`，把结果提交给 `RecordService`。`RecordService` 在自己的数据库事务中写入操作结果，并更新双方记录。由于这两个更新发生在记录服务内部，它们不再分别被放置在相关用户所在的用户分片上。

独立记录服务的核心逻辑

```
1 type MatchResult struct {
2     WinnerID int
3     LoserID  int
4     MatchID  string
5 }
6
7 func reportResult(result MatchResult) {
8     recordService.SaveResult(result)
9 }
10
11 func (s *RecordService) SaveResult(result MatchResult) {
12     s.db.Transaction(func(tx Tx) {
13         tx.Insert("match_results", result)
14         tx.Update("user_records", result.WinnerID, "wins + 1")
15         tx.Update("user_records", result.LoserID, "losses + 1")
16     })
17 }
18
19 func queryRecord(userID int) UserRecord {
20     return recordService.GetRecord(userID)
21 }
```

独立记录服务改变的不是提交协议，而是事务的边界。一次操作的记录、双方记录都写入记录服务自己的数据库，因此可以放在同一个本地事务中完成。相关用户的用户资料仍然可以位于不同分片，但记录更新不再经过这两个用户分片，也就不需要使用 2PC 协调多个提交点。代价是系统增加了一个专门服务，记录服务本身也要处理容量、备份和可用性问题。

异步消息队列和独立记录服务采用了不同的处理思路。前者保留原有数据归属，把跨分片写入移到后台执行；后者重新划分数据归属，使相关写入重新落入同一个事务边界。两种方案都降低了请求主链路中的跨分片协调成本，但分别引入了消息延迟和独立服务运维等代价。

PostgreSQL 分片在分散数据容量和请求压力的同时，也改变了数据访问的范围。单用户读取可以由路由层直接定位一个分片；全局读取需要并行查询多个分片并汇总结果，也可以通过物化视图把重复计算移到后台。

当一次写操作涉及多个分片时，多个本地事务还需要形成统一结果。2PC 可以保证它们共同提交或共同回滚，但会增加网络等待、持锁时间和阻塞风险；对于允许延迟生效的更新，异步消息或重新划分数据归属可以降低请求主链路中的协调成本。

前面的讨论针对 PostgreSQL 持久化层。应用服务器还会频繁访问 Redis 中的在线状态、实时位置和排行榜等热数据；如果这些数据仍然集中在单个 Redis 实例中，缓存层就会成为新的容

量与可用性瓶颈。下面继续讨论缓存层如何从单机 Redis 演进为 Redis 集群。

4.2.7 缓存层的分布式演进：从单机 Redis 到集群

Redis 是一种键值数据库：应用用一个字符串形式的键（Key）定位一份值（Value），例如用 `user:1001:pos` 读取用户 1001 的实时位置。在单机阶段，所有键都放在同一个 Redis 实例中，应用只需要连接这一个入口；进入多机阶段后，如果所有热数据仍然集中在这个实例中，缓存层就会成为新的集中点。在线人数增长时，单个 Redis 实例要同时承受更多键、更高网络吞吐和更多命令处理压力；一旦它不可用，原本命中缓存的读取会集中回到 PostgreSQL，前面通过数据库分片分散掉的压力又会在回源请求中被放大。因此，缓存层需要从单机 Redis 演进到 Redis 集群。

PostgreSQL 分片说明了一个基本原则：数据拆到多个节点后，请求只访问与自身查询范围相关的节点。单用户资料可以根据用户 ID 直达一个分片，全服排行榜则必须访问多个分片并合并结果。

缓存层扩展时沿用同一原则。Redis 不再把全部热数据集中在一个实例中，而是把键空间拆分给多个缓存节点，使每个节点只承担一部分 key 的存储和访问。

PostgreSQL 分片和 Redis 缓存分片的目标相同，都是把数据与请求压力分散到多个节点，但二者组织数据的方式不同。PostgreSQL 中的用户资料具有明确的业务归属，一条用户记录可以根据用户 ID 确定所属分片，全局查询则根据查询范围决定是否访问多个分片。Redis 保存的在线状态、实时位置和会话等数据具有不同结构，但每份数据都由唯一的 key 标识，因此缓存路由不需要解析具体业务字段，只需要根据 key 确定负责节点。

如果直接用 key 的哈希值和当前节点数量计算目标节点，节点数量就会成为路由公式的一部分。例如，集群从三个节点扩展到四个节点后，许多 key 的计算结果会随之改变；数据仍在原节点，请求却可能被发送到新节点，系统必须迁移大量数据，否则就会出现大范围缓存未命中。这种不稳定性来自 key 归属与节点拓扑直接绑定：每次增加或移除节点，都可能重新计算大量 key 的位置。

Redis 集群在 key 和物理节点之间增加了一层数量固定的哈希槽。key 先通过固定算法映射到槽位，槽位再通过槽位表归属于某个 Redis 节点。节点扩缩容时，key 对应的槽位编号保持不变，集群只需把部分槽位及其中的数据迁移给新节点，并更新槽位表。这里的“稳定”并不是让一个 key 永远停留在同一台物理节点上，而是让 key 到槽位的计算不受节点数量变化影响，使数据迁移能够按槽位分批进行。哈希槽（Hash Slot）就是 Redis 集群内部预先划好的编号位置。整个集群固定包含 16384 个槽位，这是 Redis 集群协议规定的槽位总数，业务侧不能把它改成其他数量。²⁴ 客户端拿到一个 key 后，先通过哈希计算得到槽位编号；随后根据本地保存的槽位表，查出这个槽位当前由哪个 Redis 主节点负责。槽位表可以理解为一张“编号区间到节点”的映射，例如槽位 0 到 5460 由节点 A 负责，槽位 5461 到 10922 由节点 B 负责，槽位 10923 到 16383 由节点 C 负责。这样，`user:1001:pos` 先被计算到某个槽位，再由槽位表定位到具体

²⁴Redis Cluster 官方规范说明了哈希槽和槽位重定向规则，见 https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/。

Redis 节点。

当系统新增 Redis 节点时，只需要把一部分槽位从旧节点迁移给新节点；迁移完成后，落在这些槽位上的 key 就由新节点负责。业务代码仍然按原来的 key 读写缓存，变化被集群客户端和槽位表吸收。如果客户端本地的槽位表已经过期，请求可能到达旧节点；旧节点会返回重定向信息，客户端更新槽位表后，再访问新的负责节点。

一次缓存请求先计算固定槽位，再依据槽位表选择 Redis 节点；扩容时，变化发生在槽位到节点的映射，而不是 key 到槽位的计算。图 46 上半部分展示 key 如何先计算出固定槽位，再通过槽位表定位到 Redis 节点；下半部分以新增节点 D 为例，展示槽位 13653 到 16383 及其中的数据如何从节点 C 迁移到节点 D，并相应更新槽位表，而 key 到槽位的计算规则保持不变。

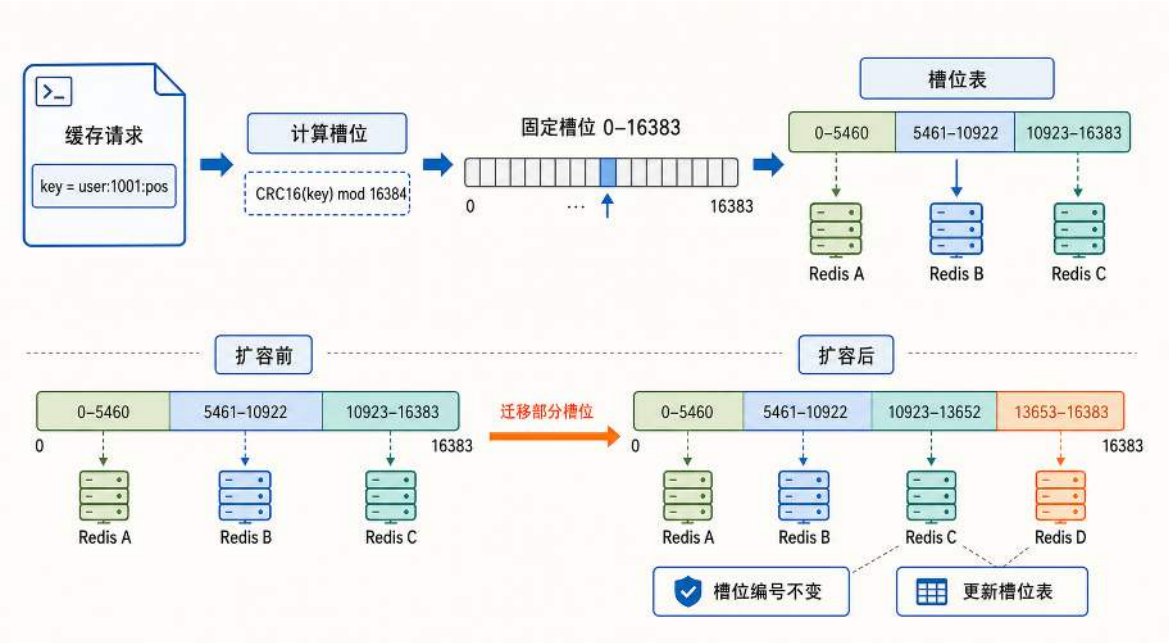


图 46: Redis Cluster 通过固定哈希槽解耦 key 与物理节点：key 先映射到槽位，槽位表再定位节点；扩缩容时只迁移部分槽位及其中的数据。

固定槽位把 key 的计算结果与当前节点数量解耦，使扩缩容可以按槽位分批迁移，而不必重新计算全部 key 的归属。

上面的讨论只处理了单个 key 的定位；当一次业务流程需要同时读写多个相关 key 时，槽位分布还会影响这条流程能否在一个 Redis 节点上完成。单个 key 可以自然定位到一个槽位，但协作状态、房间状态、同一用户多份临时状态这类操作，经常需要在一次流程中同时访问多个 key。例如，逻辑服在处理用户 1001 断线重连时，可能需要同时拿到该用户是否在线以及最后一次上报的位置。如果写成一次缓存读取，请求中会同时出现两个 key：

多 key 读取命令示例

```
1 MGET user:1001:online user:1001:pos
```

这类“一条命令同时操作多个 key”的请求，就是多 key 操作。如果这两个 key 分别落到不同 Redis 节点，单独读取任何一个 key 都没有问题；但合并为一条命令后，Redis Cluster 会直

接拒绝执行并返回 CROSSSLOT 错误，因为它不会跨节点协调多 key 操作。

Redis 集群提供哈希标签 (Hash Tag) 为这类相关 key 提供同槽位约束。²⁵ 哈希标签是一种键名约定：如果 key 中包含大括号，Redis 只取大括号内的部分来计算槽位。例如 `{user:1001}:pos` 和 `{user:1001}:online` 都使用 `user:1001` 计算槽位，因此它们一定落到同一个槽位和同一个 Redis 节点，多 key 操作就可以在该节点上完整执行。图 47 展示了这个对比：左侧使用默认键名，两份相关数据被分到不同槽位和不同节点，多 key 操作无法一次完成；右侧使用哈希标签后，两份数据落到同一节点，可以在一条命令内完整处理。

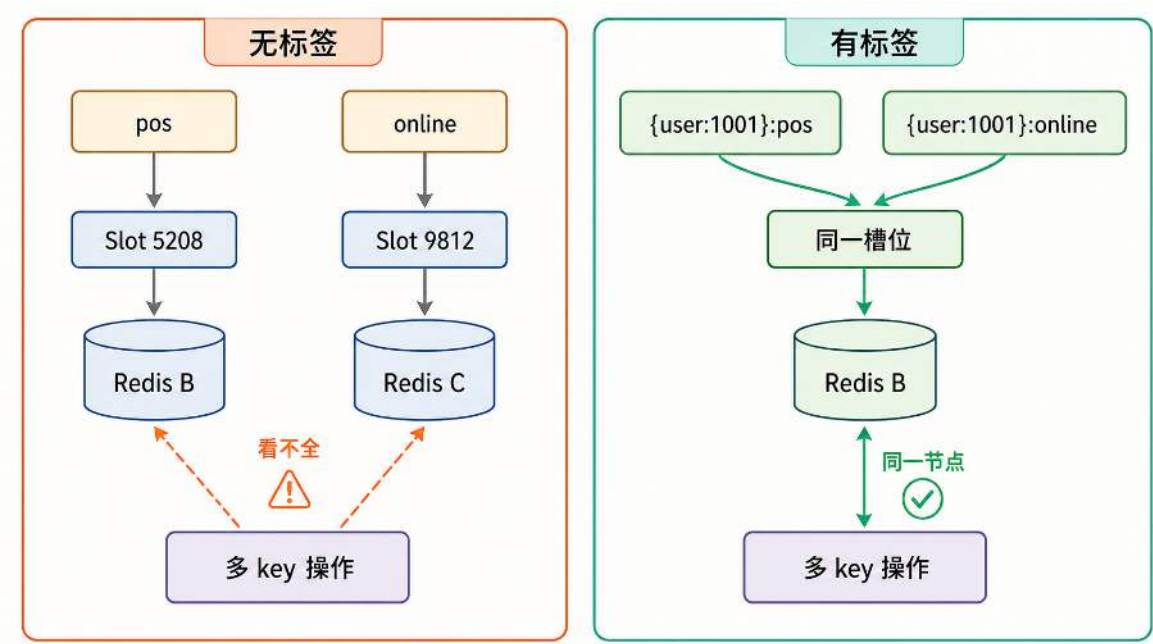


图 47: 哈希标签将相关 key 放到同一槽位：默认键名下，同一用户的在线状态和位置可能分别被放置到不同 Redis 节点；使用 `{user:1001}` 后，相关 key 可以由同一节点完整处理。

这也是缓存键设计需要考虑业务归属的原因：同一用户、同一房间或同一队伍中需要一起读写的热数据，适合使用同一个哈希标签；没有共同操作关系的 key 则不应随意放进同一标签，否则会把压力重新集中到少数槽位上。

缓存分片只决定热数据落在哪个 Redis 节点上，并不要求 Redis 节点和 PostgreSQL 分片固定绑在同一台机器上。两层存储可以分别扩展，但都围绕同一个业务主语组织数据：用户 1001 的权威记录通过 PostgreSQL 分片路由定位，用户 1001 的热数据通过 Redis 槽位定位。应用服务在处理登录、移动、断线重连或排行榜刷新时，仍然以用户 ID 作为入口，只是底层访问路径分别进入持久化层和缓存层。经过这一层拆分，数据层形成了两条可扩展路径：冷数据按分片路由进入 PostgreSQL，热数据按槽位路由进入 Redis；两类集中访问点都被拆到多个节点上，容量、吞吐和故障影响也随之分散。

²⁵Redis Cluster 对哈希标签和多 key 操作限制的说明见官方规范：https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/。

4.3 协同：跨服交互与事务

前面的“均衡”把用户、连接和数据分配到多个实例上，使每台服务器只负责一部分对象。在单机系统中，一个进程能够看到完成操作所需的全部状态，并直接确定处理顺序和最终结果；进入多机系统后，每台服务器只能观察自己的局部状态，一次业务操作却可能同时依赖多个服务器。

当一次业务操作所需的参与者、数据或处理结果分布在不同服务器上，单台服务器无法仅凭本地状态完成整个过程，这类操作称为**跨服交互**。跨服交互需要服务器通过网络交换信息，但信息能够传到其他服务器，并不意味着多台服务器已经形成业务可以接受的结果。

要让多个独立运行的服务器共同完成一项业务，还需要规定哪些状态差异可以暂时存在、由谁确定正式结果，以及结果如何传播并落实到其他服务器。这组使多个节点形成业务可接受结果的跨节点规则，构成多机系统中的**协同**。

按照跨节点处理的对象和目标，跨服交互可以分为三类：通知类传播已经确定的事件，查询类把分散的局部状态组成读取视图，事务类则让多项更新共同生效或使多个服务器接受同一个结果。三类场景能够容忍的状态差异不同，所需的协同约束也随之不同。

4.3.1 跨服交互的三类场景与约束

通知类交互传播的是已经产生的事件。例如世界 Boss 出现通知。世界 Boss 已经在服务器 A 负责的地图中生成，该服用户随即看到活动公告、地图标记和挑战倒计时。此时服务器 B 和服务器 C 可能尚未收到消息，它们的用户仍停留在原来的界面；随着消息传播，各服陆续显示活动提示。活动入口是否开放、倒计时从何时开始等业务状态，应以可持久化的权威记录为准，不能取决于服务器是否收到这条即时通知。短暂的到达时间差不会改变 Boss 已经生成这一事实，但系统仍要先区分通知的重要程度。允许用户偶尔错过的即时提示可以采用尽力而为传播；若遗漏会影响后续业务，事件就必须进入可持久化、可确认和可重放的消息路径，不能只依赖瞬时通知。同样的结构也出现在电商促销启动通知、直播间的全局广播等场景中。

查询类交互需要把多个服务器上的局部数据组成一个读取视图。例如跨服竞技场排名系统。竞技场开放报名后，服务器 A 的用户刚刚加入队伍，服务器 B 的一名用户同时退出报名，而大厅中的参赛列表仍显示上一次汇总结果。用户在刷新前可能暂时看不到新加入者，也可能看到已经退出的旧成员；只要系统在规定时间内重新汇总各服数据，列表就会恢复为最新状态。查询类场景允许读取视图短时间陈旧，但必须限定旧数据持续的最长时间，这一要求称为有界收敛。分布式搜索索引、跨区域库存预览也采用类似的异步汇总机制。

事务类交互会同时改变多份状态，或者要求多个服务器接受同一个决定。以跨服交易为例，用户 A 在服务器 1 确认支付一定资源，用户 B 在服务器 2 等待接收对应资源。如果网络中断后服务器 1 已经扣款，服务器 2 却没有交付资源，那么两名用户面对的就不是短暂延迟，而是一笔结果矛盾的交易。跨服战斗也不能让一台服务器记录甲方获胜，另一台服务器却记录乙方获胜。事务类场景不能接受这类冲突结果，需要让多项更新同时成立或同时撤销，或者由一个权威位置给出各服共同接受的唯一结果。跨账户转账、分布式库存扣减和联名票务预订都属于同一类

事务类约束。

前述三类交互都会在服务器之间传递信息，但“信息已经发出”并不等于业务已经完成。图 48 将三类场景归结为一个共同问题：当不同服务器看到的状态还不相同时，一次跨服操作在什么条件下才算完成？

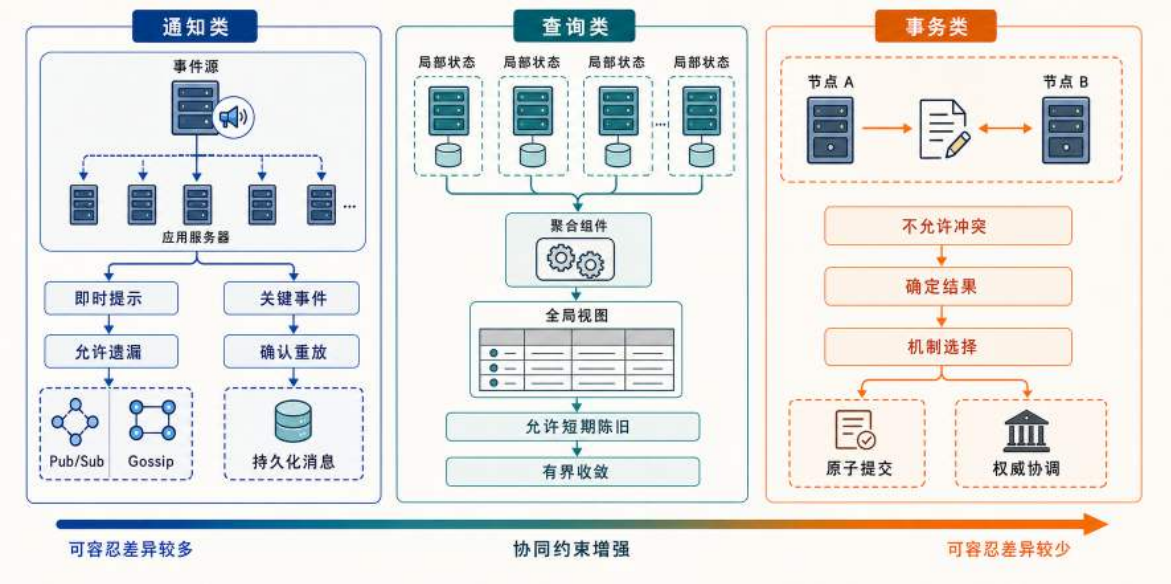


图 48: 跨服交互的典型约束与机制选择：通知类按事件重要程度选择即时传播或可靠消息，查询类通过有界收敛形成全局视图，事务类通过原子提交或权威协调形成确定结果。

对于通知类，事件在源服务器产生后已经成立，其他服务器是否必须收到取决于事件的重要程度；对于查询类，全局视图可以暂时落后，但必须在限定时间内更新；对于事务类，参与节点尚未形成同一个结果时，操作就不能算完成。完成条件的不同，分别引出了事件传播、视图聚合和事务协同三类机制。下面依次讨论这些机制的工作方式、适用条件与实现选择。

4.3.2 通知类：事件如何传播

通知类交互传播的是已经确定的事实，但不同事件需要的送达保证并不相同。允许偶尔遗漏的即时提示重视低延迟和覆盖范围。遗漏会影响后续业务的事件，还要进入可持久化、可确认、可重放的消息路径。两类传播都要控制重复消息和广播开销。

以在线游戏为例，某个地图分片生成世界 Boss 后，可以用即时通知让其他地图分片尽快显示公告。活动入口和倒计时属于业务状态，要从可持久化的权威记录中读取。

即时公告要尽快到达较多服务器，又不能占用过多网络 and 计算资源。同一事件可能从多条路径重复到达，服务器不能重复显示公告；服务器数量增加后，事件源也不宜为每个目标长期维护一条广播连接。

转发责任有两种常见安排。可以让一个专门的中间节点接收事件并负责复制，应用服务器只需要发布和订阅；也可以让已经收到事件的服务器继续转发，使传播压力分散到多个节点。前一

种方式路径短、规则集中，但中心节点会承受可用性和容量压力；后一种方式减少了中心依赖，却必须处理重复消息、传播范围和停止条件。下面分别对这两种责任分配方式详细展开。

集中式转发：从事件源到订阅者 最直接的设计是增加一个消息中间件，让所有应用服务器订阅同一个事件频道。产生世界 Boss 的服务器把事件发布到频道，中间件再把同一条消息转发给当前在线的订阅者，这种通信方式称为发布/订阅（Publish/Subscribe）。

以 Redis Pub/Sub 为例，流程如下：所有应用服务器启动后，都向 Redis 订阅一个名为 `world_events` 的频道。当服务器 A 上刷新了世界 Boss，服务器 A 向 `world_events` 频道发布一条消息，内容包含事件类型、Boss ID、刷新位置和时间戳。Redis 收到消息后，立即将它推送给当前保持订阅连接的服务器。各服务器收到后更新本地公告。活动入口和计时仍以权威活动状态为准。

这个过程可以拆成两个长期运行的组件：产生事件的服务器负责发布消息，其他服务器启动时建立订阅并持续等待新消息。订阅端收到事件后，还要在本地显示公告；Redis 只负责转发消息，不负责修改活动状态。下面的伪代码把发布、订阅和事件处理连成一次消息流转：

Redis Pub/Sub 的发布与订阅流程

```
1  const channel = "global_events"
2
3  // 事件源：全局活动在服务器 A 触发
4  func publishGlobalEvent(redis Redis, entity Entity) {
5      event := GlobalEvent{
6          ID:          newEventID(),
7          EntityID:    entity.ID,
8          Position:    entity.Position,
9          Timestamp:   now(),
10     }
11     redis.Publish(channel, encode(event))
12 }
13
14 // 其他应用服务器启动时订阅频道
15 func subscribeGlobalEvents(redis Redis) {
16     messages := redis.Subscribe(channel)
17     for message := range messages {
18         event := decode(message.Payload)
19         displayNotification(event)
20     }
21 }
```

发布者只需发送一次消息，复制和分发由中间件完成，应用服务器之间不需要彼此维护广播连接。

集中式消息分发的优势在于实现简单、延迟可控。消息从发布者到当前在线的订阅者只经过一个中间节点，转发路径较短。新增一台应用服务器时，只需让它连接到同一个 Redis 实例并订阅相应频道，不需要修改其他服务器的配置。

集中式方案也有明显的局限。中间件不可用时，所有广播都会中断；服务器数量增多后，连接数和消息复制流量也会集中在这个节点上。假设集群有 100 台应用服务器，一条广播消息需要被 Redis 复制并推送 100 次；如果每秒产生多条广播事件，Redis 的网络出口带宽和 CPU 会成为瓶颈。Redis Pub/Sub 也不保存离线期间错过的消息；需要回放的关键事件应进入持久化消息日志，而不是只依赖即时频道。如果希望减轻中心节点的连接和复制压力，可以把转发责任交给已经收到事件的应用服务器，由它们继续向其他节点传播，这就是去中心化扩散。

去中心化扩散：让接收者继续传播 在这种模式下，服务器收到事件后既完成本地处理，也承担下一轮传播任务。事件源不必直接连接集群中的每个节点，传播压力会随着知情节点数量增加而分散。

Gossip 协议（也称流行病传播协议，Epidemic Protocol）²⁶就是这种去中心化广播方法的代表。它不要求事件源一次性联系所有节点，而是让已经收到事件的节点继续向少量随机目标转发。新收到事件的节点在后续轮次中也参与传播，传播能力因而会随着知情节点数量增加而扩大。

随机选点不要求每个节点保存完整的集群成员列表，但节点至少要维护一组可以联系的成员。系统也可以通过另一套 Gossip 过程更新这份成员视图，使节点逐步发现新成员和失效成员。要判断这种方案是否可用，既要考察覆盖范围如何随轮次扩大，也要处理随机转发带来的重复到达和传播停止问题。图 49 以 $\text{fanout}=2$ 的一次扩散为例，连续呈现了同一组节点的状态变化。

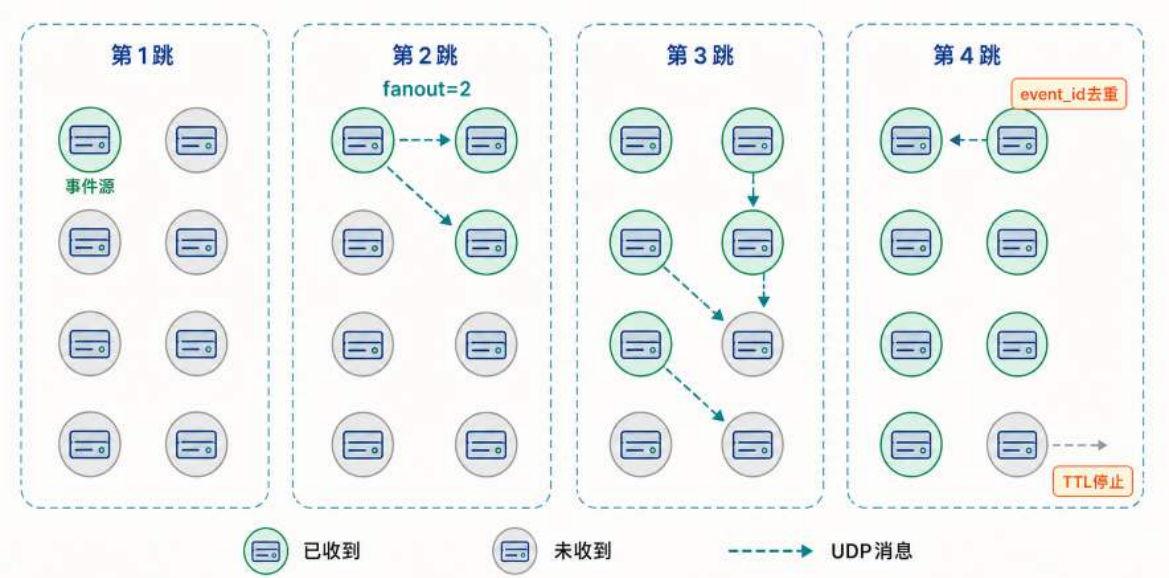


图 49: Gossip 逐跳扩散事件：已收到事件的节点在每轮选择少量随机目标转发，覆盖范围逐步扩大；节点根据事件 ID 避免重复执行业务，并通过 TTL 停止传播过期事件。图中的轮次和覆盖结果仅为示意，不表示固定轮次必然覆盖全部节点。

²⁶Gossip/Epidemic 协议的经典分析见 A. Demers et al., “Epidemic Algorithms for Replicated Database Maintenance”, PODC 1987.

事件产生时，只有事件源处于“已收到”状态。传播扇出量 $\text{fanout}=2$ 表示一个知情节点每轮选择两个随机目标；第一批目标收到事件后，会在后续轮次中继续选择新的目标。转发者由一个增加到多个，覆盖范围随之扩大，而事件源不必直接连接集群中的全部节点。

随机选点并不保证每次发送都命中尚未收到事件的节点。同一事件可能沿不同路径再次到达知情节点，固定轮次结束后也可能仍有节点没有收到消息。事件 ID 使节点在重复收到事件时不再重复执行业务，TTL 则使过期事件停止参加后续传播。二者分别控制重复处理和传播停止，但不能把概率扩散变成严格送达。

fanout 越大，消息扩散越快，但重复消息和网络流量也越多； fanout 越小，带宽开销较低，收敛时间则会变长。在一个 N 台节点的集群中，如果每轮每个已经收到事件的节点向 b 台随机节点转发，消息覆盖大部分节点所需的轮次通常随 $\log N$ 增长。当 $N = 100$ 、 $b = 2$ 、每一跳的转发延迟约为 200 毫秒时，一条消息在理想随机选择和节点持续可达的条件下，可能在 1~2 秒内到达大多数节点。这个数值只用于说明传播量级，不是覆盖全部节点的时延上界。对于公告和全局事件通知来说，这个收敛速度通常是可以接受的。

各节点按自己的周期执行转发，不需要等待全体确认。一次丢包通常不会中断整个传播过程，因为其他知情节点和后续轮次还会提供新的发送机会。这种冗余提高了覆盖概率，但仍不能保证每个节点都收到消息。

实现时，每个节点需要分别维护“该事件是否已经完成本地处理”和“该事件是否仍需继续传播”两类状态。前者由事件 ID 判断，后者由过期时间或剩余跳数判断。下面的伪代码将这两个判断分别放入接收流程和周期转发流程。

Gossip 的转发判断

```
1 func OnEvent(event Event) {
2     if now().After(event.ExpiresAt) {
3         return
4     }
5
6     // 重复到达时不重复执行业务；处理失败则等待下次重试
7     if err := applyOnce(event.ID, event); err != nil {
8         return
9     }
10
11     activeEvents.PutIfAbsent(event.ID, event)
12 }
13
14 func GossipRound() {
15     activeEvents.DeleteExpired(now())
16     for _, event := range activeEvents.All() {
17         for _, peer := range pickRandomPeers(fanout) {
18             go gossipTo(peer, event)
19         }
20     }
21 }
```

`applyOnce(event.ID, event)` 负责本地幂等处理。事件 ID 的记录和业务更新应在同一个可靠步骤中完成；如果只能分步执行，业务更新本身也要允许幂等重试，避免进程在两步之间崩溃后留下错误状态。`activeEvents` 保存仍需传播的事件，`GossipRound` 周期性地为它们选择新目标。这里用 `ExpiresAt` 表示绝对过期时间；如果实现采用剩余跳数，每次转发时还要将跳数减一。待转发状态默认保存在内存中。节点重启后如果还要继续传播，就要持久化这部分状态。即使如此，Gossip 提供的仍是概率覆盖；严格送达还需要持久化消息、确认或重放。

Gossip 并不依赖单一的中心转发节点。任一普通节点宕机后，其他已收到消息的节点仍可以沿剩余路径继续传播，广播不会因为一个中心转发节点失效而整体中断。当集群规模很大（数百台服务器）时，Gossip 会把转发压力分散到多个参与节点，通常比集中式方案更少依赖单一中间件出口；但随机选点和节点能力差异并不保证每个节点承担完全相同的流量。Gossip 的劣势是延迟不如集中式可控，且由于消息沿随机路径扩散，不同节点收到同一组消息的顺序可能不同，因此不适合需要严格有序的场景。

实际系统可以根据事件的送达要求组合使用这两种方式。单个区域内的活动公告适合集中式 Pub/Sub，路径短，实现简单；跨区域节点的存活状态探测会周期性地产生新结果，可以用 Gossip 分散转发压力。

两类事件的差别在于，漏掉一次通知后，后续消息能否纠正当前状态。某个节点没有收到一轮存活探测结果，仍有机会在后续轮次获得最新结果，因此单次漏达通常不会长期影响判断。成员加入或移出、用户权限变更则不同：每次变更都会独立改变系统状态，漏掉其中一次就可能使节点长期保留错误结果，因而不能只依赖 Gossip 传播。

对于这类不能遗漏的事件，系统应先把最终结果写入数据库或持久化事件日志，再把它作为通知传播。接收方处理成功后返回确认；未确认或暂时离线的节点恢复后，可以重放遗漏事件，或者读取数据库中的最新结果完成补齐。

通知类交互让相关服务器尽快知道一件已经发生的事实。当用户打开跨服报名列表时，系统面对的则是持续变化的多份局部状态：各服的报名和退出可能先后发生，用户需要看到的是由这些状态汇总而成的全局视图。这个视图不必在所有服务器上同时更新，但不能长期停留在旧状态，查询类交互讨论的便是这个问题。

4.3.3 查询类：分散状态如何汇成一致视图

查询类交互需要把多个服务器上的局部数据组成一个读取视图。以在线游戏为例，跨服擂台开放报名后，每台应用服务器最先知道的是本服参赛者，用户界面却需要显示全服可挑战对象。如果每次打开列表都实时询问所有服务器，请求延迟会由最慢节点决定，任一服务器超时还会使列表缺少一部分用户。

报名列表通常允许几秒钟的刷新延迟。如果上报、传输和汇总都能在规定期限内继续运行，用户后续刷新时就能看到更新后的名单。这正是本节开头提到的有界收敛：读取视图可以短暂落后，但落后时间必须有明确上限。因此可以沿用前文全服排行榜的异步汇总思路，把“每次打开列表都实时询问所有服务器”换成“后台定期汇总、用户只读缓存”。

各应用服务器定期上报本服的报名快照，或者在用户报名、退出时发送变更事件；汇总服务把这些局部状态合并成一份全服视图写入缓存。这条汇总路径可以拆成两个长期运行的组件：各服负责周期性上报自己的报名者，汇总服务负责周期性合并并刷新缓存。

跨服报名列表的异步汇总流程

```
1 // 各应用服务器：周期性上报本服当前报名者
2 func reportRegistrations(hub Aggregator, server ServerID) {
3     for range ticker(reportInterval) {
4         snapshot := localRegistrations()
5         hub.Report(server, snapshot, now())
6     }
7 }
8
9 // 汇总服务：周期性合并各服快照，全量写入缓存
10 func aggregateLoop(hub Aggregator, cache Cache) {
11     for range ticker(refreshInterval) {
12         var view []Entry
13         for server, report := range hub.Latest() {
14             if expired(report.Time) {
15                 continue // 超过刷新期限未上报，视为过期，丢弃
16             }
17             view = append(view, report.Entries...)
18         }
19         cache.Store("arena_list", view) // 每轮全量覆盖
20     }
21 }
```

代码中的 `hub.Latest()` 取每台服务器最近一次上报，用更新时间区分新旧，让新快照覆盖旧快照，而不是让先后到达的上报互相错乱。`expired(report.Time)` 对应过期标记：某台服务器超过刷新期限没有上报，它的报名者就从视图中移除，已经退出或掉线的失效对手不会长期留在列表里。

关键在最后一行：汇总服务每一轮都重新全量生成视图，而不是在旧视图上增量修补。这样即使某次上报因为网络原因丢失或乱序，也只影响当前这一轮的结果，下一周期就会被重新汇总的数据覆盖。当各服务器按固定周期持续上报、网络延迟存在可接受上限、汇总服务持续可用，并且丢失的上报会在下一周期被新快照替代时，陈旧时间才可以近似限制在“一个上报周期、一次有界传输延迟和一个汇总刷新周期”之内。这些成立条件一旦被节点长期失联或汇总服务故障破坏，系统只能剔除过期数据或进入降级状态，不能继续宣称视图会在原上界内收敛。

刷新周期越短，列表越接近实时，后台汇总和缓存写入也越频繁；周期越长，系统开销越低，用户看到旧名单的时间也越长。跨服报名列表与图 44 中的物化榜单采用了相同思想，区别只在于数据来源从 PostgreSQL 分片变成了多个应用服务器。在擂台系统中，协调服务器接收各服报名变更并维护权威参赛状态，缓存只承担列表浏览的读取视图；报名和退出是否真正生效，仍由协调服务器判断。

4.3.4 事务类：跨节点事务如何保证一致

上一小节中的全局参赛者列表只是供用户浏览的读取视图，允许短暂陈旧。甲从列表中选择乙并发起挑战后，系统开始改变双方的参赛状态、积分和战绩，交互也由查询进入事务处理。此时需要分别解决两个问题：谁来确认双方仍可参战并生成唯一结果；结果确定后，怎样保证服务器 A 和服务器 B 最终都按这份结果更新，并避免故障或重试造成遗漏和重复执行。

为聚焦跨服协调，本例将战斗过程简化为根据报名时上报的状态快照计算胜负。甲由服务器 A 管理，乙由服务器 B 管理。如果两台服务器分别依据本地状态接受挑战，并发请求可能同时把同一名参与者判断为空闲，使其进入多场比赛。因此，系统首先要把参战资格检查和正式结果生成收束到一个裁决位置；结果如何可靠地落实到两台服务器，则在后文继续解决。

唯一裁决点：协调服务器与本服代理 参赛报名时，各应用服务器把参与者标识、所在节点和状态快照上报给协调服务器，由协调服务器维护全局参赛状态。服务器 A 收到甲挑战乙的请求后，不根据本地列表直接判定，而是由本地代理把请求转发给协调服务器。

协调服务器需要把资格检查与状态占用作为一次不可分割的操作：只有甲、乙仍在报名且处于空闲状态时，才同时把双方标记为“交互中”。这样，后到的并发请求会发现甲或乙已被占用，不能再次接受同一参与者；随后协调服务器为本场比赛生成操作编号 M1024，并根据报名时保存的状态快照形成一份正式结果。

协调服务器把这份结果分别发送给服务器 A 和服务器 B，两台应用服务器只更新各自参与者的积分和战绩，不再根据本地状态重新判断胜负。正式结果、通知任务和各服的执行记录都携带同一个 `match_id`，以便识别它们属于同一次挑战。挑战请求从服务器 A 经本地代理上行到协调服务器，唯一结果再分别下发到服务器 A 和服务器 B，由此形成图 50 所示的“单点裁决、多服执行”结构。

这样就可以把裁决权与执行权分开。来自不同区域的挑战都要先经过协调服务器；一个请求占用甲或乙后，后续并发请求会在同一处发现其状态已经改变。应用服务器不再独立裁决，同一场比赛也只接受协调服务器生成的正式结果。

单点裁决解决了“谁生成正式结果”的问题，却没有保证结果一定能够落实到两台服务器。图中的两条结果只表示消息应当发往服务器 A 和服务器 B，并不表示两条消息都已经送达并执行。前文的异步记录更新方案把持久化队列中的结果事件作为后台更新依据，请求主链路不再另外保存一份正式结果。当前的跨服竞技场景则有不同要求：协调服务器既要在本地数据库中保存正式比赛结果，作为后续查询和故障恢复的依据，又要通知服务器 A 和服务器 B 更新各自参与者的积分与战绩。

保存比赛结果属于本地数据库写入，发送通知属于跨节点网络操作。这两个动作由不同系统完成，不能直接放进同一个数据库事务。无论先执行哪一个动作，服务器在两步之间发生故障，都可能只完成其中一项。

事务性发件箱：让裁决结果与待发消息原子提交 事务性发件箱 (Transactional Outbox) 的核心做法，是把“稍后要发送的通知”先写成与比赛结果同库的待发记录。为保存这两类记录，

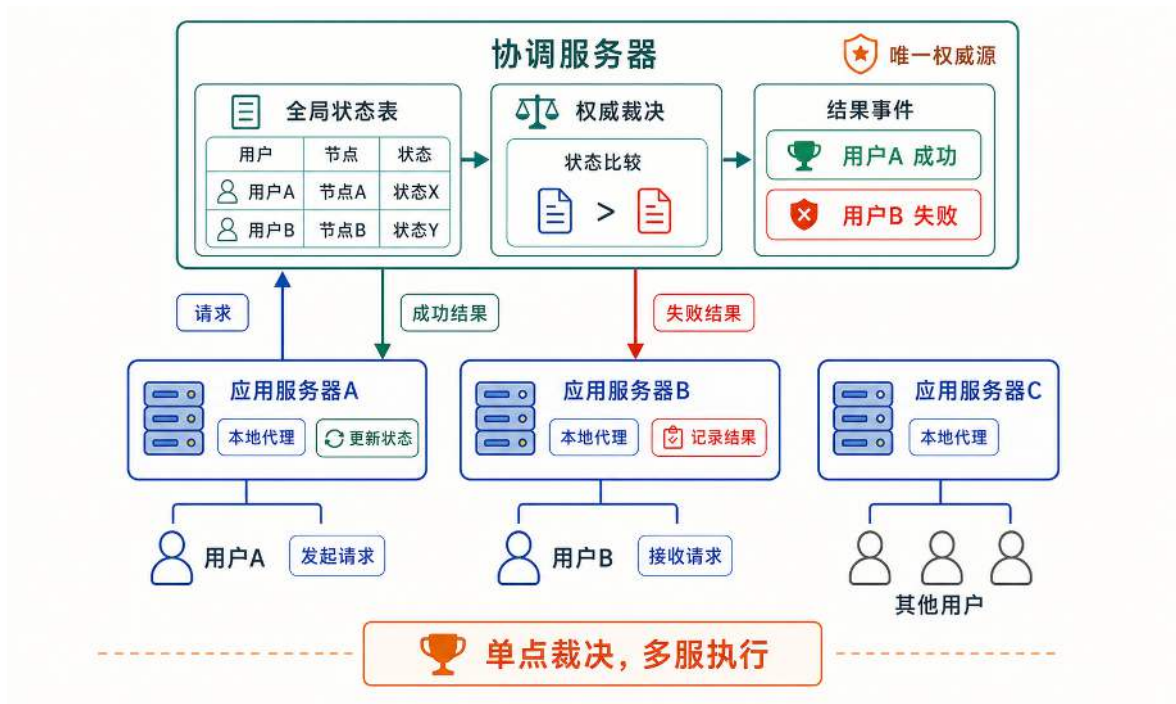


图 50: 挑战请求由本服应用服务器上到协调服务器；协调服务器依据全局参赛状态生成唯一结果，再将结果下发到相关应用服务器执行。

协调服务器会在本地数据库中建立两张表：比赛结果表 `match_results` 保存已经确定的裁决结果，发件箱表 `outbox` 保存尚待发送的通知任务。协调服务器裁决“甲获胜、乙失败”后，在一个本地事务中完成三项写入：向结果表写入比赛结果，再向发件箱表写入通知服务器 A 加分和通知服务器 B 记录败场的两条任务。

只有三项写入都成功，事务才会提交；任一写入失败，三项写入全部撤销。这样，只要正式结果已经保存，后续通知双方的任务就不会缺失。

不过，事务提交只保证结果和待发任务已经保存，并不表示两台服务器已经更新。提交之后，每条任务还要经过发送、处理和确认；未收到确认的任务必须继续保留并重试，已经处理过的任务则不能因重试而重复执行。图 51 将这一过程展开为“一次提交、首次派发和失败重试”三个连续阶段。

图中第一阶段利用原子性保证比赛结果与两条待发任务不会分离；持久性则保证协调服务器重启后仍能从数据库中恢复尚未完成的任务。

随后，发件箱派发程序（Outbox Dispatcher）逐条读取待发任务并异步发送。假设服务器 A 完成本地更新并返回处理确认，Dispatcher 就把发往 A 的任务标记为“已发送”；服务器 B 暂时不可达或没有返回确认时，发往 B 的任务仍保持“待发送”，无需撤销服务器 A 已经完成的更新。

Dispatcher 会在后续扫描中重试发往服务器 B 的任务。由于上一次发送可能已经执行、只是确认丢失，同一结果可能多次到达；服务器 B 在更新战绩前检查 `match_id`，已经处理过 M1024 就只返回原有结果，不再重复更新。这种持续重试直到收到确认、同时允许消息重复到达的方式称为至少一次投递（At-Least-Once Delivery）。

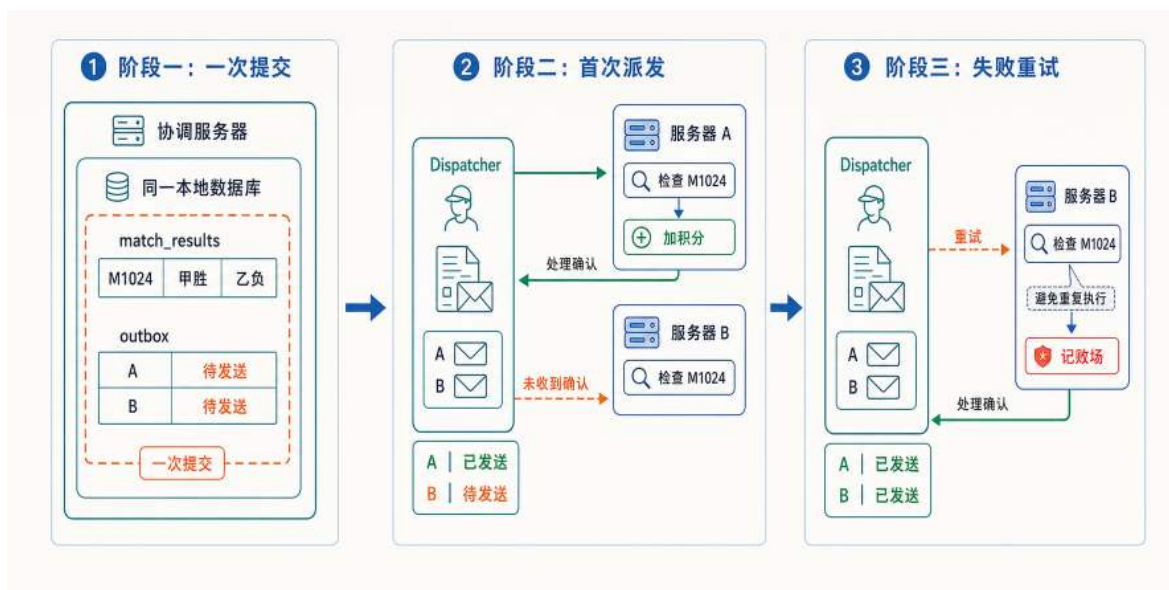


图 51: 事务性发件箱先共同提交比赛结果与两条待发任务，再逐个投递：服务器 A 确认后对应任务变为“已发送”，服务器 B 未确认时仍保持“待发送”，后续重试通过 match_id 避免重复执行。

整个过程采用两类正确性保障：本地事务保证结果与通知任务不会分离，处理确认、失败重试和幂等检查共同推动远端状态最终完成更新。事务性发件箱因此保证通知任务不会被遗忘，但不会把两台远端服务器纳入一次同步数据库提交。

这与第 4.2.6 节中的跨分片 2PC 形成两种不同的一致性边界。2PC 适用于同一个业务事实必须在多个数据库提交点上原子成立的场景，但任一参与者迟迟不能完成都会阻塞整条链路。跨服对战先把裁决事实收束到协调服务器的本地数据库，再把积分、奖励等派生状态放到异步传播阶段，从而避免让远端应用服务器共同进入一次同步提交。这种方案依赖一个业务前提：双方服务器可以在短时间内先后收到结果；如果所有远端状态都必须在响应用户之前同步生效，就仍需采用更强的跨节点协调机制。

跨节点事务由此形成两层正确性保证：唯一裁决点保证同一次操作只产生一份权威结果，事务性发件箱保证这份结果能够被持续重试并最终落实到相关服务器。权威结果的原子提交被限制在协调服务器的本地事务内，跨服派生状态则通过可靠消息和幂等重试逐步收敛。这种结构不仅适用于竞技场，也适用于其他需要“单一权威节点生成结果、多个执行节点落实更新”的跨节点事务。

这套设计仍留下一个尚未解决的问题：擂台协调服务器本身是唯一裁决位置。一旦它不可用，所有区域都无法提交报名和挑战；如果直接部署多个能够独立裁决的副本，又会重新出现多份结果。如何让多个副本在节点故障后仍沿着同一份裁决历史继续服务，构成下一节“容错”的起点。

4.4 容错：冗余、复制与一致性

均衡解决了“怎样分配”，协同解决了“怎样互动”，分布式系统还必须面对第三类问题：当节点故障、网络中断或数据损坏发生时，系统如何维持服务并恢复到正确状态？

分片和多机部署提高了系统的总容量，但节点数量增加后，系统中至少出现一台故障机器的概率也会升高。硬件损坏、进程崩溃和网络中断不会因为部署了更多机器而消失。多机容错能够改变的是故障发生后的结果：先避免故障沿共享依赖扩散，再为受影响的服务保留继续运行或恢复的路径。

第一个任务是控制故障影响范围。多台机器怎样组成请求路径，决定了一台机器故障后，影响会扩散到所有请求，还是被限制在部分用户。从故障传播方式看，请求依赖关系可以抽象为两种模型：所有请求共享必经路径的串联依赖，以及不同用户组沿独立分片路径访问的分片隔离。二者并不是互斥的部署方案，真实系统通常会在每条分片路径内部继续串联多个服务节点。

在串联依赖模型中，一次服务依次经过多个节点，每个节点都是所有用户请求的必经路径。任意节点停止工作，整条路径就中断，所有用户的请求因此共享同一个故障传播路径。在各机器故障相互独立的理想化假设下，如果每台机器的可用性为 99.9%，且 100 台机器串联、任意一台不可用都会使请求失败，那么整体可用性只有 $99.9\%^{100} \approx 90.5\%$ ；节点越多，整条路径保持可用的概率越低。

在分片隔离模型中，请求先按用户归属选择目标分片，只依赖负责该用户组的节点。比如当分片 3 的主库发生故障时，分片 1、2、4、5 的请求路径不经过该节点，仍可继续处理各自用户的请求；故障影响被限制在分片 3 负责的用户组内，而不会扩散为全服中断。

图 52 将故障发生后的两个任务放在同一幅图中：上半部分说明串联依赖如何使单点故障扩散为全局中断；下半部分先用分片隔离把影响限制在分片 3，再由该分片内部的备用副本接管服务。

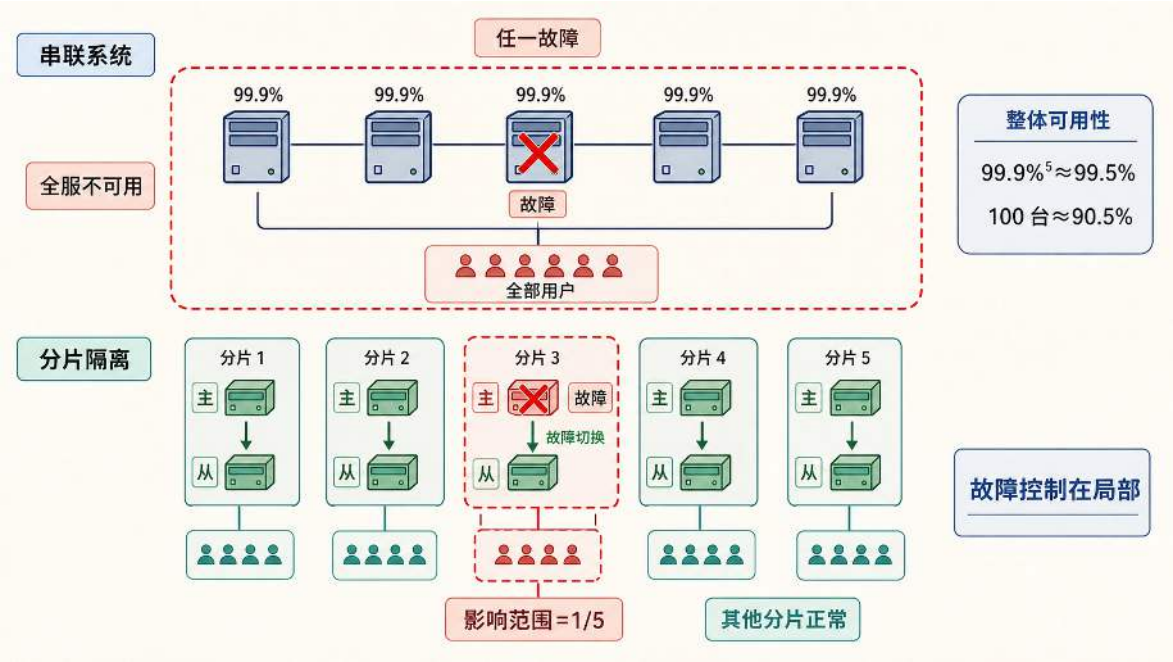


图 52: 无冗余的串联路径会把必经节点故障扩散为全服中断；分片隔离先把影响限制在对应用户组，备用副本再在分片内部接管服务。

分片隔离只完成了第一个任务，并没有恢复发生故障的分片。如果分片 3 只有一个主库且没有备用副本，其他分片虽然仍可运行，分片 3 负责的用户却只能等待主库恢复，无法继续读写数

据。第二个任务是让受影响的服务重新可用，因此系统还要为每个局部单元维护数据足够新、能够接管请求的冗余副本，并在故障发生后完成检测和切换。

冗余能够带来多大收益，也可以先用一个理想化模型估算。假设同一服务有三个故障相互独立的副本，而且任意一个存活副本都能立即、正确地接管服务，那么至少一个副本可用的概率为 $1 - (1 - 99.9\%)^3 = 99.999999\%$ 。即使在理想条件下，可用性仍未达到 100%，三个副本仍然可能同时不可用。现实系统中，多个副本可能同时受到机房断电或网络中断的影响；即使仍有副本存活，也可能因复制滞后、故障检测和请求切换而无法立即接管服务，使用户实际感知的可用性进一步低于上述理想值。上述计算因而只能说明并联冗余的数学潜力，并不是三副本状态服务的实际可用性承诺。增加副本只能降低服务完全中断的概率，不能消除故障本身，同时还会带来多倍资源成本，以及副本同步、一致性维护和故障切换等新问题。

多机架构的容错因此不能依赖“故障不会发生”，而要在故障发生后先把影响控制在局部，再为受影响的服务建立继续运行或恢复到正确状态的路径。本节先讨论三类保护机制：主从复制维持可接管的在线副本，备份恢复保留历史时间点用于回退错误操作，共识算法保证协调服务在节点故障后沿同一操作历史继续服务；最后再分析这些机制在一致性、可用性和响应延迟之间需要作出的取舍。

4.4.1 主从复制与故障切换

分片把数据分散到多台机器后，单个分片内部仍可能只有一台数据库提供服务；一旦这台数据库不可用，属于该分片的读写请求仍会被阻塞或失败。为了让同一份数据在原节点故障后仍有副本可以接管，系统可以在一个分片内保存多个持续同步的数据副本，并指定其中一个副本负责产生权威写入。这种结构称为主从复制（Master-Slave Replication）。

主从复制通常包含一个主库（Master）和一个或多个从库（Slave）。主库接收写请求并确定数据变更的先后顺序，从库持续跟随这些变更，形成可供读取或故障接管的数据副本。较新的技术文档也常将二者称为 primary 和 replica，本节代码示例采用后一组命名。从库有两种常见用途：一种是平时不处理业务请求，只在主库故障时接管；另一种是在正常运行时分担一部分读请求。后一种做法把写请求集中到主库，把部分读请求交给从库，称为读写分离。主从复制并不要求一定采用读写分离，系统需要根据读取压力和一致性要求决定是否启用。

无论从库是否承担读请求，主从复制都需要连续解决三个问题：正常请求应当进入哪个副本，主库产生的数据变更怎样传到从库，以及主库失效后怎样把写入入口切换到可接管的从库。图 53 将这三个问题组织为正常请求路径、状态复制路径和故障接管过程，为后面的机制分析提供整体结构。

沿着图中的正常请求路径，写请求统一进入主库，由主库产生该分片的权威数据。如果系统还希望利用从库分担读取压力，可以把登录时读取用户信息、查看配置和刷新排行榜等普通读请求分配给不同从库，而把写请求继续留在主库。这种读写分离可以避免主库同时承担全部查询和写入，应用层或数据库代理可以依据轮询、负载指标或连接池策略选择从库。

请求能够分流的前提，是从库持续获得主库的最新变更，这对应图中的状态复制路径。主库处理写请求时，会把数据修改及其提交顺序写入复制日志；从库接收这些日志，并在本地按

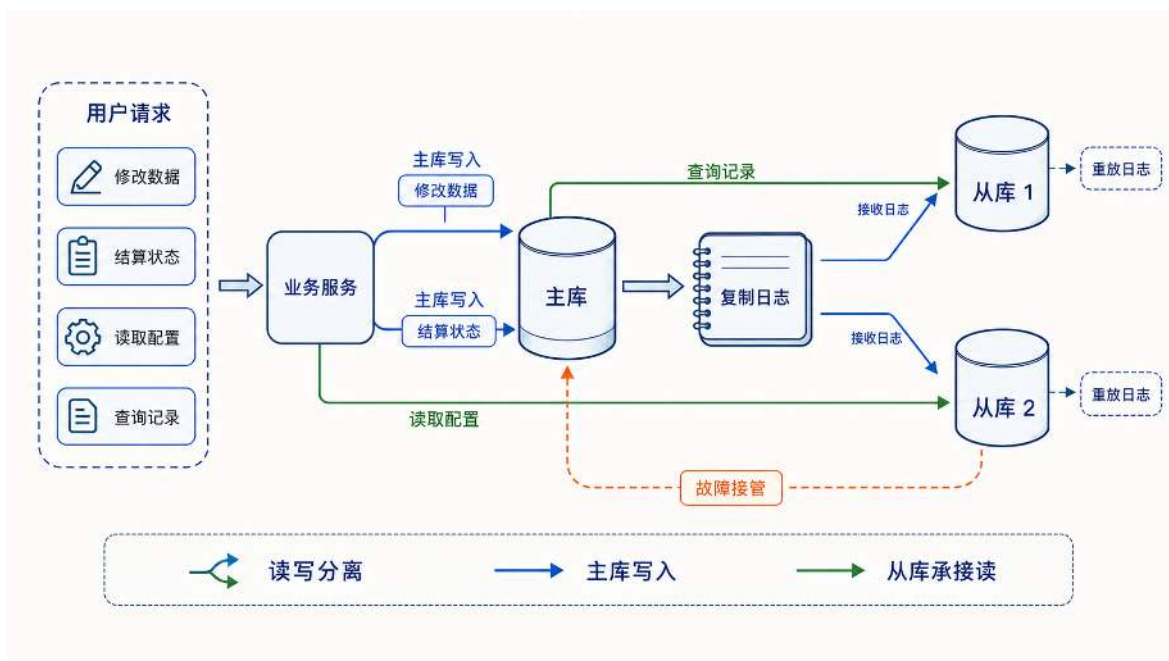


图 53: 主从复制通过请求分流确定主库和从库的正常职责，通过复制日志维持可接管的数据副本，并在主库故障后把写入入口切换到新的主库。

照相同顺序重放，使自身数据逐步追上主库。不同数据库对这类日志有不同命名：MySQL 常用 binary log（二进制日志）记录可复制的数据变更，PostgreSQL 常用 write-ahead log（WAL，预写日志）作为写入恢复和复制的基础。虽然具体格式不同，它们在主从复制中的共同作用都是保留主库的变更顺序，让从库能够重建相同的数据状态。

日志传输和重放都需要时间，因此从库与主库之间通常存在复制延迟。主库已经提交一项写入时，从库可能还没有重放对应日志，暂时保留着写入之前的数据。一旦读请求被分配到这样的从库，读写分离就会进一步带来一致性问题。

以一次写入后立刻读取为例。用户完成写入后，主库已经把更新写入成功；如果界面立刻刷新，而这次读取被分配到尚未重放对应复制日志的从库，用户就可能看到数据仍是旧值。这里的失败点在于：一次写入已经向用户确认成功，随后的读取却没有看到这次写入。这种情况通常称为“读到旧数据”；从一致性的角度看，它破坏的是写后读一致性。

为避免用户在写入成功后立即读到旧数据，系统需要确保随后的读取能够看到这次更新，常见做法有三类。第一，关键读请求直接读主库，例如关键写入后的第一次读取。第二，在一次写入后的短时间内，让同一个用户的相关读请求继续读主库，避免马上读到旧从库。第三，应用层可以关注从库的复制进度：只有当从库已经重放到本次写入之后，才把读请求交给从库。实际业务中，如果不想直接管理复杂的数据库复制进度，也可以把这个判断转换成新旧版本号的比较。例如，用户资料、配置记录或状态都可以带一个随写入递增的版本。可以先把它理解为对象状态的编号：金币余额处于旧状态时是 v41，领取奖励成功后变成 v42。后续读请求如果要求至少读到 v42，从库返回 v41 就说明它还没有重放到这次写入。下面的伪代码展示了这一流程：写入奖励时，主库返回新的资产版本；刷新金币时，读请求先查从库，再用版本号判断是否需要回到主库。

写入后的版本号读写流程

```
1 type Coins struct {
2     Amount int
3     Version int64
4 }
5
6 func ClaimReward(userID int, reward int) int64 {
7     // 主库把金币从 v41 更新到 v42, 并返回新的版本号
8     newVersion := primary.AddCoins(userID, reward)
9     return newVersion
10 }
11
12 func ReadCoins(userID int, minVersion int64) Coins {
13     coins := replica.ReadCoins(userID)
14
15     // 读到 v41, 但本次请求至少需要 v42, 说明从库仍然落后
16     if coins.Version < minVersion {
17         coins = primary.ReadCoins(userID)
18     }
19
20     return coins
21 }
```

这段代码中的 `ClaimReward` 对应写路径:奖励写入主库后,主库返回新的资产版本。`ReadCoins` 对应读路径:它先读从库,因为普通读请求仍然希望由从库承担;随后检查 `coins.Version < minVersion`。这个判断就是写后读一致性的业务表达:如果从库版本低于本次请求要求的最低版本,说明从库仍然落后,读请求需要回到主库。

例如,用户写入前有 100 个单位,资产版本是 `v41`。写入后,主库把数值写成 150,并把版本推进到 `v42`,然后把 `v42` 返回给业务服务。如果用户立刻刷新界面,业务服务会带着“至少读取 `v42`”这个要求访问从库。此时从库如果还停留在 `v41`,返回的可能仍是 100;业务服务发现版本不够,就改读主库,拿到 `v42` 对应的 150。这样,复制延迟仍然存在,但用户不会在写入成功后马上看到旧数据。

上面的伪代码只写了核心逻辑,真实实现还要进一步处理以下两种情况。第一种情况是,改读主库这一步本身也可能失败。当从库版本不够、读请求转而访问主库时,如果主库此刻正好宕机或响应很慢,这次读取就会卡住或报错。因此这一步不能无限期等待,需要设定超时时间并允许重试;如果重试后确认主库已经不可用,这次读取就应当快速失败,向用户返回“稍后再试”之类的提示,而不是让刷新请求一直悬挂在那里。至于主库本身的恢复,则由系统层面的机制接手。第二种情况是,从库的复制延迟可能长时间偏大。前面说过,从库靠重放主库的复制日志跟进数

据，正常情况下这段延迟很短。但如果主库写入太快，或者从库与主库之间的网络出问题，从库重放的进度就会明显落在后面，复制延迟持续偏大。这时几乎每个带版本要求的读都会发现从库版本不够，于是全部转去读主库，从库分担读压力的作用就失效了，主库反而被这些额外的读请求压满。为避免这种情况，实际系统会持续测量各个从库的复制延迟，一旦超过设定阈值就告警并介入处理。对于本来就允许数据短暂陈旧的读取场景，还可以在延迟过高时主动放宽要求，接受从库返回的旧版本数据，用可控的陈旧换取主库的稳定。

前面的读写与复制过程发生在主库正常运行期间；主库不可用后，故障接管过程才会启动。系统需要从已有从库中选择一个提升为新主库，把后续写请求切换到接管节点，并让其他从库改为跟随新的主库。这组动作称为故障转移（Failover），它把持续同步的数据副本转变为新的写入口，使该分片能够恢复读写服务。

故障转移的第一项困难，是无法仅凭“联系不上主库”断定它已经停止工作。主库可能已经宕机，也可能仍在运行，只是与监控组件和从库之间的网络已经中断。后一种节点仍在运行、节点组之间却无法通信的状态，称为网络分区（Network Partition）。

如果此时直接提升从库，而旧主库仍能接收部分客户端的写入，系统中就会同时出现两个主库，并形成两份相互冲突的数据。因此，新主库开始写入之前，必须先确认旧主库已经失去写入资格。系统通常通过连续健康检查判断主库是否长时间无响应，并用带有效期的主库身份限制其写入权限；身份到期后，旧主库必须停止写入。若旧主库仍可访问，还可以通过关闭其写入口或隔离节点来强制执行这一限制，这类措施统称为隔离（Fencing）。

在此基础上，一次完整的接管包含四项动作：确认旧主库不能继续写入，选择数据进度合适的从库，将其提升为新主库，并把数据库代理、服务发现或客户端连接切换到新的写入口。其他从库随后改为跟随新主库；旧主库恢复后，也必须先追赶新主库的数据或重新初始化，不能直接恢复写入。PostgreSQL 的 Patroni、pg_auto_failover 和 MySQL 的 Orchestrator 等高可用组件，可以把这些动作组织成自动执行的故障转移流程。²⁷

接管完成后，还要判断最近的写入是否保留下来。如果主库在本地提交后立即向客户端返回成功，再由从库随后复制，这种方式称为异步复制；主库突然故障时，尚未到达接管节点的写入可能丢失。如果主库必须等至少一个指定从库确认收到写入后才返回成功，这种方式称为同步复制；从已确认写入的副本中选择新主库，可以降低已确认写入丢失的风险，但从库响应变慢或不可用时，主库的写入也会等待甚至暂停。因此，系统既要选择复制进度足够新的接管节点，也要根据业务能够容忍的数据丢失程度选择复制方式。

除了确认最近写入能否保留，系统还要关注故障后服务需要多长时间才能恢复。恢复时间包括发现主库失效、提升从库以及切换请求入口所需的时间。自动化不能消除这些步骤，却能减少人工确认、执行命令和修改连接配置造成的额外等待，使接管过程更快且更稳定。

主从复制由此解决的是“当前主库不能继续工作”时如何利用在线副本接管服务。然而，在线副本保存的仍是数据库的当前状态。如果清理脚本误删了用户数据，或者程序缺陷写入了错误余额，数据库会把它们当作合法写入提交，从库也会照常复制。结果是所有副本都进入同一个错

²⁷ 参见 <https://patroni.readthedocs.io/en/latest/faq.html#automatic-failover>、<https://pg-auto-failover.readthedocs.io/> 和 <https://github.com/openark/orchestrator>。

误状态，提升任何从库都无法找回原来的数据。恢复这类错误，需要保存事故发生前的历史状态。

4.4.2 备份与恢复演练

备份把数据库的历史状态保存在独立存储中，使系统能够从错误写入传播之前重新开始恢复。最基础的做法是全量备份（Full Backup），即定期保存整个数据库的数据与结构。例如，系统可以每天凌晨生成一份完整副本，并存入独立的备份服务器或云存储服务。加载全量备份可以直接回到备份生成时的状态，但备份文件较大，生成、传输和恢复都需要较多时间，因此很难频繁执行。

为了在两次全量备份之间保存更多恢复点，系统还可以只记录发生变化的数据，这称为增量备份（Incremental Backup）。它能够减少单次备份的数据量，但恢复时需要组合全量备份和后续增量备份，而且离散的备份点仍未必足够接近误操作发生的时刻。

更细粒度的做法，是保存一份完整的数据基线，并连续记录此后发生的写入。以 PostgreSQL 为例，完整的数据基线称为基础备份（Base Backup），写入历史记录在 WAL 中并持续归档到独立存储。恢复时，系统先加载基础备份，再按顺序重放 WAL，并在指定时刻之前停止。由此，数据库可以恢复到选定的时间点，这种能力称为时间点恢复（Point-in-Time Recovery, PITR）。

假设某个数据库分片在凌晨 0 点完成基础备份，此后持续归档 WAL；下午 3 点 20 分，一条错误脚本删除了部分用户的配置数据。如果归档日志完整，恢复程序可以加载凌晨的基础备份，再把 WAL 重放到 3 点 19 分 59 秒，使数据库停在误删发生之前。

一次恢复不能只用“是否成功”来判断，还必须得到两个结果：数据能够回到哪个时刻，以及这份数据何时能够重新对外服务。对于前面的误删场景，恢复程序需要先把日志重放停在错误写入之前，再对恢复后的数据进行校验并切换服务。图 54 将这两个结果放在同一次恢复中：数据时间线用于确定日志在哪里停止，恢复流程线用于确定恢复结果何时能够接替线上服务。

沿数据时间线看，基础备份只提供恢复起点，连续归档的 WAL 才能把数据库逐步推进到误删发生之前。当日志重放在误删记录之前停止时，错误操作就不会进入恢复结果；恢复点与误删时刻之间的距离，就是实际数据丢失窗口。业务能够接受的最大数据丢失窗口称为恢复点目标（Recovery Point Objective, RPO），它会约束 WAL 归档的延迟与完整性。

沿恢复流程线看，得到一个可恢复的数据时间点并不意味着服务已经恢复。系统还要选择并校验备份，在隔离环境中加载基础备份、重放 WAL，对结果进行只读校验，最后才能切换服务。从发现错误到重新提供服务的总耗时，应当不超过恢复时间目标（Recovery Time Objective, RTO）。因此，较小的 RPO 只能说明可恢复的数据更接近事故时刻，并不能保证恢复过程同样迅速；RPO 与 RTO 分别约束数据损失和服务中断，不能相互替代。

仅仅在备份存储中看到基础备份和 WAL 文件，还不能证明这些文件真的能够恢复数据库。基础备份可能已经损坏，WAL 归档中也可能缺少一段日志；即使数据库能够启动，恢复后的用户余额、订单记录等关键数据也可能不完整。因此，系统需要定期在隔离环境中实际执行恢复：加载基础备份，把 WAL 重放到选定时刻，以只读方式启动数据库，并检查关键业务数据是否正确。这类恢复演练既不会覆盖线上分片，又能验证数据库能否恢复到指定时刻、整个恢复过程能否在规定时间内完成。下面的伪代码给出了其中的核心检查。

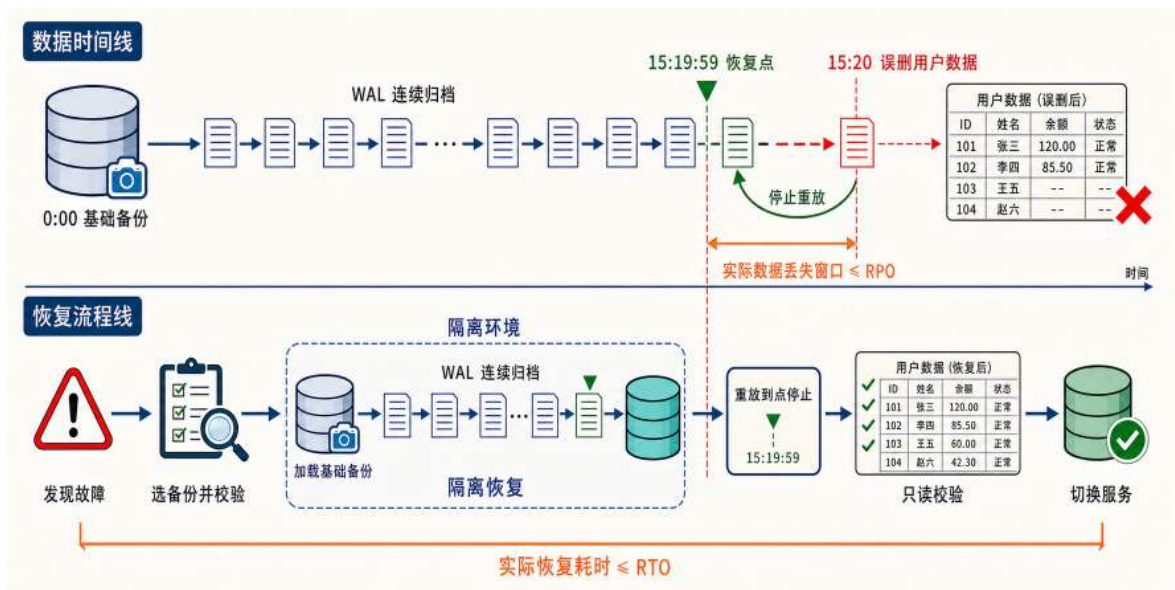


图 54: PITR 将恢复点与恢复过程分开度量：基础备份和连续 WAL 决定数据能够恢复到哪里，备份加载、日志重放、数据校验和服务切换决定数据何时能够重新提供服务。

基于 PITR 的恢复与校验流程

```

1 type RecoveryReport struct {
2     TargetTime time.Time
3     Elapsed     time.Duration
4     Verified    bool
5 }
6
7 func RestoreAndVerify(shardID string, targetTime time.Time) RecoveryReport {
8     startedAt := time.Now()
9
10    // 选择目标时刻之前最近的基础备份
11    base := backupCatalog.LatestBaseBefore(shardID, targetTime)
12
13    // WAL 链存在缺口时，数据库无法连续推进到目标时刻
14    if !walArchive.Continuous(base.EndLSN, targetTime) {
15        return RecoveryReport{TargetTime: targetTime, Verified: false}
16    }
17
18    // 在隔离环境恢复，不覆盖线上分片
19    candidate := recoveryEnv.RestoreBaseBackup(base)
20    candidate.ReplayWALUntil(targetTime)
21    candidate.StartReadOnly()
22
23    // 校验用户资产、订单数量等关键业务约束
24    verified := candidate.VerifyBusinessRules()
25    return RecoveryReport{
26        TargetTime: targetTime,
27        Elapsed:     time.Since(startedAt),
28        Verified:    verified,
29    }
30 }

```


伪代码先根据 `targetTime` 找到目标时刻之前最近的基础备份，再检查从这份备份到目标时刻之间的 WAL 是否完整。只要中间缺少一段日志，数据库就无法连续推进到目标时刻，本次演练应当直接判定失败。日志完整时，系统才会在隔离环境中加载基础备份、重放 WAL，并以只读方式启动恢复实例。最后的业务校验会检查用户资产、订单数量等关键数据；只有数据库能够启动且这些数据正确，`Verified` 才会记录为真。`Elapsed` 记录上述恢复和校验用了多长时间，但它还不包含数据补写和服务切换，因此不是完整的服务中断时间。

演练验证到这里就可以结束，因为这个恢复实例不会接收线上请求。真实事故的目标则是让恢复实例接替原来的数据库。系统通常先暂停受影响分片的新写入，避免恢复期间继续产生变化；随后完成恢复，补回仍应保留的数据并再次校验，最后才把业务连接切换到恢复实例。

恢复操作并不能只撤销那一条错误写入。恢复到误删之前，确实能够去掉错误操作，但恢复点之后发生的正确充值、交易和状态更新也不会出现在恢复结果中。系统需要根据支付流水、订单记录或消息日志找出这些已经生效的业务操作，并把它们重新写入恢复后的数据库，这个过程称为数据补偿。恢复点距离事故越远，需要重新核对和补写的数据通常越多。

因此，缩短数据丢失窗口，需要让 WAL 更及时、完整地归档；缩短服务中断时间，则需要加快备份加载、日志重放、数据校验和连接切换。恢复演练的作用，就是在事故发生之前检查这两项要求能否真正达到，而不是等到线上数据出错后再第一次执行恢复流程。

至此，数据库分片面对三类问题分别有了对应办法：主库故障时由在线从库接管，数据被误删或误改时用备份和 WAL 回到历史状态，恢复流程本身则通过演练提前验证。这些办法保护的只是一个数据库分片的数据和服务，但跨服协调服务还有不同的问题。当协调服务部署多个节点时，每个节点都可能收到新的比赛结果；系统必须保证这些节点对“由谁接收写入”和“哪些结果已经生效”作出相同判断。如果网络中断后两个节点分别把不同结果当成正式结果，后续即使恢复数据库，也无法判断应该保留哪一份操作历史。让多个节点在故障和网络延迟下仍确认唯一能够提交写入的节点，并按同一顺序确认操作，称为分布式共识。Raft 是解决这类问题的经典算法，下面从它为何采用多数派确认开始分析。

4.4.3 从全写到多数派：Raft 共识

假设擂台协调服务部署了三个副本，一场比赛的裁决结果需要被复制到这些副本中，才能在当前节点故障后由其他节点继续处理。最直接的策略是等待三个副本全部写入成功，再向应用服务器确裁决已经成立，这种策略称为“全写”（Write-All）。全写可以使所有副本立即保存相同记录，却把每个副本都变成了写入成功的必要条件：只要一个副本响应缓慢或暂时失联，新的比赛裁决就无法提交。复制本来是为了让部分节点故障时仍能继续服务，全写却使任意单个副本故障都阻塞整个协调服务，因此不能直接作为容错集群的提交规则。

Raft 不再要求全部副本都写入成功，而是把超过半数节点确认作为提交条件，这称为多数派提交。这样，少数节点暂时不可用时，集群仍有继续处理请求的可能；半数及以下的节点则不能独自提交新的结果。

多数派提交只规定一项操作在什么条件下成为正式结果，还不足以构成完整的共识算法。Raft 还需要通过领导者选举确定当前由谁接收写入，通过日志复制让各副本按相同顺序记录操

作，并通过安全性约束保证故障后的新领导者能够继承已经提交的历史。下面依次介绍这三项机制。

Leader 与任期：先确定谁能接收写入 Raft 集群中的节点有三种角色：跟随者 (Follower)、候选者 (Candidate) 和领导者 (Leader)。正常运行时，Leader 负责接收客户端写请求并组织复制；Follower 只响应 Leader 或 Candidate 发来的消息；Candidate 是选举期间的临时角色。对跨服对战来说，挑战请求统一交给当前 Leader，避免多个节点同时决定比赛结果。

Raft 用任期 (Term) 标识一轮选举及其产生的领导关系。任期编号只会递增，可以把它理解为集群内部的逻辑时钟：它不表示现实时间经过了多久，而是用来判断一条消息或一个 Leader 是否已经过期。每条选举和复制消息都会携带发送方的任期；节点发现更高任期后，会更新自己的任期并转为 Follower，收到任期较低请求则直接拒绝。一个任期可能选出一位 Leader，也可能因选票分散而没有选出 Leader，但同一任期内最多只能有一位节点取得多数票。

Leader 通过周期性心跳表明自己仍在工作，Follower 每次收到有效心跳后都会重新启动选举计时。如果等待时间超过选举超时，Follower 无法判断 Leader 是已经宕机还是网络暂时中断，只能认为当前 Leader 已经无法继续组织集群，并转为 Candidate 发起选举。为避免多个 Follower 在同一时刻发起选举并瓜分选票，每个节点会从一个时间区间内随机选择自己的选举超时；通常最先超时的节点能够在其他节点超时之前取得选票并发送新的心跳。选举超时过短，正常的网络延迟也可能频繁触发选举；选举超时过长，Leader 故障后的接管又会变慢，因此需要根据网络延迟和故障恢复目标进行配置。

一个节点需要在选举超时、收到心跳、收到投票请求和收到投票回复时更新自身状态。以下伪代码把这四类事件放在同一流程中，用于说明节点如何推进任期、发起选举并确认 Leader。其中，CandidateLogAtLeastAsNew 只表示候选者的日志新旧检查，具体比较依据将在后面的日志复制与安全性约束中说明。

Raft 任期、心跳与领导者选举流程

```
1 var role = Follower
2 var currentTerm int64 = 0
3 var votedFor = noVote
4 var grantedVotes = NewVoteSet()
5
6 // 收到更高任期时，承认新的选举轮次并退回 Follower
7 func observeHigherTerm(term int64) {
8     if term > currentTerm {
9         currentTerm = term
10        votedFor = noVote
11        role = Follower
12        Persist(currentTerm, votedFor)
13    }
```

```

14 }
15
16 // 非 Leader 节点等待心跳超时后发起一轮新选举
17 func onElectionTimeout() {
18     if role == Leader {
19         return
20     }
21     role = Candidate
22     currentTerm++
23     votedFor = selfID
24     grantedVotes = NewVoteSet(selfID)
25     Persist(currentTerm, votedFor)
26     ResetElectionTimer(RandomTimeout())
27     ParallelRequestVotes(currentTerm, LastLogInfo())
28 }
29
30 // Leader 周期性发送当前任期的心跳
31 func onHeartbeatTick() {
32     if role == Leader {
33         BroadcastHeartbeat(currentTerm)
34     }
35 }
36
37 // 接受有效心跳，并推迟下一次选举
38 func onHeartbeat(term int64) bool {
39     if term < currentTerm {
40         return false
41     }
42     observeHigherTerm(term)
43     role = Follower
44     ResetElectionTimer(RandomTimeout())
45     return true
46 }
47
48 // 同一任期只投一票，并拒绝日志较旧的候选者
49 func onVoteRequest(req VoteRequest) bool {
50     if req.Term < currentTerm {
51         return false

```

```

52     }
53     observeHigherTerm(req.Term)
54
55     canVote := votedFor == noVote ||
56         votedFor == req.CandidateID
57     if canVote && CandidateLogAtLeastAsNew(req) {
58         votedFor = req.CandidateID
59         Persist(currentTerm, votedFor)
60         ResetElectionTimer(RandomTimeout())
61         return true
62     }
63     return false
64 }
65
66 // Candidate 收集本任期的投票，超过半数后成为 Leader
67 func onVoteReply(reply VoteReply) {
68     if reply.Term > currentTerm {
69         observeHigherTerm(reply.Term)
70         return
71     }
72     if role != Candidate || reply.Term != currentTerm {
73         return
74     }
75     if !reply.Granted {
76         return
77     }
78
79     grantedVotes.Add(reply.From)
80     if grantedVotes.Size() > nodeCount/2 {
81         role = Leader
82         BroadcastHeartbeat(currentTerm)
83     }
84 }

```

任期并不是由某个中心节点统一分配的。节点在选举超时后把自己的 `currentTerm` 加一，并把新任期写入投票请求；其他节点收到更高任期后更新本地状态，因此较新的任期会随消息逐步传播到集群中。任期较低的 Leader 或 Candidate 由此会被识别为过期节点。

心跳也不会创建新任期。Candidate 取得多数票后立即发送当前任期的心跳，并在担任 Leader 期间周期性发送；Follower 接受有效心跳后重置随机选举计时器。只要心跳持续到达，Follower

就不会发起新选举；心跳中断并超过选举超时后，新的 Candidate 才会把任期推进一轮。

如果选票分散，没有 Candidate 能够进入成为 Leader 的分支，节点会在下一次超时后以更高任期重试。伪代码中的 `Persist` 还要求节点在投票前保存当前任期和投票对象，使节点重启后仍然遵守“一任期一票”。至此，角色转换、任期、心跳和投票共同组成了一次完整的 Leader 选举。

沿着这段流程，可以进一步分析网络分区时的角色变化。原 Leader 如果与多数节点失去联系，包含多数节点的一侧可以通过新一轮选举产生任期更高的 Leader。旧 Leader 可能暂时仍认为自己负责写入，甚至继续接收客户端请求，但它无法取得多数节点确认，因此这些请求不能提交，也不能作为成功结果返回。网络恢复后，旧 Leader 从消息中看到更高任期便转为 Follower；它在隔离期间产生的未提交记录不会进入集群的正式历史，后续由日志复制过程清理。

日志复制：把写请求变成可恢复的历史 Leader 选出后，外部写请求先写成日志，再由状态机执行。日志才是集群中的操作历史，状态机只是按日志顺序回放之后得到的结果。只要多个节点持有相同日志，并交给确定性的状态机按相同顺序执行，它们就会得到相同状态。这里的确定性是指：初始状态和输入相同，执行结果也相同，不能在执行时临时读取本机时间或生成随机结果。故障恢复只需补齐缺失日志并按顺序回放，不再需要比较两个数据库中哪一份数据更新。

以一次擂台挑战为例，Leader 收到请求后，先把“挑战者、被挑战者、裁决输入和待提交动作”封装为日志条目，追加到本地日志。随后 Leader 将这条日志复制给 Follower。Follower 接收日志时会检查前一条日志的位置和任期是否与自身日志匹配；如果不匹配，就拒绝本次追加，Leader 再回退到更早的位置重试，直到两边日志前缀重新对齐。这项检查用于发现并修复 Follower 与 Leader 之间不一致的日志前缀。不同节点上尚未提交的日志可能暂时冲突，但不能把这些冲突条目同时当作已经提交的操作历史；Leader 会在后续复制中删除或覆盖 Follower 上与权威前缀不一致的未提交条目。

当 Leader 收到多数节点的成功确认后，这条由当前 Leader 在本任期追加的日志被标记为已提交。在擂台场景中，已提交意味着 Leader 可以把裁决结果应用到本地状态机：原子写入 `match_results` 和 `outbox`。日志条目中应带有已经确定的裁决结果和任务标识，不能在各副本执行时重新随机生成。随后 Leader 通过后续心跳把提交位置通知给其他节点，Follower 也按同一顺序应用日志。即使某个 Follower 暂时失联，Leader 仍可在多数节点确认后继续服务；该 Follower 恢复后再通过日志同步补齐缺失记录。各副本都会得到相同的待发任务，但只有当前 Leader 运行 Dispatcher。Leader 切换后，未完成任务可能再次发送，目标服务器仍要用 `match_id` 做幂等检查。

领导者选举和日志复制并不是两条彼此独立的机制，它们会在一次故障切换中连续发生。如图 55 所示，集群由节点 A、B、C 组成，初始 Leader A 失联后，B 参与新一轮选举；日志条目 3 表示新 Leader 接管后收到的一次写操作。节点 B 不能直接接管写入，而要先通过日志新旧检查，并与节点 C 组成投票多数派，成为新任期的 Leader。随后，写请求形成日志条目 3；即使节点 A 仍然延迟，B 和 C 的确认已经构成提交多数派，这条日志仍可提交并应用到状态机。这次三节点故障切换先后形成两个多数派：投票多数派确定新的唯一写入口，提交多数派确认新

的操作已经进入集群历史。二者都要求超过半数节点参与，再结合投票时的日志新旧限制，已经提交的历史才能由故障后的 Leader 继续继承。

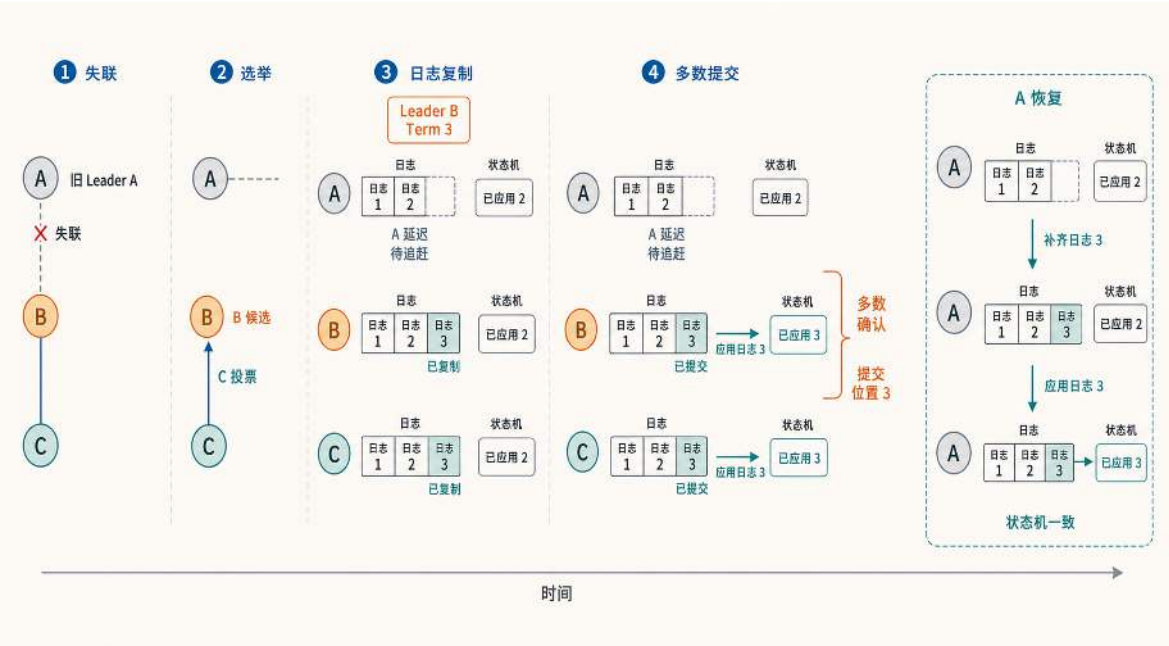


图 55: Raft 中日志复制、提交和应用是三个连续状态：B、C 先复制日志 3，再由多数派确认提交并应用；延迟的节点 A 恢复后才补齐并应用该日志，三个状态机最终一致。

前面借助三节点集群，说明了故障切换会先后经过投票多数派和提交多数派两个环节。下面换一个五节点集群，考察 Leader 崩溃、集群重新选举时，一条已经提交的日志会经历什么。

协调服务由 N1 至 N5 五个 Raft 节点组成，N1 是当前 Leader。甲挑战乙后，N1 生成“甲获胜、乙失败”的裁决，把它写成第 7 条日志，记为 E7，并复制给其他节点。N2 和 N3 回复成功后，N1、N2、N3 三个节点持有 E7，达到五节点集群所需的三节点多数派；N1 因而提交 E7，并把裁决应用到状态机。现在 N1 崩溃，集群要重新选举。问题是：新选出的 Leader 会不会正好是一个没有 E7 的节点，从而让这条已经提交的裁决从集群里消失？

E7 原本保存在 N1、N2、N3 三个节点上，现在 N1 下线，仍然在线并持有 E7 的是 N2 和 N3；还能参与投票的节点是 N2、N3、N4、N5。任何候选者都必须集齐三票才能当选，而从这四个节点里凑出三票，无论如何都会包含 N2 或 N3 中的至少一个——四个节点里选三个，不可能把 N2 和 N3 同时排除在外。换句话说，任何能够当选的新 Leader，它的票源里一定有一个持有 E7 的节点。

这一个节点就足以挡住日志落后的候选者。N2 和 N3 持有 E7，按照投票时的日志新旧规则，它们只会把票投给日志不比自己旧的候选者。N4、N5 没有 E7，日志落在后面，拿不到 N2 或 N3 的票，也就集不齐三票。结合多数派交集和日志新旧检查，缺少 E7 的候选者无法取得足够选票。最终当选的 Leader 必须包含 E7，已经提交的裁决不会因为换了 Leader 而消失。

同一条交集规则也限制了网络分区后的写入范围。五节点集群中，合法 Leader 至少需要三票。网络分区后，只有包含至少三个节点并取得多数确认的节点组能够继续提交；少数派节点组

即使还保留着旧 Leader，也不能提交新的裁决。因此，同一场挑战不会在不同节点组中产生两份合法结果。这种限制用于防止脑裂（split-brain），即不同节点组分别接受写入并形成冲突历史。

把这套机制应用到跨服擂台后，原来的单台协调服务器变成一个复制同一操作历史的协调服务器集群。应用服务器只把挑战请求发送给当前 Leader。裁决操作复制到多数节点并提交后，各副本按顺序应用同一条日志；当前 Leader 的 Dispatcher 再派发 `outbox` 中的任务。Leader 故障时，保存了已提交历史的节点重新选出负责人，新的写请求沿同一条历史继续执行。图 56 只展开当前 Leader 的写入和派发路径，Follower 上相同的本地状态机没有重复画出。右侧给出 Leader 故障后的重新选举。

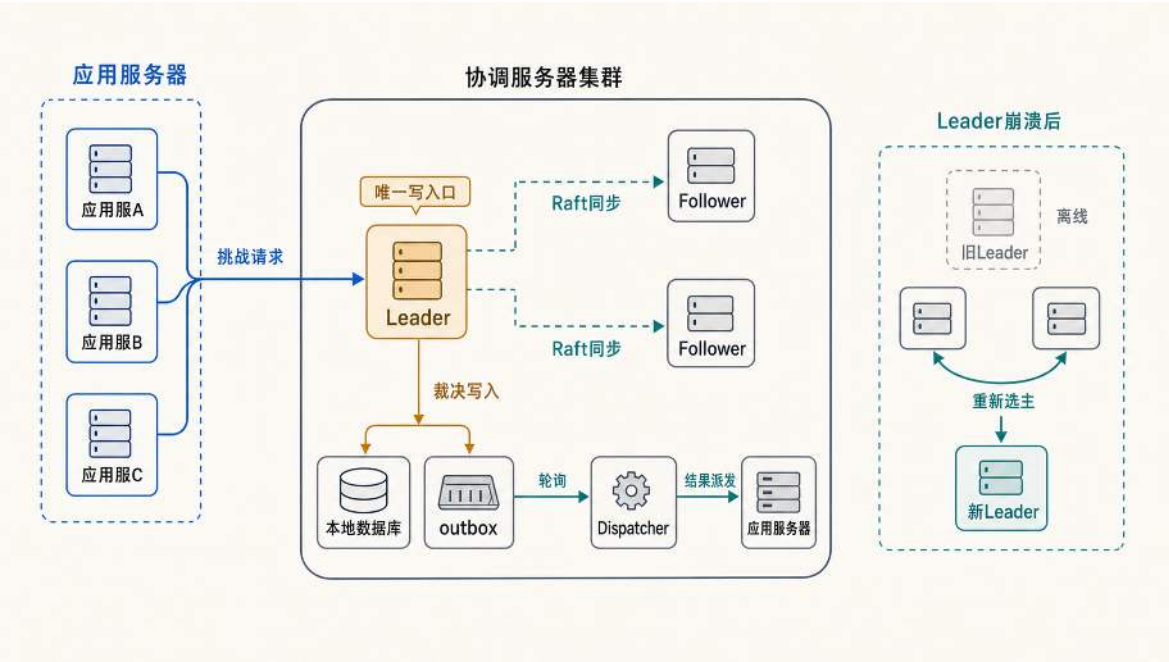


图 56: Raft 将单点裁决扩展为可接管的协调服务器集群：Leader 接收挑战请求并复制裁决日志，提交后派发结果；Follower 保存同一日志，故障后由新 Leader 接续处理。

协调服务器扩展为 Raft 集群后，事务性发件箱仍然从正式裁决提交之后开始工作。Raft 先通过多数派确认哪一条裁决已经进入集群的正式历史。各副本应用这条日志时，各自在本地事务中写入比赛结果和 `outbox`；当前 Leader 的 Dispatcher 再发送待发任务。Leader 切换可能带来重复发送，但不会丢掉已经写入日志的任务，接收端用 `match_id` 去重。

工程取舍：节点数、读路径与恢复成本 N 个节点的 Raft 集群最多可以在 $\lfloor \frac{N-1}{2} \rfloor$ 个节点故障后继续提交日志，前提是剩余节点能够互相通信并连接客户端。从奇数规模增加到相邻的偶数规模时，这个容错上限不会提高，但提交所需的多数派规模会增加。因此，Raft 集群通常采用奇数节点，避免只增加确认范围而没有提高可容忍的故障数。节点数量确定后，还要检查各节点是否位于独立的故障域，以及多数派之间的通信延迟能否满足写入要求。

读请求也需要区分语义。裁决相关查询必须执行线性一致读，因为旧状态可能造成重复裁决或遗漏已完成战斗。仅把请求发给 Leader 还不够，Leader 在返回结果前还要确认自己仍能联系

多数派；实现中常用读索引（ReadIndex）或多数派心跳，租约读则依赖对时钟偏差的限制。排名浏览、历史记录这类对实时性要求较低的查询，可以在明确接受短暂滞后的前提下从 Follower 分流。这种分流能减轻 Leader 压力，但可能放弃最新视图，不能用于需要立即反映裁决结果的路径。

Raft 日志还需要配合持久化和压缩机制。每个节点通常以 WAL 的形式顺序追加日志，崩溃重启后先读取日志恢复状态。日志持续增长后，系统会定期生成状态机快照（Snapshot），删除快照之前的旧日志。重启或新节点加入时，只需加载最近快照，再补齐快照之后的少量日志。这类工程机制不改变 Raft 的安全性原则，但决定了集群故障恢复和新节点追赶的实际成本。

经过这一层区分，容错机制形成三层保护：**主从复制保护数据分片的在线副本，备份保护历史时间点的数据回退，Raft 保护需要强一致操作历史的权威服务**。三者分层协作，不可互替。2PC、事务性发件箱和 Raft 也分别处理不同的跨节点问题：2PC 要求一次业务操作落在多个分片上的不同写入全部提交或全部回滚；事务性发件箱负责在本地事实成立后把结果可靠传播到其他服务器；Raft 则把同一条操作日志复制到多个副本，并确定哪一段历史已经得到多数节点确认。Raft 用多数派换来强一致历史，因此每次写入都要多节点通信。这正是一致性与性能取舍的来源，也是下一小节讨论的重点。

4.4.4 CAP 定理与一致性取舍

前面的故障转移和 Raft 多数派都在处理网络分区造成的后果。网络分区发生后，仍在运行的节点会被分成无法交换信息的节点组；如果不同节点组同时收到针对同一份状态的写请求，系统应当如何响应？图 57 用节点 A、B 与节点 C 之间的通信中断建立具体场景，并给出两种处理路径。

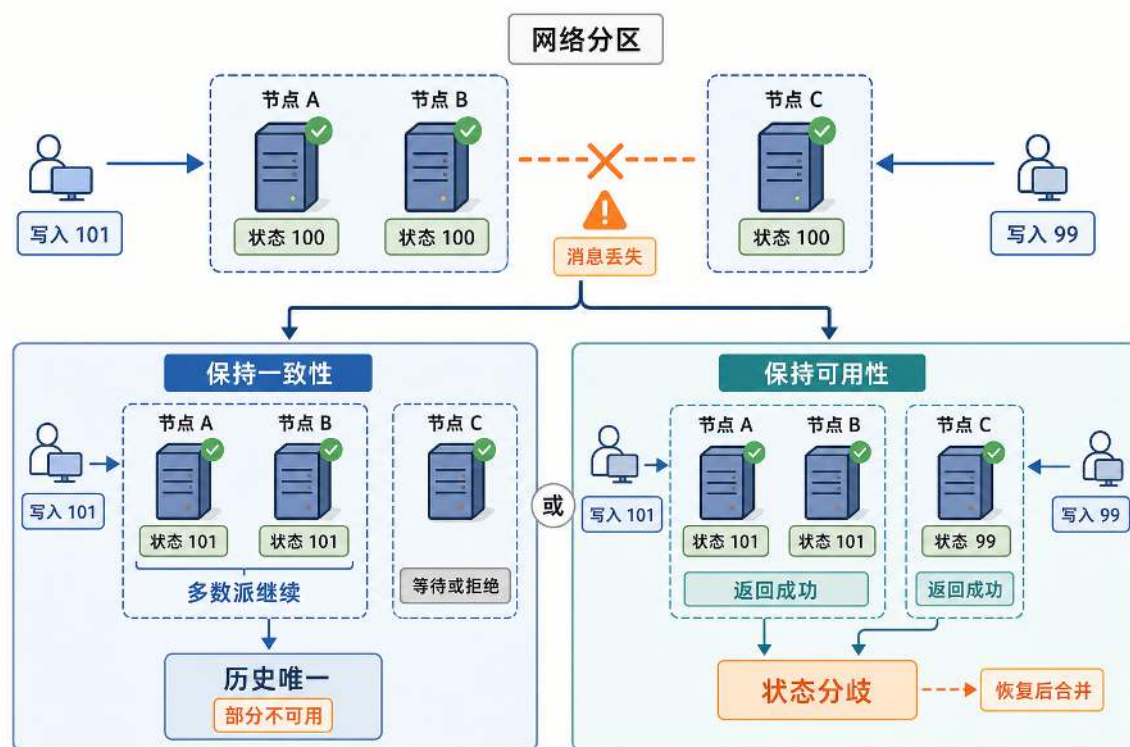


图 57: 网络分区把仍在运行的节点分成无法通信的两组: 保持一致性需要让至少一个节点组等待或拒绝请求; 保持可用性则允许两个节点组继续响应, 但必须接受暂时的状态分歧。

图的上半部分中, 节点 A、B 和 C 在分区前都保存状态 100; 分区后, 节点 A、B 所在的节点组收到写入 101, 节点 C 所在的节点组收到写入 99, 但两个节点组无法交换这两次更新。左下分支选择保持一致性: 节点 A、B 组成的多数派继续处理写入, 节点 C 则等待或拒绝请求, 因而不会把写入 99 确认为另一条成功历史; 代价是节点 C 所在的节点组暂时无法完成该请求。右下分支选择保持可用性: 两个节点组都立即返回成功, 节点 A、B 保存状态 101, 节点 C 保存状态 99; 系统继续响应了请求, 却产生了需要在网络恢复后处理的状态分歧。

这两种处理路径对应 **CAP 定理**在分区期间的核心限制。一致性 (Consistency) 要求各节点对写入顺序和最新状态给出相同的观察结果; 可用性 (Availability) 要求发送到仍在运行节点的请求能够得到非错误响应; 分区容忍性 (Partition Tolerance) 则表示系统的设计允许节点组之间的消息持续丢失或延迟, 并且仍有明确的处理策略。对跨节点保存共享状态的系统来说, 网络分区是已经发生的故障约束, 而不是日常运行时可以任意舍弃的选项。分区期间, 系统若坚持一致性, 就必须牺牲部分可用性; 若要让各节点组都继续响应, 就必须暂时放宽立即一致的要求, 而不是把三个属性任意 “选两个”。

所谓的强一致, 强调一次写入被确认后, 后续操作按照同一全局顺序观察数据, 不会在其他节点继续读到这次写入之前的旧值; 严格描述这一性质时, 通常使用线性一致性 (Linearizability)。弱一致是允许节点在一段时间内观察到不同版本的统称, 并不承诺写入立即对所有读取可见。最

终一致性是其中一种常见保证：如果不再产生新的更新，各节点经过传播后最终会收敛到相同状态。

即使网络没有分区，强一致也意味着更多通信和等待，延迟会因此上升。CAP 的延伸模型 **PACELC** 强调的正是这一点：有分区时看一致性与可用性的取舍，没有分区时仍要看一致性与延迟的取舍。²⁸

回到本章讨论过的几条路径，它们对应的是不同位置上的取舍。2PC 把多个数据库提交点绑定成一个原子提交结果，也把两轮通信、锁等待和协调者故障风险放进请求主链路。它本身解决的是共同提交或共同回滚；事务的隔离级别和客户端观察到的读写顺序，还要由参与数据库及上层协议另外保证。事务性发件箱先把业务事实收束到本地事务，再把跨服传播放到异步阶段，用幂等重试换取最终收敛。Raft 则用于保护协调服务器这类权威服务的操作历史：多数派确认之后，即使 Leader 故障，已提交的裁决记录也不会丢失。这三种机制并非简单的强弱一致性的排序，而是把其成本放在了不同位置：请求主链路、本地事务边界，或权威服务的复制日志。

架构选择的判断标准不是方案本身的一致性强度，而是每条业务链路对延迟、短暂不一致和错误的容忍程度。跨服交易里金币和物品的转移必须形成不可拆分、对外一致的结果，排行榜和一般通知可以允许几秒的延迟收敛，运营公告甚至可以接受分钟级更新。根据业务链路选择一致性强度，而不是把所有链路都拉到最强一致，是分布式系统设计的核心判断标准。

4.5 小结：均衡、协同与容错

本章讨论了应用服务器从单机走向多机后出现的核心变化：原来集中在一台机器上的对象归属、结果提交和故障恢复，被拆到多个实例和多条网络路径中。多机扩展不是简单增加服务器数量，而是要回答三类问题：对象如何分配到合适实例，多个实例怎样共同完成一次操作，节点失败后系统如何恢复到可解释的状态。

均衡解决的是对象归属和负载分摊问题。职责拆分让网关接入、业务处理、消息分发、负载监测和数据访问等功能能够分别演进；对象分片则把用户数据、计算任务和在线连接分散到多个同类实例上。分片键决定对象按什么字段归属，范围分片、哈希分片和一致性哈希决定归属如何映射到实例；分片路由、负载监测、健康实例选择、扩缩容和热迁移，则把对象归属变成运行时可调整的调度机制。

协同解决的是对象分开之后如何共同完成一次业务操作。通知类、查询类和事务类跨服交互，对一致性的要求不同，因此不能使用同一种机制处理所有链路。通知类事件要先区分尽力而为传播与可靠送达：Pub/Sub 适合在线节点的低延迟即时通知，Gossip 适合高概率扩散，需要严格送达时还必须增加持久化、确认或重放；查询类视图由各服上报异步汇总而成，在上报、网络和汇总周期均有上限时允许有界陈旧；事务类操作则或用 2PC 让多个提交点原子成立，或用事务

²⁸CAP 的形式化证明见 Seth Gilbert and Nancy Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”, ACM SIGACT News, 2002; PACELC 见 Daniel J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story”, IEEE Computer, 2012, <https://www.cs.umd.edu/~abadi/papers/abadi-pacelc.pdf>。

性发件箱把本地事务和跨服传播拆开，用幂等重试换取最终收敛。跨服事务案例说明，协调服务器的作用是给一次跨服事件提供统一裁决点，应用服务器只代理请求、执行结果和同步状态。

容错解决的是失败之后如何继续服务。主从复制维持可接管的在线副本；备份与连续归档的 WAL 保留历史状态，借助时间点恢复（PITR）把数据库停在误删、误改或程序缺陷发生之前，恢复演练则检验这份历史能否在 RPO 和 RTO 约束内真正恢复；Raft 则保护协调服务这类强一致组件的操作历史，使 Leader 故障后仍能从多数派确认的日志中恢复已提交结果。CAP 定理和 PACELC 模型提醒我们，一致性、可用性和延迟之间始终存在取舍；架构设计的重点不是把所有链路都拉到最强一致，而是根据业务能否容忍延迟、短暂不一致和错误结果来选择机制。

至此，多机架构在逻辑上已经成型：数据有分片承载，连接有负载调度分配，跨服事件有协调点裁决，关键状态有副本、备份或共识机制保护。下一步的问题不再只是“这些组件应该怎样设计”，而是“这些组件如何被稳定地交付和管理”。当网关、逻辑服、缓存、数据库分片、协调器和副本集群都需要部署、升级、扩缩容和故障替换时，手工操作会成为新的瓶颈。第五章将从这个交付问题出发，讨论容器、镜像和编排平台如何把多组件系统变成可复制、可调度、可恢复的运行单元。

4.6 思考题

1. 某游戏需要同时支持“按用户查询数据”和“全服战力排行榜”两类访问。如果用用户 ID 做哈希分片键，按用户查询会得到什么好处？排行榜查询又会遇到什么困难？请说明跨分片聚合的成本，并设计一种异步汇总排行榜的基本思路。
2. 一个一致性哈希环上原有 4 个物理节点，每个物理节点对应 150 个虚拟节点。现在新增第 5 个物理节点。请估算理想情况下大约有多少比例的键空间需要迁移，并解释虚拟节点数量为什么会影响负载均匀度和迁移波动。
3. 跨服擂台的战斗结算需要同时更新玩家 A 所在分片和玩家 B 所在分片的战绩。请分别用 2PC 和事务性发件箱两种方案描述处理流程，并比较它们在延迟、阻塞风险和一致性语义上的取舍。进一步思考：为什么事务性发件箱方案要求消费端必须做幂等处理？
4. 一个 5 节点 Raft 集群中，Leader 已经把一条日志复制到多数节点并提交，但还没来得及把新的 `commitIndex` 通知给所有 Follower 就崩溃了。请推演后续选举过程，并解释为什么“两个多数派必有交集”能够保护已提交日志不被丢失。
5. 服务器 A 正在进行一场 100 人公会战，CPU 使用率达到 95%；服务器 B 和 C 的 CPU 使用率只有 30%。如果继续使用简单轮询，新玩家仍可能被分配到 A。请设计一种基于心跳上报的动态调度策略，说明需要上报哪些指标，综合负载值如何计算，以及如何避免所有新玩家同时涌向同一台“最空闲”的服务器。
6. 假设网络分区已经发生，给出三个跨服场景：(a) 跨服战斗的命中判定；(b) 全服排

行榜的实时刷新；(c) 玩家跨服迁移时的资产交接。请分别判断它们更偏向 CP 还是 AP，并说明理由。进一步思考：哪些场景可以接受几秒钟的最终一致性，哪些场景必须在请求路径上保证强一致？

7. 一个游戏同时需要存储分片（按 PlayerID 拆数据库）、计算分片（按 MapID 分配战斗服务）和连接分片（按区域分配网关）。请为每种分片设计路由查找的基本流程，并说明三类分片的分片键为什么不同。
8. 主从复制、备份恢复和 Raft 分别保护什么对象？如果主库故障且最新从库的复制进度落后 5 秒，故障切换后这 5 秒内的写入会怎样？与基础备份加 WAL 连续归档的时间点恢复相比，故障切换的优势和局限分别是什么？
9. Raft 通过复制日志，使多个副本按照相同顺序执行相同命令；2PC 则协调多个独立数据库的本地事务共同提交或回滚。为什么跨分片交易不能简单地把操作写进一个 Raft 日志，就认为它已经替代了 2PC？请从参与对象、提交范围和故障后的恢复责任三个方面比较二者。